Machine Learning Foundations

Concise revision notes across machine learning foundations

Mathematical foundations • Representations • Objectives
Architectures • Optimisation • Stability
Evaluation • Adaptation • No Free Lunch
Decision Trees • Random Forests • Gradient Boosting
CNNs • Transformers • LLMs • Inference • ML Systems

September 2026

Contents

Initial Commentary v
- Goals . . . v
- Philosophy . . . v
- Assumed Understanding . . . vi
- How to Read This . . . vi
- Types of Callout . . . vi
- Formatting Guide . . . vii
- AI Editing Rules . . . vii
- Tone . . . vii

1 Mathematical and Statistical Foundations 1
- 1.1 Linear maps, directions and information . . . 1
  - 1.1.1 Independence, span, rank and null space . . . 1
- 1.2 Eigenstructure and low-rank structure . . . 1
  - 1.2.1 Eigenvectors, eigenvalues and PSD structure . . . 1
  - 1.2.2 SVD as the rectangular low-rank view . . . 2
- 1.3 Probability, uncertainty and estimation . . . 2
  - 1.3.1 Expectation, variance and covariance . . . 2
  - 1.3.2 Bayes, likelihood and priors . . . 2
  - 1.3.3 Entropy and distribution mismatch . . . 3
- 1.4 Local derivatives, curvature and confidence . . . 3
  - 1.4.1 Jacobian: local input-to-output sensitivity . . . 3
  - 1.4.2 Hessian: local curvature of a scalar objective . . . 3
  - 1.4.3 Convexity . . . 3
  - 1.4.4 Confidence intervals and standard error . . . 4

2 Data Representation and Preprocessing 5
- 2.1 The curse of dimensionality . . . 5
- 2.2 Scaling, standardisation and leakage . . . 5
- 2.3 Dimensionality reduction: selection versus transformation . . . 5
- 2.4 PCA: an eigenvector view . . . 6
- 2.5 Whitening: PCA and ZCA . . . 6
- 2.6 Whitening versus normalisation . . . 6
- 2.7 t-SNE, UMAP and latent visualisation . . . 7
- 2.8 Embeddings as operational geometry . . . 7

3 Learning Paradigms and Objectives 9
- 3.1 Supervised, unsupervised, self-supervised and reinforcement learning . . . 9
- 3.2 Supervised losses for continuous targets . . . 9
- 3.3 Classification losses and logits . . . 9
- 3.4 Ranking and preference objectives . . . 10
- 3.5 Representation-learning objectives . . . 10
- 3.6 Variational and generative objectives . . . 11
- 3.7 Reinforcement-learning objectives . . . 11
- 3.8 What different objectives favour . . . 11

4 Models, Architectures and Their Assumptions 12
- 4.1 Linear models: a transparent baseline . . . 12
- 4.2 KNN and similarity-based learning . . . 12
- 4.3 Support vector machines and kernels . . . 12
- 4.4 Decision trees, bagging and boosting . . . 12
- 4.5 Neural networks and nonlinear activations . . . 13
- 4.6 CNNs: local structure and weight sharing . . . 13
- 4.7 RNNs: sequential state . . . 14
  - 4.7.1 LSTM: explicit cell memory and three gates . . . 14

4.7.2 GRU: a compact gated recurrent unit . . . 14
4.8 Transformers: content-dependent interaction . . . 14
4.9 State-space models, S4 and Mamba . . . 15
4.10 Autoencoders and generative model families . . . 15
4.11 Gumbel–Softmax: differentiable categorical choices . . . 15
4.12 Model assumptions and inductive biases . . . 15

5 Optimisation and Training Mechanics 17
5.1 Gradient descent — follow the local slope . . . 17
5.2 Full-batch, stochastic and mini-batch gradients . . . 17
5.3 Backpropagation — calculate derivatives, do not choose updates . . . 18
5.4 Momentum — remember persistent directions . . . 18
5.5 AdaGrad and RMSProp — adapt step size by gradient scale . . . 18
5.6 Adam — momentum plus adaptive scaling . . . 19
5.7 AdamW — decouple weight decay from adaptive scaling . . . 19
5.8 How optimiser ideas evolved: a branching genealogy . . . 19
5.9 Newton's method and curvature-aware updates . . . 20
5.10 Learning-rate schedules and warmup . . . 20
5.11 Gradient accumulation and mixed precision . . . 20
5.12 The full training loop . . . 21

6 Training Stability and Regularisation 23
6.1 Weight initialisation — start in a trainable region . . . 23
6.2 Xavier / Glorot and He / Kaiming initialisation . . . 23
6.3 Normalisation — control intermediate scale . . . 23
6.3.1 Batch Normalisation . . . 24
6.3.2 Layer Normalisation . . . 24
6.3.3 RMSNorm . . . 24
6.4 Regularisation — L1 and L2 . . . 25
6.5 Dropout and stochastic regularisation . . . 25
6.6 Data augmentation and symmetry assumptions . . . 25
6.7 Early stopping — stop before fitting validation-specific noise . . . 25
6.8 Residual connections: an additional stability lever . . . 25

7 Evaluation, Validation and Model Selection 27
7.1 Classification evaluation — the confusion matrix . . . 27
7.2 Precision, recall and F1 . . . 27
7.3 Decision thresholds . . . 27
7.4 ROC curves, AUC-ROC and precision–recall . . . 28
7.5 Regression metrics . . . 28
7.6 Cross-validation and fold choice . . . 28
7.7 Overfitting, underfitting and the bias–variance tradeoff . . . 29
7.8 Model selection and data leakage . . . 30
7.9 Retrieval and ranking metrics . . . 30
7.10 Generative and RAG evaluation . . . 31
7.11 Online experiments and A/B testing . . . 32

8 Transfer Learning, Domain Adaptation and Few-/Zero-Shot Learning 33
8.1 Transfer learning — reuse what the model has learned . . . 33
8.2 Domain adaptation — the data distribution changes . . . 33
8.3 Few-shot learning — learn from very few examples . . . 34
8.4 Meta-learning and MAML . . . 35
8.5 Zero-shot learning — no task-specific labelled examples . . . 35
8.6 LoRA: a parameter-efficient adaptation mechanism . . . 35
8.6.1 QLoRA: quantised base plus low-rank adaptation . . . 35
8.7 Simulation-to-real adaptation for scientific ML . . . 36

9 No Free Lunch and the Limits of Learning 37
9.1 Why finite observations are not enough . . . 37
9.2 A mathematical formulation of No Free Lunch . . . 37
9.3 Two rules, two compatible worlds . . . 37

9.4 Inductive bias — the assumptions that make learning possible . . . 38
9.5 Bayesian priors: a direct form of inductive bias . . . 38
9.6 Occam's razor is also a bias . . . 38
9.7 Why statistical learning guarantees do not contradict NFL . . . 38
9.8 Optimisation NFL is related but separate . . . 38
9.9 Connection to scientific machine learning . . . 39

10 Tree-Based Models and Ensembles 40
10.1 A single decision tree: rules become regions . . . 40
10.1.1 Learning a split . . . 41
10.2 Tree structure, greedy search and regularisation . . . 41
10.2.1 Greedy splits need not find the best tree . . . 41
10.2.2 Depth, leaves and pruning . . . 41
10.2.3 Growth strategy and split geometry . . . 42
10.3 Bagging, Random Forests and ExtraTrees . . . 42
10.4 Boosting: learning successive corrections . . . 43
10.4.1 Gradient boosting as functional optimisation . . . 43
10.4.2 Ranking-aware boosting: LambdaMART . . . 44
10.4.3 Curvature and second-order updates . . . 45
10.4.4 Learning rate, depth and early stopping . . . 45
10.5 XGBoost, LightGBM and CatBoost: what changed? . . . 45
10.5.1 XGBoost: explicit regularisation and efficient optimisation . . . 46
10.5.2 Histogram split search . . . 46
10.5.3 LightGBM: selective use of computation . . . 46
10.5.4 CatBoost: ordered categories and symmetric trees . . . 47
10.6 Research optionality: what can be changed? . . . 47
10.7 Inductive bias and model selection . . . 48
10.7.1 Extrapolation and feature transformations . . . 48
10.8 Evaluation, explanation and failure modes . . . 49
10.8.1 Evaluate the deployment task, not a random row split . . . 49
10.8.2 Metrics, importance and causal interpretation . . . 49
10.9 Extensions and a complete research example . . . 50
10.9.1 Beyond standard trees . . . 50
10.9.2 Worked design: failure forecasting on repeated measurements . . . 50
10.10Further reading . . . 50

11 Architecture: CNNs 52
11.1 Convolution as a reusable local detector . . . 52
11.2 Receptive field, resolution and depth . . . 52
11.3 Efficient convolution: separate spatial filtering from channel mixing . . . 53
11.4 Classification, detection and cheap classical stages . . . 53
11.5 Video: use time when single frames are ambiguous . . . 53
11.6 Pretrained visual representations: CLIP-like versus DINO-like . . . 54
11.7 How vision systems evolved . . . 54

12 Architecture: Transformers 55
12.1 One picture first: communication then computation . . . 55
12.2 What actually flows through the network? . . . 55
12.3 Attention before Q/K/V: choose information routes . . . 55
12.4 Queries, keys and values . . . 57
12.5 Causal masking: delete arrows into the future . . . 57
12.6 Multi-head attention: several routing spaces . . . 57
12.7 Residual stream and normalisation . . . 57
12.8 Position: content alone is not order . . . 58
12.9 Why long context is expensive . . . 58
12.10Research evolution: how should sequence information move? . . . 59

13 Architecture: LLMs 60
13.1 Tokenisation and autoregressive pretraining . . . 60
13.2 Generation: logits, probabilities and the autoregressive loop . . . 60
13.3 Supervised fine-tuning: teach interaction behaviour . . . 60

13.4 Preference optimisation: ranking generated responses . . . 61
13.5 Prompting, RAG and fine-tuning solve different missing pieces . . . 61
13.6 RAG: external evidence as an inference-time memory . . . 61
13.6.1 Chunking: choose the unit that retrieval can retrieve . . . 61
13.6.2 Bi-encoder retrieval versus cross-encoder reranking . . . 62
13.6.3 Approximate nearest neighbours: HNSW versus IVF . . . 62
13.7 Agentic systems: an LLM inside a stateful control loop . . . 63
13.8 LLM research optionality . . . 63

14 Inference and Model Compression **64**
14.1 Quantisation: spend fewer bits on tolerant values . . . 64
14.1.1 PTQ versus QAT . . . 64
14.2 Knowledge distillation: learn from a teacher distribution . . . 65
14.3 Cascades: reserve expensive models for hard cases . . . 65
14.4 LLM inference: prefill versus decode . . . 66
14.5 KV cache: save repeated attention work . . . 66
14.6 Batching and continuous batching . . . 66
14.7 Inference evolution and design axes . . . 67

15 ML Systems and Production **68**
15.1 The full production loop . . . 68
15.2 Batch versus streaming . . . 68
15.3 Training-serving consistency . . . 68
15.4 Model registry, deployment and rollback . . . 69
15.5 Monitoring as a stack . . . 69
15.6 Drift and retraining . . . 69
15.7 Queues, caching and fallbacks . . . 69
15.8 Build versus buy . . . 70
15.9 Training versus lifetime inference economics . . . 70

16 Use Case: Real-Time Moderation **71**
16.1 The moderation system is a cascade . . . 71
16.2 Text and image representations . . . 71
16.3 Class imbalance: accuracy can be meaningless . . . 71
16.3.1 Class weighting versus focal loss . . . 71
16.4 Thresholds encode the product cost . . . 72
16.5 Human review is an expensive model tier . . . 72
16.6 Degrees of experiment . . . 72

17 Use Case: Ranking Systems **73**
17.1 The ranking funnel . . . 73
17.2 Retrieval with two towers . . . 73
17.3 Ranking and reranking . . . 74
17.4 Why NDCG is top-heavy . . . 74
17.5 Position bias: clicks mix relevance with exposure . . . 75
17.6 Inverse propensity scoring . . . 75
17.7 Evaluation: diagnose the funnel stage . . . 75
17.8 Degrees of experiment . . . 75

18 Use Case: Personalisation **77**
18.1 From interaction matrix to latent preference space . . . 77
18.2 Two towers generalise stored latent vectors . . . 77
18.3 Cold start: combine collaborative and content evidence . . . 77
18.4 Long-term preference versus session intent . . . 78
18.5 Temporal decay: encode how quickly preference becomes stale . . . 78
18.6 The recommendation feedback loop . . . 78
18.7 Popularity bias and self-reinforcement . . . 79
18.8 Exploration versus exploitation . . . 79
18.9 Fast state updates versus slow parameter updates . . . 79
18.10 Degrees of experiment . . . 79

Initial Commentary

Goals

This book is not aimed at a student, graduate, or academic learning machine learning from scratch. It is a revision tool intended to give a high-level understanding of the wide research area we call ML. Being targeted at senior or expert-level practitioners looking for a refresher, update, or review, this book aims to:

- represent the topics mental model quickly
- connect topics to the assumptions, consequences, and neighbouring ideas
- expose the design choices and research degrees of freedom around a method

Philosophy

This book follows Henry's learning policies:

- **Allow the reader their own language.** Learning is done by understanding, and we all understand in different ways. Leaving naming to the reader gives the reader maximum space to use their own definitions. It also means that we can rely on the maths talking for us, and we do not need long "where X is Y and A is B" text surrounding maths. For this reason, favour descriptive mathematics, or even "plain English equations". Some examples:
  - Instead of using the Jacobian, J, in an expression, use its base definition $\left(\frac{\partial y}{\partial x}\right)$ where that is clearer. If the fact that it is a Jacobian matters, an underbrace can name the term without introducing another symbol.
  - Instead of assigning counts to arbitrary letters, use descriptive subscripts such as N_{experts} or N_{trees}.
  - The term $\lambda_{\text{e-val(A)}}$ is clearly representing the eigenvalue of matrix A without wasting text space.
- **Short and sweet.** Some textbooks spell out everything for the reader, making reading passive. Jumping a few logical steps and naming a conclusion rather than reproducing a full derivation makes reading active: the reader fills in the small gaps in their own terms.
- **Minimal mathematics, maximum intuition.** The reader should not need to wade through definitions and mathematical fuckery to understand a topic. After one read they should be able to discuss and reason around a topic.
- **Minimal jargon.** Some technical vocabulary is necessary, but terminology should earn its place. Prefer descriptive plain-English wording when a specialist label does not add useful precision. Henry believes that humans naturally obscure their discoveries to nurse their own ego, and impress others - avoid this.
- **Memory Hook.** A Memory Hook is a short sentence that encodes information and acts as a retrieval cue for a much larger mental model. People struggle to memorise large amounts of text and maths, but often remember a compact story or rule quickly. Therefore, explain the story as concisely as possible and summarise it with a Memory Hook. Some synonyms I have heard them described as: micro-heuristics, concept capsules, retrieval rules, compressed principles, rules of thumb, Compressed Heuristics, Knowledge in One Breath, High-Leverage Facts, or simply Memory Hooks. They may be further split into categories:
  - **Mechanisms**. e.g: *Vanishing gradients make early layers slow learners*, e.g: *More parameters reduce bias; regularisation restrains variance.*
  - **Actions**. e.g: *With ReLU activations, start with He/Kaiming initialisation to handle the negatives.*
  - **Special Cases**. e.g: *L1 Regularisation is basically feature selection.*
  - **Differentiator**. e.g: *AdamW seperates weight decay and scaling; adding an L2 penalty inside Adam does not produce the same update.*

  Once memorised, one should be able to rebuild a mental model of each sub-topic from a single sentence and reason outward from that sentence.

Assumed Understanding

Because this is aimed at revision rather than first-time teaching, we do not need to dive into easy material. We assume some knowledge of linear algebra (for example, what orthogonal means), basic maths (differentiation, the chain rule, probability), computer science (for example, what a bit is), physics-style reasoning (for example, entropy), and basic world knowledge (for example, what an image is). If a prerequisite is only needed to understand one consequence, remind the reader of the minimum required fact rather than reteaching the whole prerequisite topic.

How to Read This

This textbook is not a "teach me ML" book with full derivations and exhaustive depth. It is a high-level overview of key points, with a focus on links, consequences, design choices, and practical usage. A few examples:

1. We teach how Adam interacts with L2 regularisation and why AdamW changes the update; we do not derive convergence boundaries.
2. We describe the evolution of tree architectures as an accumulation of methods and ideas toward modern systems; we do not provide an end-to-end derivation of every incremental algorithm.
3. We explain what a design choice changes, which assumptions make it sensible, and what failure mode it addresses; we do not catalogue every historical variant simply because it exists.

A useful reading pattern is: **mechanism** → **consequence** → **trade-off** → **practical choice**. Equations, figures, timelines, and callouts exist to compress that chain, not to interrupt it.

Types of Callout

Callouts are used strictly for these rules, but are often memory hooks.

Example

A concrete worked or real-world instance of the idea just described. For example: *if two classifiers have the same accuracy but one misses nearly all rare positive cases, class imbalance has made accuracy hide the behaviour we actually care about.*

Practical Tip

A directly useful action, implementation choice, or debugging rule. For example: *With ReLU activations, start with He/Kaiming initialisation.*

Memorise

A fact or equation that is worth recalling accurately rather than reconstructing each time. For example:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Intuition

A compact way to think about a mechanism without pretending the analogy is the full definition.
L1 regularisation is basically feature selection on weight abs-size.

Inductive Bias

State the assumption a method builds in, what that assumption buys us, and when it can become a restriction. Use the form: *if <assumption/data/method>, then <consequence>.*
For example: *if nearby pixels are more related than distant pixels, convolution's locality bias is useful; if feature order is arbitrary, that same bias may be inappropriate.*

Special Case

A useful reduction where a broader idea becomes a familiar one under a particular setting. For example: *the F1 score is the special case of the Fβ score with* $\beta = 1$.

Memory Hook

One sentence that should let the reader reconstruct a larger mental model. It may encode a mechanism, action, special case, or differentiator. For example: *Vanishing gradients make early layers slow learners.*

Degree of Experiment

A dimension along which researchers or experimenters can change a system while keeping the broader method recognisable. For example, for normalisation one can vary *which axes define the statistics, where normalisation is inserted, whether the statistics depend on the batch, and whether learned scale/shift parameters are used.*

Formatting and AI Guide

AI Editing Rules

AI edits should behave like careful maintenance of one coherent textbook, not like a fresh author rewriting the surrounding material.

1. **Respect the Initial Commentary.** Conduct work with Goals, philosophy and formatting in context.
2. **Respect edit scope.** Change only the material requested. Do not silently rewrite other sections.
3. **Integrate; do not append.** New work belongs at the point in the existing text where the concept is most relevant. Do not create a trailing "questions", "additional notes", or "follow-ups" section unless explicitly requested. Add to sections, tables, reference it, etc.
4. **Use the callout taxonomy literally.** Example = concrete instance; Practical Tip = action; Memorise = exact recall; Intuition = mental model; Inductive Bias = assumption → consequence; Special Case = useful reduction; Memory Hook = compressed retrieval cue; Degree of Experiment = research/design axis. Do not create callout categories.
5. **Cross-reference before duplicating.** If the prerequisite idea is already taught elsewhere, link to it and provide only the minimum reminder needed locally.
6. **Use visuals for structure.** All visuals need visual reviewing, never 'tkviz and forget'.
7. **Preserve document mechanics.** Keep labels, references, equations, figures, terminology, and LaTeX conventions consistent with the existing book. After edits, compile and visually verify the affected pages rather than assuming valid source means valid layout.

Tone

This is informal for a textbook, but it should remain clear and technically precise. For example: "`L1 regularisation is basically feature selection on abs(weights)`". Here, `abs(weights)` is deliberate shorthand for the more long-winded "absolute size of the weights". Shorthand is welcome when it compresses an idea without changing its meaning.

Prose and hierarchy

- New latex line per sentence (not a new compiled/pdf line!)
- Prefer short paragraphs. A paragraph should normally carry one conceptual step: mechanism, consequence, contrast, or caveat.
- Use **bold** for the local concept that the sentence is about, *italics* for compact rules or deliberately memorable phrasing, and `monospace` for code-like shorthand such as `abs(weights)`.
- Introduce jargon only when the term will be useful later. If the plain-English phrase is clearer, use it first and put the formal name next to it in bold.

Mathematics

- Use equations when the mathematical relationship itself is the memory hook, when notation is more compact than prose, or when a downstream consequence depends on the form of the equation.
- Prefer descriptive notation and subscripts over arbitrary letters. Avoid introducing a symbol merely to save one line of typing.
- Do not include a derivation by default. Show the minimal step needed to expose the mechanism or consequence.
- When a particular equation is genuinely recall-worthy, put it in a **Memorise** callout. Most equations should remain ordinary body content.
- Explain non-obvious terms next to the equation, but do not spell out standard notation the assumed reader already knows.
- Generally, condense an idea down to one equation through labelling and substitution. For example, the Adam equation is often expressed as

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

  with some definitions, but we prefer a 'whole story' equation:

$$\theta_{t+1} = \theta_t - \eta \frac{\overbrace{\beta_1 m_{t-1} + (1-\beta_1) g_t}^{=m_t, \text{momentum}}}{\sqrt{\underbrace{\beta_2 v_{t-1} + (1-\beta_2) g_t^2}_{=v_t, \text{scale}}} + \epsilon}$$

- Some non-standard notation, which we standardise here:
  - Model Weights w should be represented as parameter weights: θ_{weights}.
  - Any count is N with an english subscript.
  - Indexing a matrix if with subscripts like A_{ij} and selecting a matrix subset is pythonic like $A_{[:i]}$ for the first i columns of A.

Research evolution

When methods evolved through a recognisable sequence show the evolution as an accumulation of problems/-fixes rather than as isolated definitions. A compact table-timeline is preferred when it makes the sequence clear. The goal is not historical completeness. Include the steps that change the reader's mental model of the modern method.

Step	What changed	Why it was added	Consequence / new trade-off
Baseline	Starting mechanism	What it cannot do well	The limitation that motivates the next step
Next method	One important addition	Which limitation it targets	What improves, plus what becomes more complex or biased
Modern variant	Further separation or control	Which earlier coupling/failure it fixes	Current practical behaviour and remaining choices

Callout discipline

A useful test is whether the boxed text can be removed without breaking the main explanation: the body should still teach the topic, while the callout should provide a separable retrieval cue, action, example, or qualification. Avoid stacking several callouts in a row in ordinary chapters.

Formatting Example

The following miniature section demonstrates the intended rhythm: short mechanism, minimal maths, then a small number of purpose-specific callouts.

L1 regularisation. Add a penalty proportional to the absolute size of the parameters:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_i |w_i|.$$

The penalty has a constant-magnitude pull toward zero away from zero, so weakly useful parameters are more likely to be driven exactly to zero than under an L2 penalty. The practical consequence is sparse parameter vectors and, in models where weights map directly to input features, an effect similar to feature selection.

Intuition

L1 is basically feature selection on weight abs-size.

Example

In a linear model with thousands of candidate input features, increasing λ can drive many feature weights to exactly zero, leaving a smaller active subset.

Degree of Experiment

The research/design knobs include the penalty strength λ, which parameter groups are penalised, whether sparsity is applied to individual weights or structured groups, and whether sparsity is imposed during training or by a later pruning step.

This is the intended level of depth: enough mathematics to expose the mechanism, enough consequence to make the choice useful, and no derivation that does not change the mental model.

SECTION 1

Mathematical and Statistical Foundations

1.1 Linear maps, directions and information

For ML, the useful geometric question is usually not "what is a vector?" but "which directions contain distinct information, and what does a transformation preserve or destroy?" For nonzero vectors, the dot product separates magnitude from directional alignment:

$$\underbrace{x^\top y}_{\text{magnitude + alignment}} = \|x\|\,\|y\| \underbrace{\cos\phi}_{\text{directional alignment}}, \qquad x \perp y \iff x^\top y = 0.$$

Cosine similarity removes magnitude; the raw dot product does not. This distinction matters in embeddings and similarity search when vector norm may or may not carry task information.

1.1.1 Independence, span, rank and null space

Linear independence means a direction cannot be reconstructed from the others; the **span** is everything their combinations can represent; **rank** counts independent surviving directions. For a linear map $y = Ax$, rank–nullity gives the information-capacity picture directly:

$$\underbrace{N_{\text{input dimensions}}}_{\text{available directions}} = \underbrace{\operatorname{rank}(A)}_{\text{directions that survive}} + \underbrace{\dim\ker(A)}_{\text{directions destroyed}}. \tag{1.1}$$

The image/column space is what the map can output; the null space contains input perturbations that become invisible. Redundant features and low-rank parameterisations are both statements about how many independent directions are actually needed. This is the prerequisite for PCA in Section 2.4 and LoRA in Section 8.6.

Memory Hook

Rank counts independent directions (columns of matrix); the null space counts directions the map deletes.

An inverse exists only when a square linear map loses no direction and covers its output space. In numerical ML code, solve a linear system or use a factorisation rather than explicitly forming A^{-1} unless the inverse itself is required.

1.2 Eigenstructure and low-rank structure

1.2.1 Eigenvectors, eigenvalues and PSD structure

An eigenvector is a direction a linear map does not rotate away from itself:

$$A \underbrace{v}_{\text{preserved direction}} = \underbrace{\lambda_{\text{along } v}}_{\text{scale on that direction}} v.$$

The same mental model appears in covariance/PCA, recurrent dynamics and curvature: eigenvectors identify special directions; eigenvalues say how strongly the operator acts along them. PCA uses covariance eigenstructure explicitly in Section 2.4; the optimisation consequence reappears with Hessian curvature in Section 5.9.

A symmetric matrix is positive semidefinite (PSD) when

$$\underbrace{x^\top A x}_{\text{quadratic response along } x} \geq 0 \qquad \text{for every } x.$$

For symmetric A, this is equivalent to nonnegative eigenvalues. Covariance matrices are PSD.

Memory Hook

A PSD Hessian means convex space.

1.2.2 SVD as the rectangular low-rank view

For a rectangular matrix, singular value decomposition exposes the same directional story without requiring a square matrix:

$$A = \underbrace{U}_{\text{output directions}} \quad \underbrace{\Sigma}_{\text{direction-wise scales}} \quad \underbrace{V^\top}_{\text{input directions}} .$$

Large singular values identify directions the map transmits strongly; truncating small ones gives a low-rank approximation. That is the common structural idea behind compression, PCA implementations and parameter-efficient low-rank updates such as LoRA (Section 8.6).

1.3 Probability, uncertainty and estimation

For a discrete random variable, a PMF assigns probability mass and sums to one; for a continuous variable, a PDF integrates to one and probabilities are areas under the density. Conditional and joint distributions connect through $p(x, y) = p(y \mid x)p(x)$; marginalisation removes variables by summing or integrating them out.

1.3.1 Expectation, variance and covariance

Expectation is the distribution-weighted average. Variance and covariance capture spread and linear co-variation:

$$\operatorname{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] , \tag{1.2}$$

$$\operatorname{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] , \tag{1.3}$$

$$\rho_{XY} = \frac{\operatorname{Cov}(X, Y)}{\sqrt{\operatorname{Var}(X)\operatorname{Var}(Y)}}. \tag{1.4}$$

Zero covariance is weaker than independence; independence implies zero covariance when the relevant moments exist. For vector-valued data, the covariance matrix $\Sigma = \mathbb{E}[(x - \mathbb{E}[x])(x - \mathbb{E}[x])^\top]$ packages pairwise linear co-variation and feeds directly into PCA/whitening in Sections 2.4 and 2.5.

1.3.2 Bayes, likelihood and priors

Memorise

$$\boxed{P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}}.$$

Bayes' rule says a posterior combines prior belief with how compatible the observation is under each hypothesis. For rare events, the prior/base rate can dominate the interpretation of a positive test; this is the probability-side version of precision in Section 7.2.

Example

If prevalence is 1%, sensitivity is 95%, and the false-positive rate is 5%, then $P(\text{fault} \mid +) \approx 16\%$: a sensitive detector can still produce mostly false alarms when the event is rare.

For N_{examples} conditionally independent observations, maximum likelihood chooses parameters that maximise summed log-likelihood:

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \sum_{i=1}^{N_{\text{examples}}} \log p(x_i \mid \theta).$$

MAP adds a prior preference directly to the same objective:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \left[\underbrace{\log p(D \mid \theta)}_{\text{fit to observations}} + \underbrace{\log p(\theta)}_{\text{parameter preference}} \right].$$

Gaussian-like priors produce L2-style penalties; Laplace-like priors produce L1-style penalties, linking Bayesian preference to regularisation in Section 6.4 and inductive bias in Section 9.5.

1.3.3 Entropy and distribution mismatch

Memorise

$$\boxed{H(X) = -\sum_{x} p(x) \log p(x)}.$$

Entropy measures uncertainty in one distribution. Cross-entropy measures coding/surprise when observations follow p but predictions use q; KL is the extra cost relative to using p itself:

$$\underbrace{H(p, q)}_{\text{cross-entropy}} = \underbrace{H(p)}_{\text{irreducible uncertainty}} + \underbrace{D_{\text{KL}}(p \| q)}_{\text{mismatch penalty}}.$$

KL is asymmetric and can be infinite when q assigns zero probability where p does not. Jensen–Shannon divergence symmetrises this comparison through the mixture $(p + q)/2$ and is bounded. Cross-entropy becomes a classification objective in Section 3.3; entropy reduction also drives common tree-split criteria in Section 10.1.1.

1.4 Local derivatives, curvature and confidence

1.4.1 Jacobian: local input-to-output sensitivity

For a vector-valued map $f : x \mapsto y$, the local first-order response is

$$\Delta y \approx \underbrace{\frac{\partial y}{\partial x}}_{\text{Jacobian: local sensitivity map}} \Delta x.$$

The Jacobian tells which input directions are amplified, suppressed or locally invisible; its products are the algebra underneath backpropagation and recurrent gradient propagation. See Sections 5.3 and 6.1 for those consequences rather than reteaching them here.

Memory Hook

The Jacobian maps a small input perturbation to its first-order output perturbation.

1.4.2 Hessian: local curvature of a scalar objective

For a scalar loss, the second derivative matrix controls the local quadratic correction:

$$\mathcal{L}(\theta + \Delta) \approx \mathcal{L}(\theta) + \nabla_\theta \mathcal{L}(\theta)^\top \Delta + \frac{1}{2} \Delta^\top \underbrace{\nabla^2_\theta \mathcal{L}(\theta)}_{\text{Hessian: local curvature}} \Delta.$$

Large positive Hessian eigenvalues mean sharp positive curvature; small magnitudes mean flat directions; negative eigenvalues identify locally downhill curvature directions around a saddle/non-minimum. Newton and curvature-aware methods use this structure in Section 5.9.

1.4.3 Convexity

A convex objective has no suboptimal local minima: averaging two parameter points cannot raise the function above the corresponding average of their objective values. For twice-differentiable objectives on a convex domain, a PSD Hessian everywhere is the useful local test. Deep-network losses are generally nonconvex, so convex guarantees do not transfer automatically.

1.4.4 Confidence intervals and standard error

For $N_{\text{independent observations}}$ with sample standard deviation s, the mean estimator has standard error

$$\mathrm{SE}(\hat{\mu}) = \frac{s}{\sqrt{N_{\text{independent observations}}}}.$$

A confidence interval is a sampling-procedure statement, not a posterior probability that the fixed parameter lies in the realised interval. Use a Student-t critical value for normally distributed observations with unknown variance and small samples; use resampling or a model matched to the dependence structure when independence is not credible. Evaluation-specific consequences are revisited in Sections 7.6 and 7.8.

SECTION 2

Data Representation and Preprocessing

2.1 The curse of dimensionality

An input with N_{features} coordinates does not automatically contain N_{features} independent useful directions. As dimension grows, fixed datasets occupy a smaller fraction of the available space, local neighbourhoods can become less informative, and flexible models often need more data. The severity depends on intrinsic structure, feature scaling and the data distribution rather than dimension alone. Distance-based methods expose this strongly; see KNN in Section 4.2.

A simple grid picture makes the scaling visible: $N_{\text{points per axis}}^{N_{\text{features}}}$ samples are needed to fill every grid location.

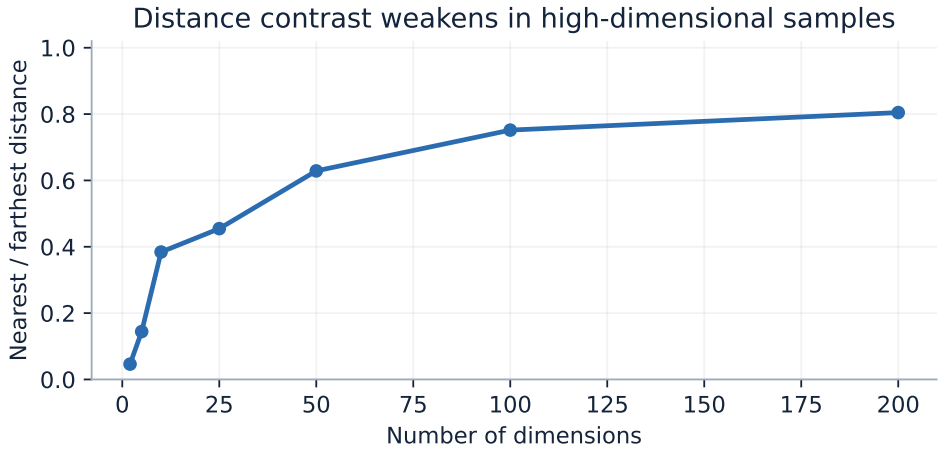


Figure 2.1: For one synthetic sample, nearest and farthest distances become less distinguishable as dimension increases; this illustrates distance concentration rather than asserting a universal law.

2.2 Scaling, standardisation and leakage

Feature standardisation is the coordinate-wise transform

$$z_j = \frac{x_j - \mu_j}{\sigma_j}.$$

It equalises marginal scale but does not remove correlations. Alternatives include min–max scaling, robust median/quantile scaling, or physically meaningful unit conversion.

Using all observations to estimate preprocessing statistics leaks validation/test information even when labels are absent. The corresponding split discipline is in Sections 7.6 and 7.8.

2.3 Dimensionality reduction: selection versus transformation

Feature selection keeps original variables; **feature extraction** creates new coordinates. Selection usually preserves interpretation; extraction can compress correlated structure more efficiently. L1 regularisation gives a model-based route to sparse selection in settings where individual parameter weights map meaningfully to input features (Section 6.4).

Method	What it preserves	Main caution
Filter / wrapper selection	Original variables chosen by a criterion	Selection must occur inside CV
PCA	High-variance linear directions	Variance need not predict labels
Autoencoder	Information useful for reconstruction	Reconstruction need not preserve task signal
t-SNE / UMAP	Local neighbourhood structure for visualisation	Global geometry can be distorted

2.4 PCA: an eigenvector view

For centred data, PCA diagonalises the covariance matrix and keeps the largest-variance eigen-directions:

$$\underbrace{\Sigma}_{\text{feature covariance}} = \underbrace{Q}_{\text{variance directions}} \underbrace{\operatorname{diag}(\lambda_1, \ldots, \lambda_{N_{\text{features}}})}_{\text{variance scales}} Q^\top. \tag{2.1}$$

The eigenstructure reminder is in Section 1.2.1. Keeping the $N_{\text{components}}$ largest-variance directions gives

$$z = Q_{[:N_{\text{components}}]}^\top (x - \mu).$$

Under squared reconstruction error, this is the optimal linear subspace of that rank for the observed data. Because PCA ignores labels, low-variance predictive signal can be removed.

2.5 Whitening: PCA and ZCA

Whitening aims for transformed covariance near the identity. Using the PCA eigensystem, two common transforms are

$$\underbrace{z_{\text{PCAwhite}}}_{\text{whitened in PCA coordinates}} = \underbrace{\operatorname{diag}\Big((\lambda_i + \epsilon)^{-1/2}\Big)}_{\text{equalise variance by direction}} Q^\top (x - \mu), \tag{2.2}$$

$$\underbrace{z_{\text{ZCA}}}_{\text{whitened near original orientation}} = Q \operatorname{diag}\Big((\lambda_i + \epsilon)^{-1/2}\Big) Q^\top (x - \mu). \tag{2.3}$$

The stabiliser ϵ prevents tiny eigenvalues from exploding noise; exact identity covariance is recovered only in the ideal unstabilised full-rank case. PCA whitening ends in eigenvector coordinates; ZCA rotates back toward the original coordinate orientation.

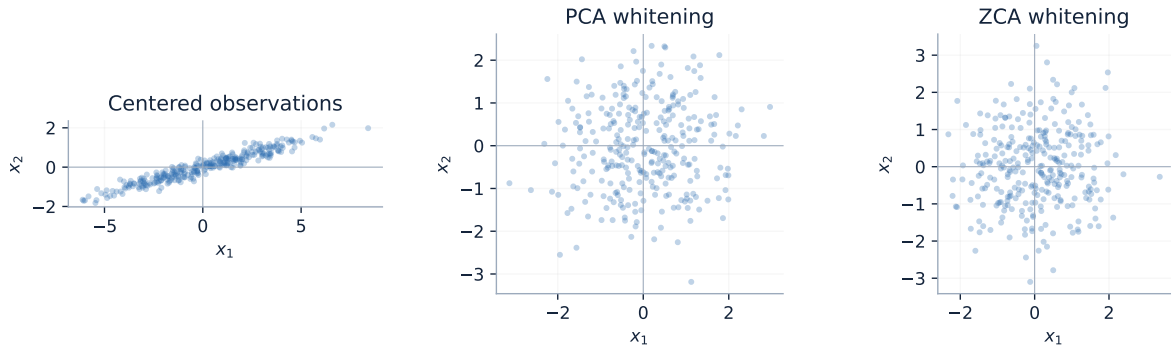


Figure 2.2: Synthetic correlated data after centring, PCA whitening and ZCA whitening; both decorrelate/rescale, but their output orientations differ.

2.6 Whitening versus normalisation

Whitening targets cross-feature linear correlation as well as scale. Ordinary standardisation changes each feature independently. BatchNorm, LayerNorm and RMSNorm also normalise selected statistics rather than generally whitening the representation; see Section 6.3.

2.7 t-SNE, UMAP and latent visualisation

t-SNE and UMAP build low-dimensional views that prioritise neighbourhood relationships rather than globally faithful geometry. UMAP assumes that local neighbour relations reveal a lower-dimensional structure; t-SNE has a neighbourhood-scale hyperparameter commonly called **perplexity**. Use either primarily as an exploratory view, not proof that visual clusters are real physical classes. Check preprocessing, neighbourhood-scale settings, random seeds and external labels before interpreting a plot.

Inductive Bias

Method	Useful assumption	Failure to test
PCA	High variance contains useful linear structure	Signal lies in low-variance directions
Whitening	Linear correlation/scale are nuisance structure	Small-variance directions are mostly noise
t-SNE / UMAP	Local neighbours reveal meaningful organisation	Global distance/cluster appearance is over-interpreted
Autoencoder	Reconstruction preserves useful information	Rare task-critical signal contributes little to average reconstruction

2.8 Embeddings as operational geometry

An embedding turns a complex object into a vector whose geometry is useful for a downstream task. Text, images, users and items can therefore enter the same broad system pattern:

$$\underbrace{\text{object}}_{\text{text / image / user / item}} \longrightarrow \underbrace{f_\theta(\text{object})}_{\text{encoder}} \longrightarrow \underbrace{z \in \mathbb{R}^{N_{\text{embedding dims}}}}_{\text{searchable representation}}.$$

For retrieval, the useful assumption is that nearby vectors represent objects that should be treated similarly. This turns representation learning into an operational design choice because vector dimension, similarity and compression now change storage and search cost as well as model quality.

For nonzero vectors, cosine similarity removes magnitude while the raw dot product keeps it:

$$\underbrace{\cos(z_a, z_b)}_{\text{direction only}} = \frac{z_a^\top z_b}{\|z_a\|_2 \|z_b\|_2}, \qquad \underbrace{z_a^\top z_b}_{\text{direction + magnitude}}.$$

If embeddings are L2-normalised, the two scores are identical. The design question is therefore whether embedding norm should carry task information.



Figure 2.3: An embedding becomes operational when its geometry is used directly for retrieval or matching.

Dimension and nested representations. Larger embeddings can preserve more distinctions but cost more memory, bandwidth and search work. A Matryoshka-style representation trains useful prefixes of the same vector, so a deployment can choose a shorter or longer prefix without changing encoder family:

$$[z_1, \ldots, z_{64}] \subset [z_1, \ldots, z_{128}] \subset [z_1, \ldots, z_{N_{\text{embedding dims}}}].$$

This moves part of the quality–cost choice from retraining time to serving time.

Vector quantisation. Storing an embedding with fewer bits replaces its exact coordinates with nearby representable values. The important downstream question is not whether coordinates changed, but whether neighbour ordering changed enough to hurt retrieval. This connects directly to model quantisation in Section 14.1 and vector search in Sections 13.6 and 17.2.

Visual similarity and deduplication. A large image collection can use the same geometry directly:

$$\text{image} \rightarrow \text{visual embedding} \rightarrow \text{ANN neighbours} \rightarrow \text{near-duplicate / semantic candidates.}$$

An optional image-caption path can add a text-semantic representation when visual appearance alone misses the desired notion of similarity. PCA and low-bit vector storage can reduce index memory, but should be evaluated by whether the relevant neighbour ordering survives.

Memory Hook

Embeddings make geometry operational: search quality depends on which objects become neighbours.

Degree of Experiment

Vary encoder objective, embedding dimension, normalisation, similarity function, compression/quantisation and whether one representation must support several retrieval budgets.

SECTION 3

Learning Paradigms and Objectives

3.1 Supervised, unsupervised, self-supervised and reinforcement learning

Paradigm	Training information	Typical goal
Supervised	Input–target pairs	Classification, regression, forecasting
Unsupervised	Inputs without task labels	Density, clustering, compression
Self-supervised	Targets manufactured from inputs	Representation learning from structure in the data
Semi-supervised	Few labels plus many unlabelled examples	Label-efficient prediction
Reinforcement learning	States, actions and rewards through interaction	Policy maximising expected return

Table 3.1: Learning paradigms mainly differ in where the training signal comes from.

3.2 Supervised losses for continuous targets

For residuals $e_i = y_i - \hat{y}_i$ over N_{examples} examples:

$$\text{MSE} = \frac{1}{N_{\text{examples}}} \sum_i e_i^2, \tag{3.1}$$

$$\text{MAE} = \frac{1}{N_{\text{examples}}} \sum_i |e_i|, \tag{3.2}$$

$$\text{RMSE} = \sqrt{\text{MSE}}. \tag{3.3}$$

MSE makes large residuals disproportionately expensive and matches constant-variance Gaussian observation noise up to scale/constants. MAE grows linearly with residual size and matches a Laplace-like noise model. RMSE restores target units and is often more useful for reporting than for changing the optimiser itself; see Section 7.5.

Huber loss is quadratic near zero and linear for large residuals:

$$\ell_\delta(e) = \begin{cases} \frac{1}{2}e^2, & |e| \le \delta, \\ \delta\left(|e| - \frac{1}{2}\delta\right), & |e| > \delta. \end{cases}$$

It preserves a smooth small-error region while reducing the influence of large outliers relative to MSE.

Memory Hook

MSE squares outliers; MAE does not; Huber is quadratic near zero and linear far away.

3.3 Classification losses and logits

A binary logit a becomes $p = \sigma(a)$, and binary cross-entropy is

$$\ell_{\text{BCE}} = -\underbrace{y \log p}_{\text{reward positive probability when } y=1} - \underbrace{(1-y)\log(1-p)}_{\text{reward negative probability when } y=0}.$$

Use implementations that combine the logit and BCE calculation for numerical stability. Independent sigmoid/BCE outputs suit labels that can be simultaneously true.

For N_{classes} mutually exclusive classes, softmax converts logits to a probability simplex:

Memorise

$$p_i = \frac{e^{a_i}}{\sum_{j=1}^{N_{\text{classes}}} e^{a_j}}.$$

Categorical cross-entropy for the true class is $-\log p_{\text{true class}}$. Training loss and deployment metric are separate design choices; accuracy, precision/recall and F1 are in Sections 7.1 and 7.2.

3.4 Ranking and preference objectives

Ranking differs from ordinary regression because the useful output is usually an ordering rather than a numerically calibrated score. Three common objective shapes move from independent items toward the ordered list itself.

Objective	Training question	Main trade-off
Pointwise	Is this item good?	Simple, but treats ranking as independent prediction
Pairwise	Should item a rank above item b?	Direct ordering signal; creates many possible pairs
Listwise	Is the whole ordered list useful?	Closest to ranking metrics; optimisation is less direct

A pairwise model needs only score differences:

$$\underbrace{P(a \succ b)}_{a \text{ should rank above } b} = \sigma\left(\underbrace{s_a - s_b}_{\text{relative score}}\right).$$

This same relative-preference structure reappears in LLM preference optimisation in Section 13.4.

Listwise metrics such as NDCG care much more about mistakes near the top of the ranking. Sorting is discrete, so small score changes can leave NDCG unchanged until two items swap order. Methods such as LambdaRank therefore use smooth pairwise updates whose strength reflects how much a swap would change the ranking metric; the boosted-tree form LambdaMART is linked in Section 10.4.2.

Memory Hook

Pointwise scores items; pairwise orders pairs; listwise cares about the ordered list.

Inductive Bias

If relative preference is more reliable than an absolute target, pairwise learning uses that structure directly; it is less useful when calibrated scores themselves are the deployment output.

3.5 Representation-learning objectives

A reconstruction objective asks an encoder/decoder pair to preserve information needed to rebuild the input:

$$\mathcal{L}_{\text{reconstruction}} = \|x - g_{\theta_{\text{decoder}}}(f_{\theta_{\text{encoder}}}(x))\|_2^2.$$

Reconstruction does not guarantee preservation of task-relevant information; model families are summarised in Section 4.10.

Contrastive objectives instead shape relative geometry. A triplet-style objective says "positive closer than negative by a margin":

$$\mathcal{L}_{\text{triplet}} = \max\left(0, \underbrace{d(z_{\text{anchor}}, z_{\text{positive}})}_{\text{pull together}} - \underbrace{d(z_{\text{anchor}}, z_{\text{negative}})}_{\text{push apart}} + \underbrace{m_{\text{margin}}}_{\text{required separation}}\right).$$

The important experimental choices are how positives/negatives are constructed, which distance is used, and whether augmentations preserve task signal.

3.6 Variational and generative objectives

A VAE balances data fit with a structured latent distribution:

$$\underbrace{\mathcal{L}_{\text{VAE}}}_{\text{negative ELBO}} = \underbrace{-\mathbb{E}_{q_{\theta_{\text{encoder}}}(z|x)} \log p_{\theta_{\text{decoder}}}(x \mid z)}_{\text{reconstruction / likelihood}} + \underbrace{D_{\text{KL}}(q_{\theta_{\text{encoder}}}(z \mid x) \| p(z))}_{\text{latent prior compatibility}}.$$

GANs, diffusion models and normalising flows use different objectives; “generative model” does not imply this ELBO.

3.7 Reinforcement-learning objectives

A policy aims to maximise expected discounted reward:

$$\underbrace{J(\theta)}_{\text{policy objective}} = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{N_{\text{horizon}}} \underbrace{\gamma^t}_{\text{discount}} \underbrace{r_t}_{\text{reward}}\right].$$

The optimisation loss may combine policy-gradient, value-error and entropy terms. Reward, training loss and evaluation metric therefore need not be identical. Entropy itself is defined in Section 1.3.3.

3.8 What different objectives favour

Inductive Bias

Objective	Preference encoded	When to question it
MSE / Gaussian likelihood	Large residuals are disproportionately costly	Heavy-tailed noise or outliers
MAE / Laplace likelihood	Error cost grows linearly	Very large errors should matter much more
Cross-entropy	Probabilities should match class outcomes	Deployment costs are strongly asymmetric
Contrastive learning	Chosen positives share useful information	Augmentation removes signal or negatives are false
Reconstruction	Reconstructible information is worth preserving	Rare predictive signal has low reconstruction weight

SECTION 4

Models, Architectures and Their Assumptions

4.1 Linear models: a transparent baseline

Linear regression predicts with an affine score:

$$\hat{y} = \underbrace{\theta_{\text{weights}}^{\top} x}_{\text{weighted features}} + \underbrace{\theta_{\text{bias}}}_{\text{offset}}.$$

Logistic regression passes the same score through a sigmoid to model a binary probability. "Linear" refers to the chosen features: adding nonlinear engineered features can create nonlinear relationships in raw-input space while the model remains linear in its parameters. The baseline is valuable because it exposes which performance gains truly require nonlinear interactions.

4.2 KNN and similarity-based learning

K-nearest neighbours (KNN) predicts from the $N_{\text{neighbours}}$ closest training examples under a chosen distance. Small $N_{\text{neighbours}}$ gives flexible local behaviour and more noise sensitivity; large $N_{\text{neighbours}}$ smooths over a broader region. The core assumption is that proximity in the chosen representation predicts output similarity. That makes scaling and high-dimensional distance behaviour central; see Sections 2.1 and 2.2.

Practical Tip

For KNN, scale heterogeneous features and validate both the distance function and $N_{\text{neighbours}}$.

4.3 Support vector machines and kernels

A linear soft-margin SVM trades margin size against hinge-loss violations:

$$\min_{\theta_{\text{weights}}, \theta_{\text{bias}}} \underbrace{\frac{1}{2}\|\theta_{\text{weights}}\|_2^2}_{\text{prefer a wider margin / smaller weights}} + \underbrace{C \sum_{i=1}^{N_{\text{examples}}} \max\left(0, 1 - y_i(\theta_{\text{weights}}^{\top} x_i + \theta_{\text{bias}})\right)}_{\text{penalise margin violations}}.$$

The first term is an L2-style parameter preference, directly linking the SVM objective to Section 6.4. Larger C makes training violations more expensive relative to margin width.

A kernel replaces explicit feature construction with an inner product in an implicit feature space:

$$\underbrace{k(x, x')}_{\text{similarity used by the model}} = \left\langle \underbrace{\phi(x)}_{\text{implicit features}}, \phi(x') \right\rangle.$$

A linear separator in that feature space can therefore become nonlinear in the original coordinates.

4.4 Decision trees, bagging and boosting

A decision tree routes examples through hierarchical feature tests and returns a leaf prediction. This naturally represents nonlinear interactions but makes a single tree sensitive to early split choices. Bagging reduces that instability by averaging trees fitted to perturbed datasets; Random Forests add split-feature randomness to decorrelate the trees. Boosting instead adds trees sequentially to correct the current ensemble under a chosen objective. Bagging is mainly averaging for variance reduction; boosting is sequential objective-driven correction. Section 10 develops these mechanisms and their modern variants.

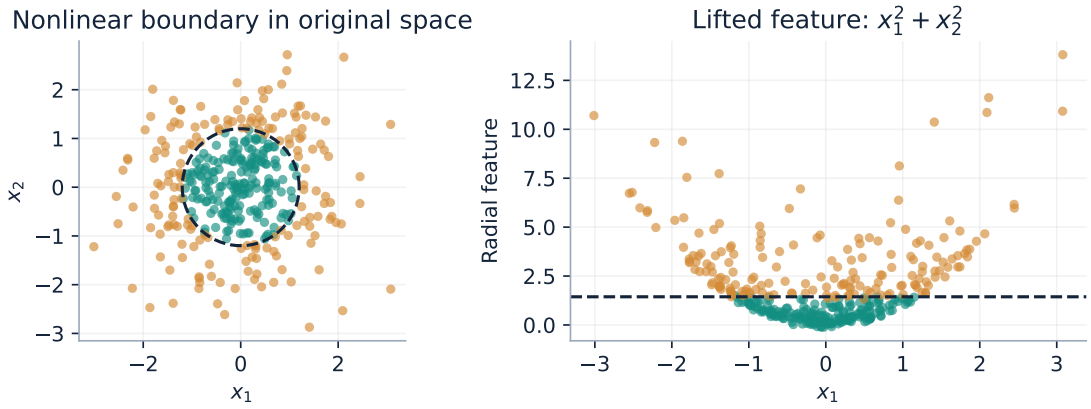


Figure 4.1: A nonlinear boundary in raw coordinates can become simple after a feature transformation; kernels can compute some such similarities without explicitly materialising the feature map.

4.5 Neural networks and nonlinear activations

A feed-forward layer is

$$\underbrace{h^{(\ell+1)}}_{\text{next representation}} = \underbrace{\phi}_{\text{nonlinearity}} \left(\underbrace{\theta^{(\ell)}_{\text{weights}} h^{(\ell)} + \theta^{(\ell)}_{\text{bias}}}_{\text{affine transform}} \right).$$

Without nonlinearities, stacked affine layers collapse to one affine map. Sigmoid/tanh saturate at large magnitude; ReLU keeps a constant positive-side derivative but can create inactive units; GELU gives a smooth input-dependent gate-like response. Activation choice interacts with initialisation and gradient scale in Section 6.2.

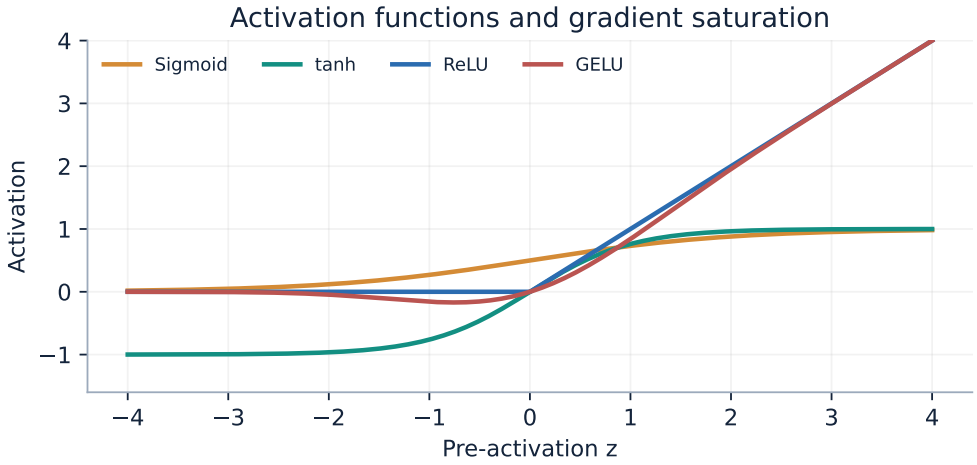


Figure 4.2: Common nonlinearities differ mainly in saturation, zeroing behaviour and local derivative scale.

4.6 CNNs: local structure and weight sharing

A convolutional layer can be read as

$$\underbrace{h}_{\text{output feature map}} = \phi \left(\underbrace{\theta_{\text{filter}} * x}_{\text{same local weighted pattern reused across positions}} + \theta_{\text{bias}} \right).$$

The exact indexing is implementation detail; the important assumptions are **locality** and **weight sharing**. Pooling and striding aggregate across positions, gaining larger receptive fields while sacrificing some spatial precision. Efficient convolutions, pretrained visual encoders and production vision pipelines are developed in section 11.

Inductive Bias

If nearby positions contain reusable local patterns, convolution shares evidence efficiently; if feature order is arbitrary, locality can be the wrong bias.

4.7 RNNs: sequential state

A recurrent network reuses one state-update rule across time:

$$\underbrace{h_t}_{\text{current state}} = \phi\left(\underbrace{\theta_{\text{input}} x_t}_{\text{new information}} + \underbrace{\theta_{\text{state}} h_{t-1}}_{\text{remembered context}} + \theta_{\text{bias}}\right).$$

Backpropagation through time multiplies local Jacobians across the unrolled sequence. This creates vanishing/exploding-gradient risk; see Section 1.4.1 and the stability discussion in Section 6.1.

4.7.1 LSTM: explicit cell memory and three gates

LSTM separates persistent cell memory c_t from exposed hidden state h_t. The cell update carries the mechanism:

$$\boxed{\underbrace{c_t}_{\text{new memory}} = \underbrace{f_t \odot c_{t-1}}_{\text{retain old memory}} + \underbrace{i_t \odot \tilde{c}_t}_{\text{write candidate memory}}}. \tag{4.1}$$

Forget, input and output gates are sigmoid-valued functions of the current input and previous hidden state; they control retention, writing and exposure respectively. The exposed state is $h_t = o_t \odot \tanh(c_t)$. The separate additive memory path makes long-range retention easier than in a plain RNN, but does not guarantee stable gradients indefinitely.

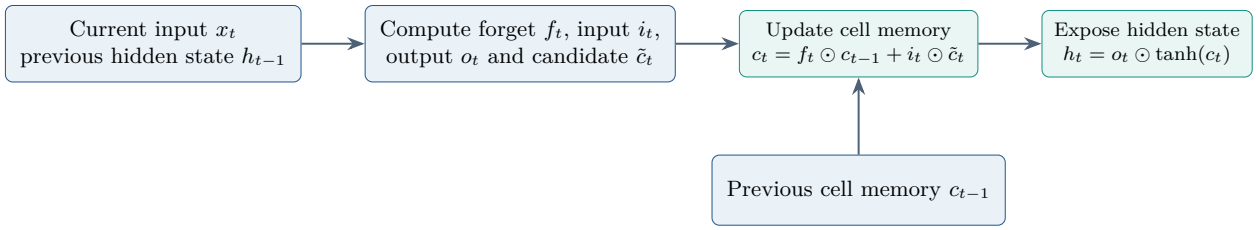


Figure 4.3: LSTM: compute gates, update persistent memory, then expose a hidden state.

4.7.2 GRU: a compact gated recurrent unit

A GRU keeps one state and uses reset/update gates instead of a separate cell-memory vector. Its key update is

$$\boxed{\underbrace{h_t}_{\text{new state}} = \underbrace{z_t \odot h_{t-1}}_{\text{retain old state}} + \underbrace{(1 - z_t) \odot \tilde{h}_t}_{\text{write candidate state}}}.$$

A reset gate controls how much previous state participates when constructing the candidate $\tilde{h}_t$. GRUs usually use fewer parameters than same-width LSTMs, but performance is task-dependent rather than ordered universally.

	GRU	LSTM
Stored state	One hidden vector	Hidden state plus cell memory
Main gates	Reset, update	Forget, input, output
Memory path	Direct hidden-state mixture	Separate additive cell-state path
Typical size at equal width	Smaller	Larger

4.8 Transformers: content-dependent interaction

A Transformer lets each token representation gather a weighted mixture of information from other tokens. Unlike an RNN, the route between positions is chosen from content rather than being forced through one

recurrent state. Attention therefore supplies content-dependent communication, while an MLP transforms each token after that communication. The detailed query/key/value mechanism, causal masking, multi-head attention and context-length trade-offs are developed visually in section 12.

Inductive Bias

If useful relationships between sequence elements depend on their content, attention can learn which positions should interact; dense attention becomes expensive when very long contexts make all-pairs interaction unnecessary.

4.9 State-space models, S4 and Mamba

A state-space model compresses history into an evolving latent state:

$$\underbrace{\dot{h}(t)}_{\text{state change}} = \underbrace{\theta_{\text{state}} h(t)}_{\text{memory dynamics}} + \underbrace{\theta_{\text{input}} x(t)}_{\text{new input}}, \qquad \underbrace{y(t)}_{\text{output}} = \theta_{\text{readout}} h(t) + \theta_{\text{skip}} x(t).$$

Structured state-space systems such as S4 choose parameterisations that make long-range sequence processing efficient. Selective systems such as Mamba let parts of the state update depend on the current input, giving content-dependent filtering. The shared inductive bias is that a compact evolving state can summarise a long history; Section 9.4 gives the broader view.

4.10 Autoencoders and generative model families

An autoencoder learns a latent representation by reconstructing its input. A VAE makes that latent representation probabilistic and regularises it toward a prior; see Sections 3.5 and 3.6. GANs train generator and discriminator adversarially; diffusion models learn to reverse a corruption process; normalising flows use invertible transformations with tractable density changes; energy-based models learn relative energy scores. The family name identifies a training/generative mechanism, not a universal quality ranking.

4.11 Gumbel–Softmax: differentiable categorical choices

A hard categorical sample blocks ordinary pathwise gradients. Gumbel–Softmax replaces the hard one-hot choice during optimisation with a temperature-controlled softmax of noisy category logits:

$$\underbrace{y_i}_{\text{soft category weight}} = \operatorname{softmax}_i\left(\frac{\underbrace{\log \pi_i}_{\text{category log-weight}} + \underbrace{g_i}_{\text{Gumbel noise}}}{\underbrace{\tau}_{\text{temperature}}}\right).$$

Lower temperature makes the vector more nearly one-hot but also makes gradients more brittle. A straight-through variant uses a hard forward choice with an approximate backward gradient.

Memory Hook

Gumbel–Softmax makes discrete choice approximately differentiable by softening it during optimisation.

4.12 Model assumptions and inductive biases

An architecture is a package of assumptions about which functions should be easy to represent. Section 9.4 explains why some such bias is unavoidable.

Table 4.1: Model families, their structural preference, and a representative useful regime.

Model	Structural preference / assumption	Potential useful setting
Linear / logistic regression	Chosen features combine additively in an affine score	Interpretable baseline effects
KNN	Nearby observations should have similar targets	Local smooth structure

Model	Structural preference / assumption	Potential useful setting
Kernel SVM	A chosen similarity creates a useful separating feature space	Nonlinear boundaries, moderate data
Decision tree	Hierarchical thresholds capture useful interactions	Heterogeneous tabular relations
Random forest	Averaging decorrelated trees reduces instability	Nonlinear tabular baselines
Gradient boosting	Sequential corrections improve a chosen objective	Structured/tabular data
PCA	High-variance linear directions are useful	Compression/exploration
Gaussian process	Kernel covariance describes function similarity	Small-data regression with uncertainty
MLP	Learned nonlinear compositions capture the relation	General vector inputs
CNN	Local patterns repeat across positions	Images/gridded signals
RNN / GRU / LSTM	Recurrent state summarises useful past context	Sequences/streaming
Transformer	Content-dependent token interactions matter	Context-rich sequence/multimodal tasks
GNN	Graph connectivity defines relevant neighbours	Molecules/relational systems
SSM / Mamba	Long history can be compressed into evolving state	Long or streaming sequences
Autoencoder / VAE	Reconstruction preserves useful latent structure	Compression/generative latents
GAN / diffusion / flow	Different generative mechanisms match complex data distributions	Sample/density modelling
Mixture of experts	Different inputs benefit from specialist subnetworks	Large conditional-capacity systems
Siamese / contrastive	Related examples should be close in learned space	Retrieval/few-shot tasks

SECTION 5

Optimisation and Training Mechanics

The objective says what "better" means (section 3); the optimiser turns derivatives of that objective into parameter updates.

5.1 Gradient descent — follow the local slope

For model parameters $\theta \in \mathbb{R}^{N_{\text{parameters}}}$ and loss $\mathcal{L}(\theta)$, the first-order baseline is:

Memorise

$$\theta_{t+1} = \theta_t - \underbrace{\eta}_{\text{step size}} \underbrace{\nabla_\theta \mathcal{L}(\theta_t)}_{\text{local uphill direction}}.$$

The negative gradient is the steepest local descent direction under Euclidean geometry. The learning rate controls how far that local information is trusted: too small wastes updates; too large can overshoot or diverge.

Example

For $\mathcal{L}(\theta) = (\theta - 3)^2$, gradient descent repeatedly moves toward 3; changing only η changes whether the trajectory converges smoothly, oscillates, or diverges.

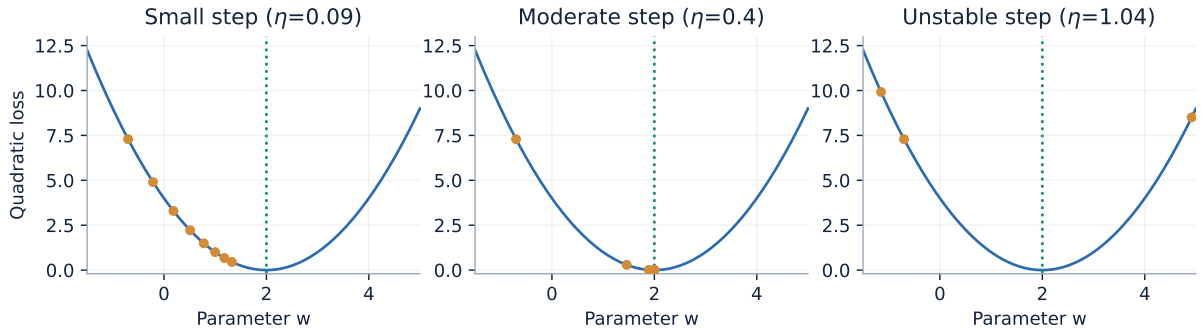


Figure 5.1: Gradient-descent step-size behaviour on a quadratic objective; the point is the stability trade-off, not the specific quadratic.

5.2 Full-batch, stochastic and mini-batch gradients

For N_{examples} training examples, full-batch descent uses every example before each update; strict SGD uses one; mini-batch SGD uses a sampled subset $\mathcal{B}_t$. The mini-batch update is already the whole story:

$$\theta_{t+1} = \theta_t - \eta \underbrace{\frac{1}{N_{\text{batch}}} \sum_{i \in \mathcal{B}_t} \nabla_\theta \ell_i(\theta_t)}_{\text{mini-batch estimate of the full gradient}}, \qquad N_{\text{batch}} = |\mathcal{B}_t|.$$

Smaller batches make updates cheaper and noisier; larger batches reduce gradient noise but consume more memory/compute per update. Under ordinary uniform sampling, the mini-batch gradient is an unbiased estimate of the full gradient, but a noisy step need not decrease the full training loss.

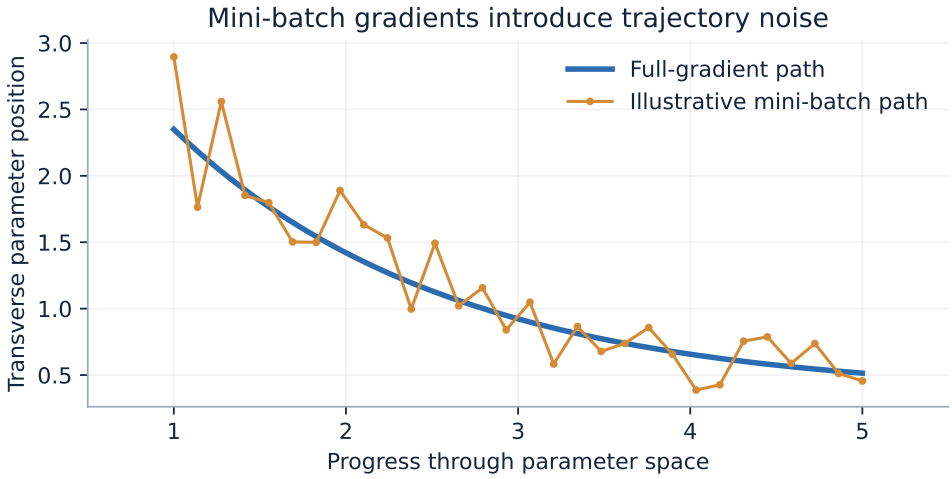


Figure 5.2: A schematic mini-batch trajectory versus a smooth full-gradient trajectory; stochasticity changes the path, not the underlying objective.

5.3 Backpropagation — calculate derivatives, do not choose updates

Backpropagation is reverse-mode differentiation through the computation graph. It repeatedly applies the chain rule so downstream sensitivity is multiplied by local sensitivity:

$$\underbrace{\frac{\partial \mathcal{L}}{\partial \theta_j}}_{\text{parameter sensitivity}} = \underbrace{\frac{\partial \mathcal{L}}{\partial h}}_{\text{downstream sensitivity}} \underbrace{\frac{\partial h}{\partial \theta_j}}_{\text{local sensitivity}}.$$

The Jacobian view in Section 1.4.1 is the vector-valued version of the same mechanism. Backprop computes gradients; SGD, Adam and Newton-like methods decide how to use them. In frameworks that accumulate gradients, clear previous gradients unless accumulation is intentional.

5.4 Momentum — remember persistent directions

Momentum adds a decaying memory of earlier gradients:

$$\underbrace{m_t}_{\text{running direction}} = \underbrace{\beta m_{t-1}}_{\text{memory}} + \underbrace{(1-\beta) g_t}_{\text{new gradient}}, \qquad \theta_{t+1} = \theta_t - \eta m_t.$$

Persistent directions accumulate; sign-changing directions partly cancel. This often accelerates motion along shallow valleys while reducing zig-zagging across steep ones. Different libraries absorb $(1-\beta)$ or η into the accumulator, so compare parameterisations before comparing hyperparameters.

5.5 AdaGrad and RMSProp — adapt step size by gradient scale

AdaGrad remembers the full history of squared gradients:

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{\underbrace{s_{t-1} + g_t^{\odot 2}}_{s_t:\ \text{accumulated squared gradients}}} + \epsilon}.$$

Coordinates with repeatedly large gradients receive smaller effective steps. Because s_t only grows, long runs can make the effective step size vanish.

RMSProp replaces that permanent memory with exponential forgetting:

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{\underbrace{\rho v_{t-1} + (1-\rho) g_t^{\odot 2}}_{v_t:\ \text{recent squared-gradient scale}}} + \epsilon}.$$

The square, square root and division act coordinate-wise; $\epsilon > 0$ prevents unstable division by tiny scales.

5.6 Adam — momentum plus adaptive scaling

With $m_0 = v_0 = 0$, Adam combines a first-moment gradient memory with a second-moment gradient-scale memory and corrects their early zero-initialisation bias:

Memorise

$$\theta_{t+1} = \theta_t - \eta \frac{\underbrace{\frac{\beta_1 m_{t-1} + (1-\beta_1) g_t}{1-\beta_1^t}}_{\hat{m}_t\text{: bias-corrected momentum}}}{\sqrt{\underbrace{\frac{\beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2}}{1-\beta_2^t}}_{\hat{v}_t\text{: bias-corrected gradient scale}}} + \epsilon}.$$

Typical starting values are $\beta_1 = 0.9$ and $\beta_2 = 0.999$, but they are design choices rather than constants of nature. Adam stores two state vectors per parameter vector. Its squared-gradient statistic is not the Hessian: Adam remains a first-order adaptive method.

5.7 AdamW — decouple weight decay from adaptive scaling

For plain SGD, an L2 penalty and multiplicative weight decay can produce the same update form under a simple convention. Inside Adam, adding $\lambda\theta$ to the task gradient sends the regularisation gradient through adaptive moment scaling, so coordinates are no longer shrunk uniformly. AdamW keeps the task-gradient adaptation and parameter shrinkage separate:

$$\theta_{t+1} = \theta_t - \underbrace{\eta \frac{\frac{\beta_1 m_{t-1} + (1-\beta_1) g_t}{1-\beta_1^t}}{\sqrt{\frac{\beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2}}{1-\beta_2^t}} + \epsilon}}_{\text{Adam task-gradient step}} - \underbrace{\eta\lambda\theta_t}_{\text{decoupled weight decay}}.$$

The moment states are computed from the task gradient, not from the decay term. Biases and normalisation scales are often excluded from decay through parameter groups. See Section 6.4 for the regularisation interpretation.

Memory Hook

AdamW shrinks parameters outside Adam's adaptive gradient scaling.

5.8 How optimiser ideas evolved: a branching genealogy

Step	What changed	Problem targeted	New trade-off
SGD	No state beyond gradient	Cheap scalable updates	Step size and conditioning dominate behaviour
Momentum	Running gradient direction	Zig-zagging / slow aligned progress	Extra state; can overshoot
AdaGrad	Per-coordinate squared history	Unequal coordinate scales / sparse gradients	Learning rates can decay too far
RMSProp	Forget old squared gradients	AdaGrad's permanent accumulation	Decay constant becomes another design choice
Adam	Momentum + RMSProp-like scale + bias correction	Combine directional memory and adaptive steps	More state; not automatically best generalisation
AdamW	Separate parameter decay	Adam couples L2 gradient to adaptive scaling	Decay and learning rate still interact through training dynamics

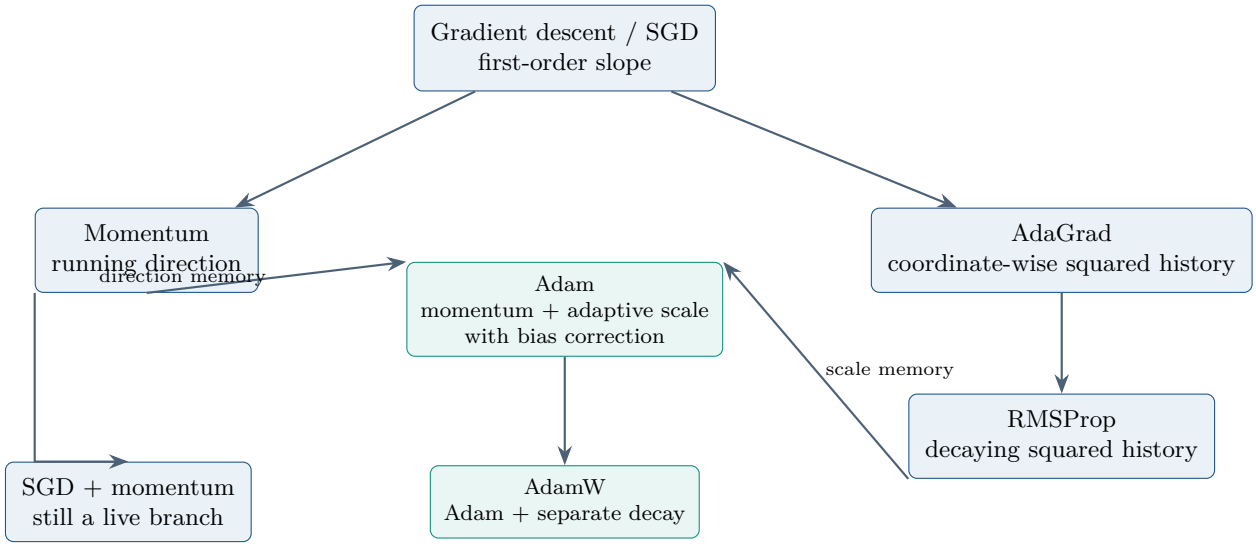


Figure 5.3: Optimiser evolution as accumulated fixes: remember direction, adapt coordinate scale, combine both, then separate regularisation from adaptation.

Practical Tip

Use AdamW as a strong default for many deep networks, but keep SGD with momentum as a meaningful baseline and tune the learning-rate schedule with the optimiser.

Degree of Experiment

Vary update rule, momentum/decay constants, N_{batch}, learning-rate schedule, gradient clipping, decay strength, and which parameter groups receive decay.

5.9 Newton's method and curvature-aware updates

The Hessian from Section 1.4.2 changes a slope-only step into a local curvature-adjusted step:

$$\underbrace{\nabla_\theta^2 \mathcal{L}(\theta)}_{\text{Hessian}} \quad \underbrace{\Delta_{\text{Newton}}}_{\text{curvature-adjusted step}} = -\underbrace{\nabla_\theta \mathcal{L}(\theta)}_{\text{gradient}}.$$

Solve this linear system rather than explicitly forming a matrix inverse. A full Hessian scales quadratically with $N_{\text{parameters}}$, so exact Newton steps are usually impractical for large networks. Negative curvature can also make an undamped Newton direction unsuitable for descent. Practical families therefore approximate or constrain curvature: L-BFGS stores limited gradient-history information, Gauss–Newton uses problem structure, and Hessian-vector-product methods avoid materialising the full matrix.

Memory Hook

Adam rescales first-order gradients; Newton methods use cross-parameter curvature.

5.10 Learning-rate schedules and warmup

Warmup limits early update size; decay reduces later update size; cosine schedules vary it smoothly; plateau schedules react to a monitored metric. The schedule is part of the optimiser system: batch size, optimiser state, architecture and training duration all change which schedule works well.

5.11 Gradient accumulation and mixed precision

Large models often hit memory limits before the mathematical training rule changes. **Gradient accumulation** splits one effective batch across several forward/backward passes, adds their gradients, then performs

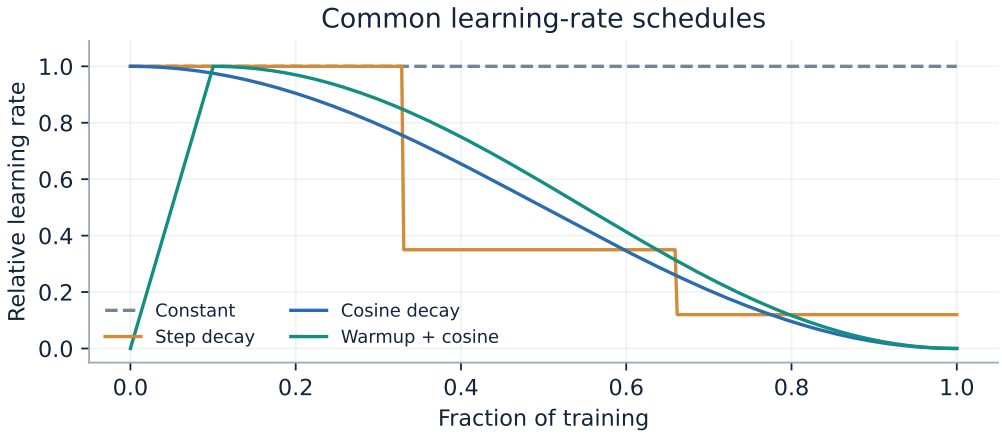


Figure 5.4: Common relative learning-rate profiles; the important design variable is how aggressively step size changes over training.

one optimiser update:

$$\underbrace{N_{\text{effective batch}}}_{\text{examples per update}} = \underbrace{N_{\text{microbatch}}}_{\text{examples resident at once}} \times \underbrace{N_{\text{accumulation steps}}}_{\text{backward passes before update}} \times \underbrace{N_{\text{data-parallel workers}}}_{\text{workers contributing}}.$$

This lowers peak activation memory but does not remove the compute required for those examples. Changing effective batch size can change gradient noise and should therefore be considered jointly with the learning rate and schedule.

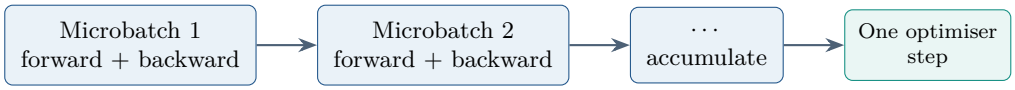


Figure 5.5: Gradient accumulation delays the parameter update so several small resident batches act like one larger effective batch.

Mixed precision stores or computes suitable tensors in lower precision while keeping numerically sensitive state at higher precision. FP16 has more mantissa precision but a narrower exponent range; BF16 has an FP32-like exponent range and is usually more tolerant of large/small values. FP16 training can use loss scaling so tiny gradients are multiplied into a representable range before being unscaled for the update.

Memory Hook

Accumulation solves peak batch memory by delaying the update; mixed precision solves memory/bandwidth by representing numbers more cheaply.

Degree of Experiment

Vary microbatch size, accumulation count, numeric precision, loss scaling, data-parallel worker count and the learning-rate schedule together rather than treating them as independent implementation details.

5.12 The full training loop

```
for inputs, targets in train_loader:
    optimizer.zero_grad()        # clear accumulated gradients
    predictions = model(inputs) # forward pass
    loss = loss_fn(predictions, targets)
    loss.backward()              # reverse-mode derivatives
    optimizer.step()             # parameter update
```

Validation runs separately from parameter updates. Early stopping and untouched-test-set discipline are covered in Sections 6.7 and 7.8. A scheduler should be stepped according to its intended semantics: per

update, epoch, or monitored metric.

SECTION 6

Training Stability and Regularisation

Stable optimisation and good generalisation are different goals: a model can train cleanly and still overfit.

6.1 Weight initialisation — start in a trainable region

A deep network repeatedly applies parameterised transforms, so forward activations and backward Jacobian products can grow or shrink layer after layer. If typical directional gains are too small, gradients vanish; if too large, they explode. The local-sensitivity view is in Section 1.4.1. Zero-initialising all weights in a symmetric hidden layer also prevents neurons from differentiating because equivalent units receive equivalent gradients. Randomness breaks symmetry; variance scaling keeps the starting signal numerically usable.

6.2 Xavier / Glorot and He / Kaiming initialisation

For a dense layer, common Gaussian starting variances are

$$\theta_{ij,\text{Xavier}} \sim \mathcal{N}\left(0, \underbrace{\frac{2}{N_{\text{inputs}} + N_{\text{outputs}}}}_{\text{balance forward/backward scale}}\right), \tag{6.1}$$

$$\theta_{ij,\text{He}} \sim \mathcal{N}\left(0, \underbrace{\frac{2}{N_{\text{inputs}}}}_{\text{compensate ReLU half-zeroing}}\right). \tag{6.2}$$

Xavier is a common starting point for linear/tanh-like activations; He/Kaiming is designed around ReLU-like activations. These are approximate signal-scale arguments, not guarantees that training remains well conditioned.

Practical Tip

ReLU-like activation: start with He/Kaiming; linear or tanh-like activation: Xavier/Glorot is a common baseline.

6.3 Normalisation — control intermediate scale

Normalisation methods mainly differ in **which axes define the reference statistics**.

For N_{values} values in the selected normalisation group:

$$\underbrace{\hat{u}_i}_{\text{centred/scaled value}} = \frac{u_i - \underbrace{\frac{1}{N_{\text{values}}}\sum_j u_j}_{\mu_{\text{group}}}}{\sqrt{\underbrace{\frac{1}{N_{\text{values}}}\sum_j (u_j - \mu_{\text{group}})^2}_{\sigma^2_{\text{group}}} + \epsilon}}.$$

A learned scale and optional shift can restore useful amplitude/offset after normalisation.

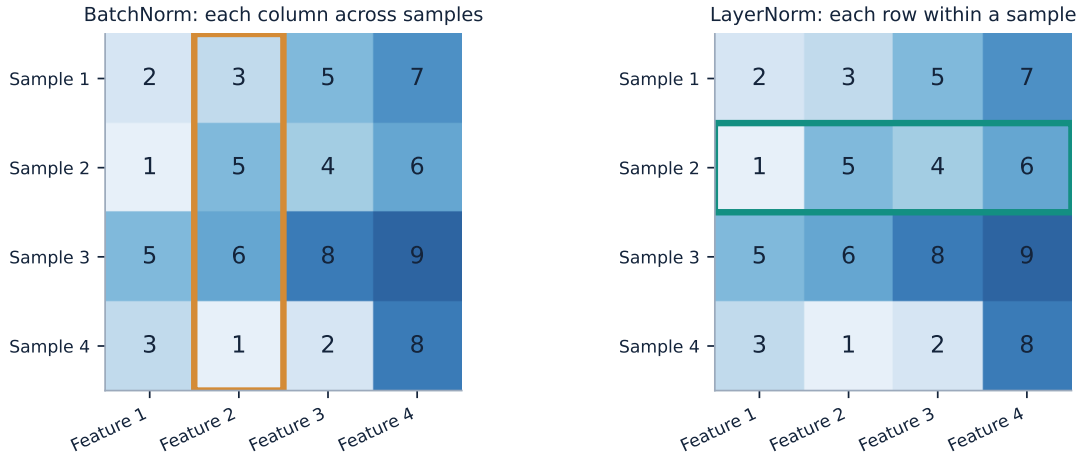


Figure 6.1: BatchNorm and LayerNorm normalise different slices of an activation tensor; the chosen axes are the important design choice.

6.3.1 Batch Normalisation

BatchNorm shares statistics across examples for corresponding features/channels during training and usually uses running training statistics at inference. It can improve optimisation, but tiny or nonrepresentative batches make its reference statistics noisy or mismatched.

6.3.2 Layer Normalisation

LayerNorm computes statistics within each example/token across a chosen feature set, so its statistics do not depend on unrelated examples in the batch. This is why it fits sequence models with variable or small batch sizes well.

6.3.3 RMSNorm

RMSNorm controls vector magnitude without subtracting the mean:

$$\boxed{\mathrm{RMSNorm}(h)_i = \theta_{\mathrm{scale},i} \frac{h_i}{\sqrt{\frac{1}{N_{\mathrm{features}}} \sum_j h_j^2 + \epsilon}}}.$$

It is cheaper than full centring-and-scaling and is common in modern language models.

	BatchNorm	LayerNorm	RMSNorm
Statistics over	Batch, per feature/channel	Features within one example/token	Features within one example/token
Subtract mean?	Yes	Yes	No
Scale by	Standard deviation	Standard deviation	Root mean square
Depends on other examples?	During training, yes	No	No

Table 6.1: Normalisation methods encode different reference groups and therefore different invariances.

Degree of Experiment

Vary statistic axes, centring, batch dependence, residual/nonlinearity placement, learned scale/shift, and numerical stabilisation.

Memory Hook

BatchNorm normalises across examples; LayerNorm/RMSNorm normalise within each example or token.

These methods do not generally whiten cross-feature covariance; whitening is separated in Sections 2.5 and 2.6.

6.4 Regularisation — L1 and L2

A regularised objective exposes the trade-off directly:

$$\underbrace{\mathcal{L}_{\text{total}}(\theta)}_{\text{what optimisation sees}} = \underbrace{\mathcal{L}_{\text{data}}(\theta)}_{\text{fit observations}} + \underbrace{\lambda R(\theta)}_{\text{parameter preference}} .$$

L2 uses $R(\theta) = \|\theta\|_2^2$ and usually shrinks parameters smoothly. L1 uses $R(\theta) = \|\theta\|_1$ and can drive coefficients exactly to zero; Elastic Net combines both. The interpretation depends on parameterisation, so "small weights" is not universally the same as "simple realised function".

Intuition

L1 is basically feature selection when individual weights map cleanly to input features; L2 mostly shrinks rather than deletes them.

The Bayesian MAP view is in Section 1.3.2. For Adam, decoupled weight decay changes the update relative to putting an L2 gradient inside Adam; see Section 5.7.

6.5 Dropout and stochastic regularisation

In inverted dropout, each activation is randomly kept with probability $1 - p_{\text{drop}}$ and rescaled so its conditional expectation is unchanged:

$$\tilde{h}_j = \underbrace{\frac{m_j}{1 - p_{\text{drop}}}}_{\text{random keep + expectation-preserving scale}} h_j, \qquad m_j \sim \text{Bernoulli}(1 - p_{\text{drop}}).$$

Dropout discourages dependence on one exact active pathway, but too much stochasticity can make optimisation unnecessarily difficult. It is normally disabled at inference.

6.6 Data augmentation and symmetry assumptions

Augmentation asserts that a transformation should preserve or predictably transform the target. A horizontal flip may preserve animal identity but invert left/right anatomy; time reversal may violate causality in a physical signal. Augmentation is therefore an inductive-bias choice, not free extra data; see Section 9.4.

6.7 Early stopping — stop before fitting validation-specific noise

Training loss may keep falling after held-out performance stops improving. Early stopping selects a checkpoint using validation behaviour; patience controls how much non-improvement is tolerated before stopping. The test set must remain outside that selection loop (Section 7.8).

Practical Tip

If training loss falls while validation performance worsens, restore the best validation checkpoint rather than the final training checkpoint.

6.8 Residual connections: an additional stability lever

A residual block learns a correction to an identity path:

$$\underbrace{h_{\text{out}}}_{\text{block output}} = \underbrace{h_{\text{in}}}_{\text{identity path}} + \underbrace{F_\theta(h_{\text{in}})}_{\text{learned correction}} .$$

The direct path gives information and gradients a route that does not require every learned transform to preserve them perfectly. Residual connections do not remove the need for sensible initialisation, normalisation or optimisation, but they change the conditioning of very deep networks.

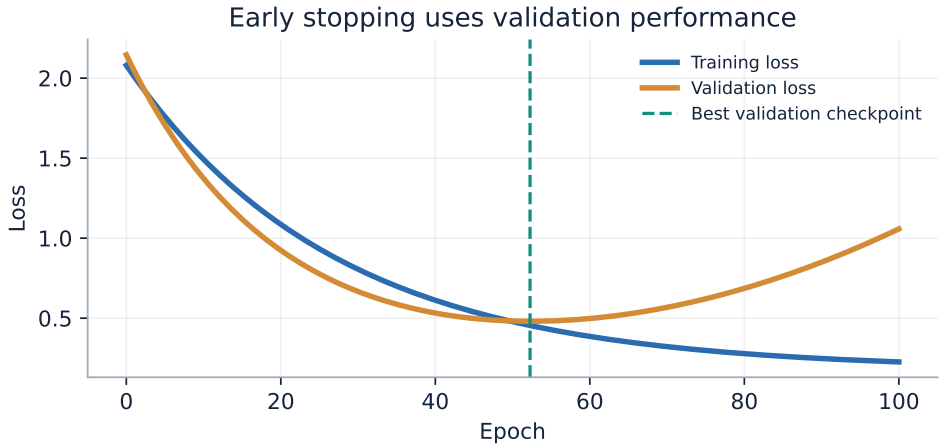


Figure 6.2: Training can continue improving while validation performance deteriorates; early stopping selects by held-out behaviour rather than training loss alone.

Inductive Bias

If the desired transformation is close to identity plus a correction, residual parameterisation makes that solution easy; if not, it is still expressive but the bias may be less useful.

SECTION 7

Evaluation, Validation and Model Selection

Evaluation asks whether the trained system works on the deployment distribution and decision problem, not merely whether training loss decreased.

7.1 Classification evaluation — the confusion matrix

For binary classification, predictions fall into true positive (TP), false positive (FP), true negative (TN) and false negative (FN).

	Actual positive	Actual negative
Predicted positive	TP	FP
Predicted negative	FN	TN

Accuracy $(\mathrm{TP}+\mathrm{TN})/N_{\text{examples}}$ can hide rare-class failure because the majority class dominates the count.

7.2 Precision, recall and F1

$$\underbrace{P}_{\text{precision: correctness of alerts}} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}, \tag{7.1}$$

$$\underbrace{R}_{\text{recall: coverage of positives}} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}, \tag{7.2}$$

$$F_\beta = (1+\beta^2)\frac{PR}{\beta^2 P + R}. \tag{7.3}$$

Larger β weights recall more strongly; smaller positive β weights precision more strongly. The prevalence/base-rate connection is the same one shown by Bayes' rule in Section 1.3.2.

Special Case

F_1 is F_β with $\beta = 1$.

F1 ignores TN directly, so it is not automatically suitable when correctly rejecting negatives is itself important.

7.3 Decision thresholds

A score becomes a binary decision through a threshold:

$$\hat{y} = \mathbb{1}\left[s(x) \geq \underbrace{\tau}_{\text{deployment threshold}}\right].$$

Lowering τ increases or preserves the number of positive predictions and cannot reduce recall on a fixed dataset; precision may move either way. Threshold selection is therefore a deployment decision layered on top of a scoring model (Section 3.3). Calibration is separate again: among predictions near probability 0.7, roughly 70% should occur if the model is well calibrated for that population.

7.4 ROC curves, AUC-ROC and precision–recall

ROC curves trace true-positive rate against false-positive rate as the threshold moves. ROC-AUC is equivalently the probability that a random positive is ranked above a random negative, with standard tie handling. AUC measures ranking across thresholds; it does not choose a deployment threshold or guarantee calibrated probabilities.

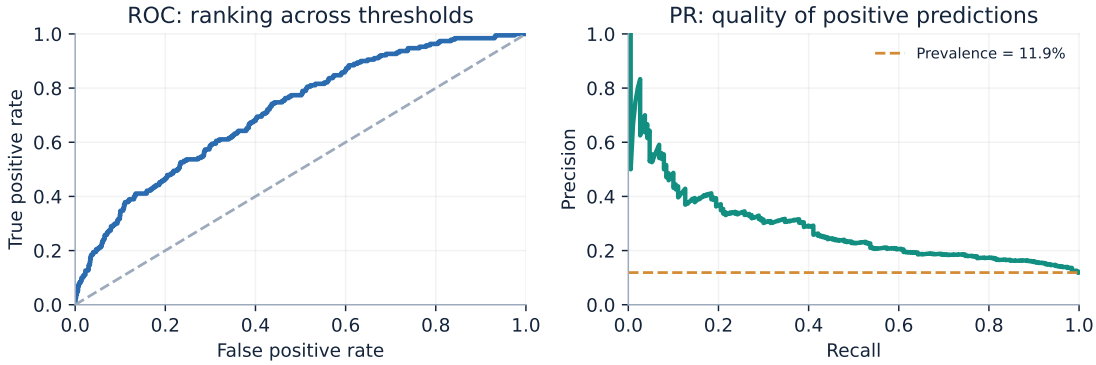


Figure 7.1: ROC and precision–recall curves expose different threshold trade-offs; the PR baseline depends on positive prevalence.

For rare events, a small false-positive rate can still create many more false alerts than true alerts. Precision–recall curves expose that directly, so rare-event systems should not rely on accuracy or ROC-AUC alone. Average Precision and trapezoidal PR-AUC are different conventions; report which one is used.

Practical Tip

Rare positives: report precision, recall and a precision–recall curve, then choose threshold from false-alarm versus miss cost.

7.5 Regression metrics

For N_{examples} targets, useful summary metrics are

$$\text{MAE} = \frac{1}{N_{\text{examples}}} \sum_i |y_i - \hat{y}_i|, \tag{7.4}$$

$$\text{RMSE} = \sqrt{\frac{1}{N_{\text{examples}}} \sum_i (y_i - \hat{y}_i)^2}, \tag{7.5}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \tag{7.6}$$

RMSE has target units and magnifies large errors relative to MAE. $R^2 = 0$ means the model matches the evaluation-set mean baseline under squared error; it can be negative and is undefined in the usual formula if every target is identical.

7.6 Cross-validation and fold choice

In N_{folds}-fold cross-validation, each fold is validation once while the others train the model. Preprocessing is refitted inside every training fold (Section 2.2). The mean fold score is

$$\bar{s} = \frac{1}{N_{\text{folds}}} \sum_{i=1}^{N_{\text{folds}}} \underbrace{s_i}_{\text{score on held-out fold } i}.$$

Fold scores are correlated because their training sets overlap, so their ordinary standard deviation is descriptive rather than automatically a deployment confidence interval.

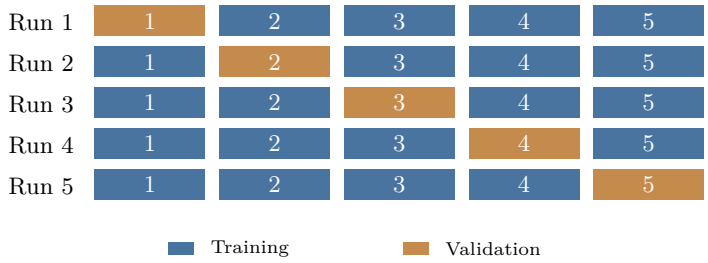


Figure 7.2: Five-fold cross-validation: each fold is held out once.

Split strategy	Suitable setting	Main leakage controlled
Random K-fold	Examples are approximately independent and similarly distributed	Dependence on one arbitrary split
Stratified K-fold	Class proportions vary strongly	Unrepresentative class balance
Group K-fold	Repeated measurements share subject/run/instrument identity	Same group appearing on both sides
Forward / time-series split	Future prediction	Training on future observations
Nested CV	Hyperparameter selection with limited data	Reusing selection folds for final assessment

Practical Tip

Split by the independent unit: keep repeated measurements from one subject/run together, and preserve time order for forecasting.

7.7 Overfitting, underfitting and the bias–variance tradeoff

Underfitting misses important structure in both training and validation data. Overfitting fits training-specific structure that does not generalise. A train/validation gap can also come from leakage or distribution shift, so diagnose the data split before naming the failure.

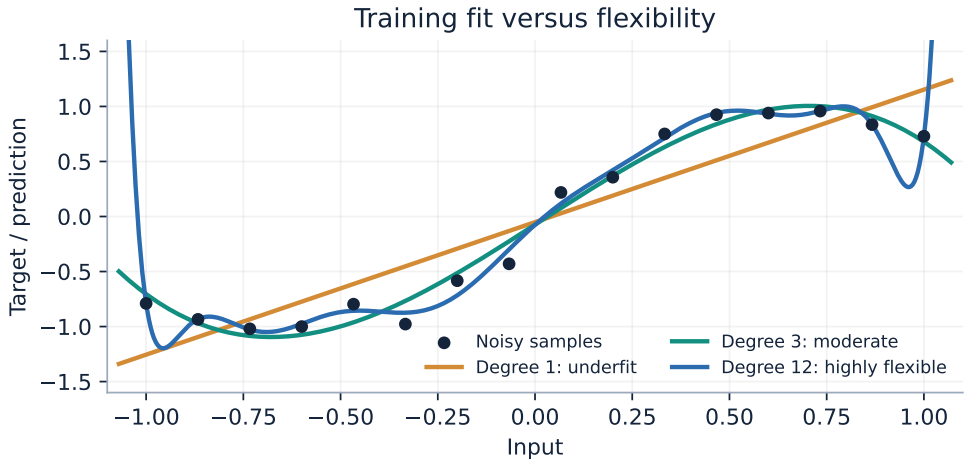


Figure 7.3: Increasing flexibility can move from systematic underfit toward fitting incidental training variation.

For squared-error regression, expected test error at a fixed input decomposes as

$$\mathbb{E}[(Y - \hat{f}_D(x))^2] = \underbrace{(\mathbb{E}_D \hat{f}_D(x) - f(x))^2}_{\text{bias}^2} + \underbrace{\operatorname{Var}_D(\hat{f}_D(x))}_{\text{variance across training sets}} + \underbrace{\sigma^2_{\text{noise}}(x)}_{\text{irreducible noise}} . \tag{7.7}$$

This classical picture explains why regularisation can trade fit for stability and why bagging can reduce model

variance. Modern overparameterised networks can depart from a simple U-shaped bias–variance curve, so treat the decomposition as a mental model rather than a universal empirical law. See Sections 6.4 and 10.3.

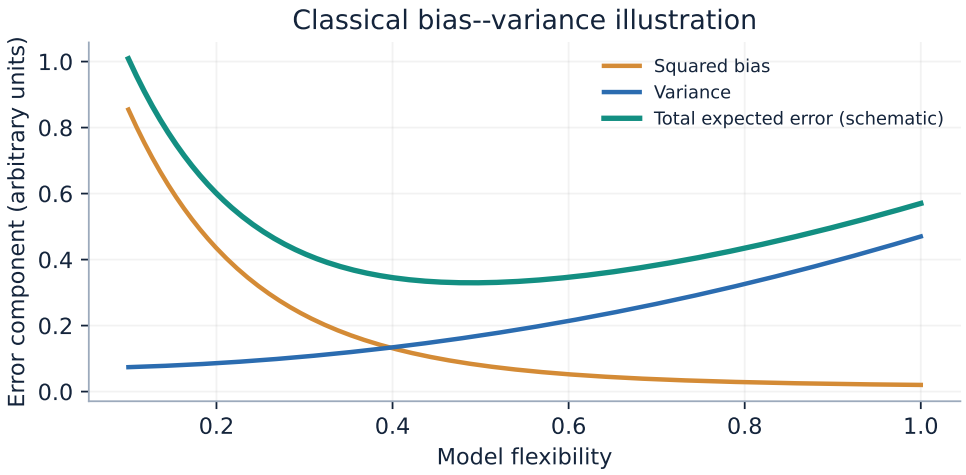


Figure 7.4: Classical bias–variance schematic; the curves are explanatory rather than universal.

7.8 Model selection and data leakage

Training fits parameters; validation selects hyperparameters/checkpoints/thresholds; testing estimates final performance after those choices are fixed. Repeatedly choosing models from test performance turns the test set into another validation set. Leakage includes fitting preprocessing before splitting, label-dependent feature selection on all data, near-duplicates across splits, shared subjects/runs across folds, and future or target information entering features.

For $N_{\text{independent cases}}$ Bernoulli outcomes, the plug-in standard error of an accuracy/proportion estimate is

$$\mathrm{SE}(\hat{p}) \approx \sqrt{\frac{\hat{p}(1-\hat{p})}{N_{\text{independent cases}}}}.$$

This relies on independence and representative sampling; the foundations of standard error/confidence intervals are in Section 1.4.4. Wilson intervals or suitable bootstrap procedures are preferable in many small-sample or nonstandard-metric settings. Resample at the true independent unit: subject, run, machine, block, or another domain-defined unit.

7.9 Retrieval and ranking metrics

A multi-stage retrieval/ranking system should be evaluated at the stage where a failure can still be corrected. Early retrieval cares about whether relevant items survived; later ranking cares about where they appear.

Recall@K. If N_{relevant} items exist and $N_{\text{relevant in top K}}$ survive the first stage,

$$\underbrace{\text{Recall@K}}_{\text{candidate coverage}} = \frac{N_{\text{relevant in top K}}}{N_{\text{relevant}}}.$$

A downstream reranker cannot recover an item that retrieval never produced.

MRR. Mean Reciprocal Rank rewards putting the first relevant result near the top:

$$\mathrm{MRR} = \frac{1}{N_{\text{queries}}} \sum_{q} \frac{1}{\underbrace{\mathrm{rank}_{q,\text{first relevant}}}_{\text{where useful information first appears}}}.$$

NDCG. Discounted Cumulative Gain gives top positions more weight and allows graded relevance:

$$\underbrace{\text{NDCG@K}}_{\text{normalised ranking quality}} = \frac{\sum_{r=1}^{K} \frac{2^{\text{relevance}_r}-1}{\log_2(r+1)}}{\underbrace{\text{IDCG@K}}_{\text{best possible ordering for this query}}}.$$

The logarithmic discount is why swapping ranks 1 and 2 matters much more than swapping ranks 101 and 102.

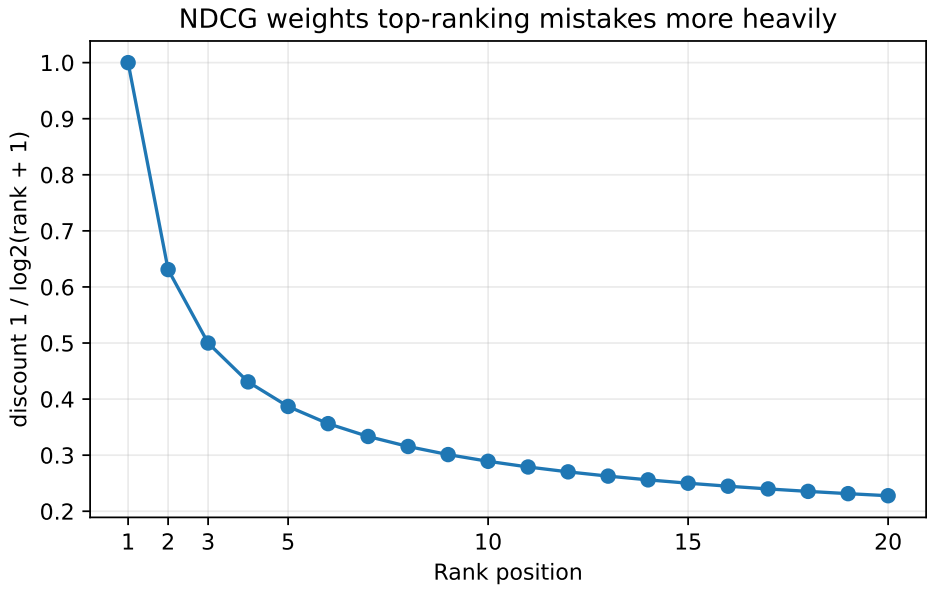


Figure 7.5: NDCG's position discount falls quickly near the top, so high-ranking mistakes receive most of the metric weight.

Practical Tip

Evaluate candidate generation with coverage-oriented metrics such as Recall@K, then evaluate the later ordering with NDCG/MRR; one scalar for the whole funnel can hide which stage failed.

7.10 Generative and RAG evaluation

A wrong generated answer can come from retrieval, context selection or generation, so evaluation should decompose the pipeline instead of assigning one opaque score. For retrieval-augmented generation, useful questions are: did retrieval surface the necessary evidence, was the supplied context mostly useful, did the answer stay supported by that context, and did it answer the user.

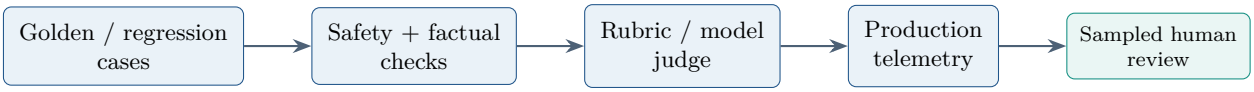


Figure 7.6: A generative evaluation stack combines deterministic regression cases, targeted checks, scalable semantic judging and sampled human review.

A **golden dataset** acts like a regression test suite: representative prompts are rerun whenever the system changes. A model-based judge can scale semantic evaluation, but it is still an instrument with its own bias and prompt sensitivity rather than ground truth. Human labels therefore remain useful for calibration and for failure modes that automated judges systematically miss.

RAGAS-style evaluation names useful RAG dimensions such as context precision/recall, answer relevancy and faithfulness. HELM-style benchmarking instead asks broader base-model capability questions across standardised scenarios. Treat such frameworks as structured metric suites rather than substitutes for deployment-specific failure analysis.

Memory Hook

For RAG, separate "did we retrieve the evidence?" from "did the generator use the evidence correctly?".

7.11 Online experiments and A/B testing

Offline improvement does not prove that deploying a new system changes user outcomes. Random assignment turns the deployment comparison into a causal experiment:

$$\underbrace{\widehat{\Delta}}_{\text{estimated treatment effect}} = \underbrace{\bar{y}_{\text{treatment}}}_{\text{new system}} - \underbrace{\bar{y}_{\text{control}}}_{\text{old system}} .$$

Persistent assignment, such as a stable hash of user identity, keeps one user in one condition and avoids mixing treatment exposure.

The null hypothesis is commonly $H_0 : \Delta = 0$. A p-value asks how extreme the observed statistic would be under the null; it is not the posterior probability that the null is true. The standard-error scaling in Section 1.4.4 gives the useful sample-size intuition:

$$\underbrace{\text{MDE}}_{\text{smallest reliably detectable effect}} \propto \frac{1}{\sqrt{N_{\text{independent cases}}}} \quad \text{(all else equal)}.$$

Halving the target MDE therefore costs roughly four times as many independent observations under the same noise/decision assumptions.

	Test says no effect	Test says effect
No real effect	Correct	Type I error
Real effect	Type II error	Correct detection

Repeatedly checking an ordinary fixed-horizon p-value and stopping the first time it crosses a threshold changes the testing procedure and inflates false-positive risk. Use a predeclared horizon or a sequential method whose stopping rule is part of the design.

Ranking systems can also use **interleaving**: mix candidates from two rankers into one list so the same user/context provides a sensitive relative comparison. Interleaving is useful for direct ranker preference but does not replace product-level A/B metrics such as retention or revenue.

Memory Hook

Randomisation isolates the intervention; sample size controls how finely the effect can be measured.

Degree of Experiment

Vary assignment unit, traffic split, metric, MDE, test duration, stopping rule, segmentation and whether competing ranking systems are compared by A/B assignment or interleaving.

SECTION 8

Transfer Learning, Domain Adaptation and Few-/Zero-Shot Learning

Reuse is useful when a previous model has already learned structure that survives into the new task or domain. The design question is therefore *what is shared, what changed, and how much of the model should be allowed to adapt?*

8.1 Transfer learning — reuse what the model has learned

A pretrained encoder can be frozen, partially updated, fully fine-tuned, or adapted through a small parameter-efficient module. More trainable capacity allows more target-specific change but increases data/compute requirements and overfitting risk. The architectural assumptions being transferred are summarised in Section 4.12.

Strategy	What changes?	Main trade-off
Frozen encoder / linear probe	New prediction head only	Cheap; representation cannot adapt
Partial fine-tuning	Head + selected pretrained layers	More flexibility for moderate cost
Full fine-tuning	Most pretrained parameters	Most flexible; highest overfit/compute risk
Parameter-efficient adaptation	Small modules / low-rank updates	Restricts the directions of change

Table 8.1: Transfer learning is a continuum of adaptation capacity.

Negative transfer means reuse performs worse than an appropriate non-transfer baseline. Source/target similarity is not enough: the source objective may have discarded exactly the feature the target task needs.

Practical Tip

With little labelled target data, start with a frozen encoder + linear probe before paying the variance/-compute cost of broader fine-tuning.

8.2 Domain adaptation — the data distribution changes

The useful distinction is which probability relationship changed:

$$\underbrace{P_S(x) \neq P_T(x),\ P_S(y \mid x) = P_T(y \mid x)}_{\text{covariate shift: inputs change, rule stays}}, \tag{8.1}$$

$$\underbrace{P_S(y) \neq P_T(y),\ P_S(x \mid y) = P_T(x \mid y)}_{\text{label shift: class prevalence changes}}, \tag{8.2}$$

$$\underbrace{P_S(y \mid x) \neq P_T(y \mid x)}_{\text{concept shift: prediction rule changes}}. \tag{8.3}$$

Real deployment change can combine these idealised cases.

Under pure covariate shift with support overlap, source examples can be reweighted to represent the target input distribution:

$$\underbrace{\mathbb{E}_{P_T}[\ell]}_{\text{target risk}} = \underbrace{\mathbb{E}_{P_S}\left[\frac{P_T(x)}{P_S(x)}\ell(f(x), y)\right]}_{\text{source risk weighted toward target-like inputs}}.$$

If important target regions have tiny source probability, those weights become unstable and source-only adaptation has little evidence to work with. Representation alignment is another option, but matching marginal features does not guarantee that classes remain correctly aligned. Validate under the actual deployment shift rather than a random row split; see Section 7.6.

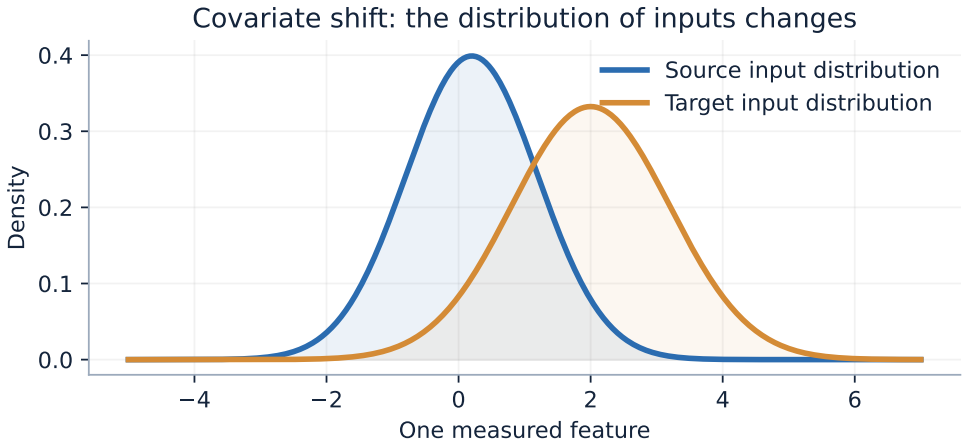


Figure 8.1: Synthetic covariate shift in one input coordinate; the unchanged conditional label relation is an assumption, not something this marginal plot proves.

Memory Hook

Reweighting can fix input-frequency shift; it cannot fix a changed input-to-label rule.

8.3 Few-shot learning — learn from very few examples

In the usual "N-way K-shot" terminology, an episode contains a small support set for each candidate class plus query examples for evaluation. Few-shot learning can mean metric learning, gradient-based adaptation, frozen embeddings, or in-context prompting; it is not one algorithm.

A Prototypical Network reduces each class support set to a centroid in learned embedding space:

$$\underbrace{c_{\text{class } j}}_{\text{class prototype}} = \frac{1}{N_{\text{support per class}}} \sum_{x_i \in S_j} f_\theta(x_i), \qquad \hat{y}_{\text{query}} = \arg\min_j d(f_\theta(x_{\text{query}}), c_{\text{class } j}) .$$

This works only if the learned geometry makes same-class examples cluster in a transferable way.

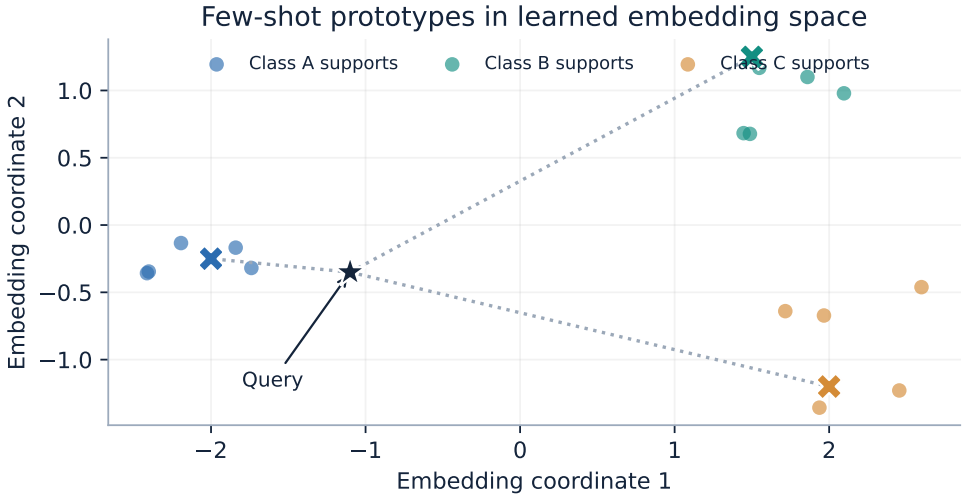


Figure 8.2: Few labelled support examples define class prototypes; a query is classified by embedding-space proximity.

8.4 Meta-learning and MAML

Meta-learning trains across tasks so that a new related task can be adapted quickly. MAML makes that idea explicit by optimising query performance *after* a task-specific support-set update:

$$\min_{\theta} \sum_{j=1}^{N_{\text{tasks}}} \mathcal{L}_j^{\text{query}} \left(\underbrace{\theta - \alpha \nabla_\theta \mathcal{L}_j^{\text{support}}(\theta)}_{\theta'_j\text{: one task-adaptation step}} \right).$$

Exact differentiation through the inner update introduces second-order dependence; first-order variants drop that cost. The Hessian/second-order connection is in Sections 1.4.2 and 5.9.

8.5 Zero-shot learning — no task-specific labelled examples

Zero-shot learning solves a target task without labelled target-task examples in that adaptation setting. For a shared image/text embedding, classification can be "choose the text description with highest cosine similarity":

$$\hat{y} = \arg\max_j \underbrace{\frac{f_\theta(x)^\top g_\phi(t_j)}{\|f_\theta(x)\|_2 \|g_\phi(t_j)\|_2}}_{\text{image–text directional similarity}}.$$

Performance therefore depends on the pretraining semantics, candidate class set and prompt wording. Zero-shot does not mean zero prior knowledge.

Memory Hook

Prompting changes context; fine-tuning changes persistent parameters.

8.6 LoRA: a parameter-efficient adaptation mechanism

LoRA freezes a pretrained weight matrix and learns a low-rank update:

$$\underbrace{\theta'_{\text{weights}}}_{\text{adapted matrix}} = \underbrace{\theta_{\text{weights}}}_{\text{frozen pretrained matrix}} + \underbrace{\theta_{\text{up}}\theta_{\text{down}}}_{\Delta\theta\text{: low-rank update}}.$$

Its shapes expose the bottleneck:

$$\underbrace{\theta_{\text{down}}}_{\text{compress}} \in \mathbb{R}^{N_{\text{adaptation directions}} \times N_{\text{input features}}}, \qquad \underbrace{\theta_{\text{up}}}_{\text{expand}} \in \mathbb{R}^{N_{\text{output features}} \times N_{\text{adaptation directions}}}.$$

The parameter saving is the comparison

$$\underbrace{N_{\text{adaptation directions}} \left(N_{\text{input features}} + N_{\text{output features}} \right)}_{\text{trainable LoRA parameters}} \quad \text{vs.} \quad \underbrace{N_{\text{input features}} N_{\text{output features}}}_{\text{full matrix parameters}}.$$

The underlying assumption is that useful task adaptation lives in relatively few independent directions; the rank/span prerequisite is in Section 1.1.1. Low trainable-parameter count does not imply every training/inference cost shrinks proportionally.

8.6.1 QLoRA: quantised base plus low-rank adaptation

QLoRA combines two different restrictions: keep the pretrained base model in a low-bit representation to save memory, while training LoRA adapters for task-specific change. Conceptually,

$$\underbrace{\text{quantised frozen base}}_{\text{cheap storage}} + \underbrace{\text{trainable low-rank update}}_{\text{adaptation capacity}}.$$

Quantisation and low-rank adaptation solve different bottlenecks, so they compose naturally. Quantisation mechanics and their quality–memory trade-off are developed in Section 14.1.

Memory Hook

QLoRA compresses the frozen base; LoRA restricts the trainable change.

8.7 Simulation-to-real adaptation for scientific ML

Simulation-trained models fail on real data when the simulator misses deployment-relevant noise, calibration, physics or operating regimes. Domain randomisation deliberately varies nuisance factors; calibration makes the simulator more faithful; scarce real examples can support fine-tuning; known invariances can be built into architecture or loss. Domain randomisation helps only when its variation covers the real shift that matters. This is an inductive-bias choice (Section 9.4) and should be checked with deployment-matched validation.

Inductive Bias

Approach	Shared structure assumed	Failure to test
Frozen transfer	Source representation retains target signal	Target feature was suppressed upstream
Fine-tuning	Pretrained parameters are a useful starting point	Small target set overfits
Domain invariance	Some task-relevant features persist across domains	Alignment mixes different classes
Prototypical networks	Classes form meaningful embedding centroids	Classes are multimodal/overlapping
MAML	New tasks resemble meta-training tasks	Task family changes substantially
Zero-shot multimodal	Modalities share semantic geometry	Prompt/class semantics are ambiguous
LoRA	Adaptation is approximately low rank	Task needs broad independent parameter changes

SECTION 9

No Free Lunch and the Limits of Learning

Generalisation is impossible without assumptions about which unseen worlds are more plausible than others. No Free Lunch (NFL) makes that dependence explicit.

9.1 Why finite observations are not enough

The observations $(1, 2)$, $(2, 4)$ and $(3, 6)$ fit both

$$f_1(x) = 2x, \qquad f_2(x) = 2x + (x-1)(x-2)(x-3),$$

yet the functions disagree at unseen inputs. Choosing the linear continuation is therefore a preference for a simpler/regular pattern, not a logical consequence of the sample alone.

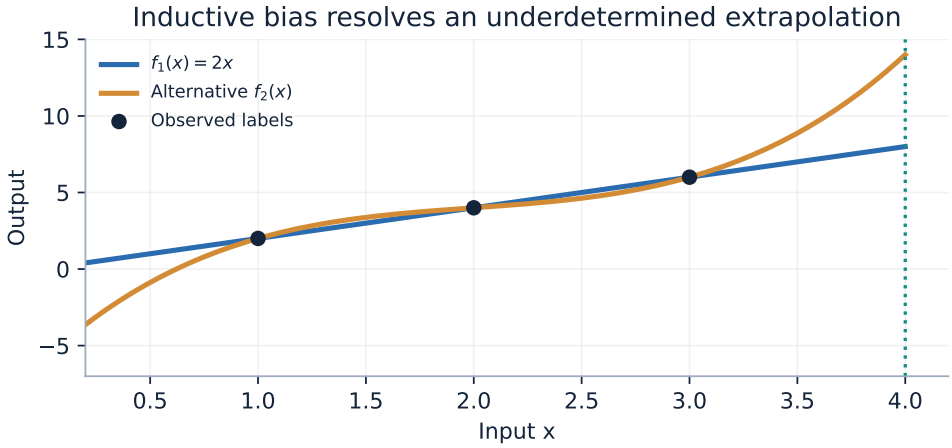


Figure 9.1: Two functions can fit the same observations perfectly and disagree elsewhere; prediction requires a preference among compatible hypotheses.

9.2 A mathematical formulation of No Free Lunch

For N_{inputs} possible inputs and N_{labels} possible labels, there are $N_{\text{labels}}^{N_{\text{inputs}}}$ possible labelling functions. Under a uniform prior over all of them, observing labels at some inputs gives no information about an unseen input:

$$\boxed{P(f(x_{\text{unseen}}) = y_j \mid D) = \frac{1}{N_{\text{labels}}}}.$$

Under zero–one loss, every fixed learner therefore has the same average unseen accuracy under that very specific uniform-function averaging setup. This is a statement about the averaging assumptions, not a claim that all algorithms perform equally on real structured datasets.

9.3 Two rules, two compatible worlds

If observations are compatible with both "predict from feature 1" and "predict from feature 2", unseen examples can separate the rules even though training data cannot. The learner must prefer one rule before evidence at the unseen point exists.

Intuition

NFL says performance advantages come from assumptions about which problems are likely, not from an assumption-free learner.

9.4 Inductive bias — the assumptions that make learning possible

An inductive bias is any preference that changes predictions beyond what the observed examples logically force. Linear models prefer affine structure; KNN prefers local similarity; CNNs prefer repeated local patterns; sequence models prefer useful state/history; physics-informed models prefer stated physical constraints. Architecture examples are collected in Section 4.12, with tree-specific assumptions in Section 10.7. Bias helps when it matches the task population and hurts when it excludes useful structure.

9.5 Bayesian priors: a direct form of inductive bias

Bayesian inference states the preference explicitly through $p(f)$:

$$p(f \mid D) \propto \underbrace{p(D \mid f)}_{\text{fit observations}} \quad \underbrace{p(f)}_{\text{prior preference}} .$$

The MLE/MAP parameter view is in Section 1.3.2.

A squared-exponential Gaussian-process kernel encodes the assumption that nearby inputs should have correlated function values:

$$\underbrace{k(x, x')}_{\text{prior function similarity}} = \underbrace{\sigma^2_{\text{function}}}_{\text{prior output scale}} \exp\left(-\frac{\|x - x'\|_2^2}{2\,\underbrace{\ell^2_{\text{length scale}}}_{\text{distance over which similarity decays}}} \right).$$

This is another kernel-defined similarity, linked conceptually to Section 4.3; the probabilistic role is different, but the kernel still encodes which inputs should behave similarly.

9.6 Occam's razor is also a bias

"Prefer simple explanations" is useful only after choosing what simplicity means. A periodic physical law can be compact in a Fourier representation and awkward in a polynomial one. L1, L2, architecture constraints and description-length principles therefore encode different simplicity preferences rather than one universal notion; see Section 6.4.

9.7 Why statistical learning guarantees do not contradict NFL

Generalisation bounds add assumptions that NFL deliberately averages away: a sampling process, a restricted hypothesis family, a loss, and a confidence statement. The useful scaling intuition is

$$\underbrace{\text{generalisation gap}}_{\text{population error}-\text{training error}} \sim \sqrt{\frac{\underbrace{\text{model-class complexity}}_{\text{more hypotheses cost more evidence}} + \underbrace{\text{confidence penalty}}_{\text{stronger guarantee costs evidence}}}{N_{\text{examples}}}}.$$

The exact theorem changes with the assumptions and complexity measure. The important point is that guarantees are conditional on structure; they do not contradict an assumption-free averaging theorem.

9.8 Optimisation NFL is related but separate

Optimisation NFL results average search algorithms over broad objective-function families under their own assumptions. Practical optimisers exploit gradients, curvature, smoothness and architecture-specific structure, so they are not being compared under an assumption-free real-world distribution. The optimiser evolution in Section 5.8 is therefore a sequence of useful structural bets, not a contradiction of NFL.

9.9 Connection to scientific machine learning

Scientific ML often has legitimate prior structure: symmetries, topology, conservation laws, known dynamics, simulator structure or measurement constraints. Encoding those assumptions can improve data efficiency and extrapolation when they are correct. An incorrect constraint can instead create systematic error, so the right question is not “bias or no bias” but “which bias matches the physical setting?” See simulation-to-real adaptation in Section 8.7 and hybrid physical/tree modelling in Section 10.7.1.

SECTION 10

Tree-Based Models and Ensembles

A decision tree models a prediction through successive conditional rules. Tree ensembles then address different weaknesses of individual trees: bagging reduces instability by averaging diverse fits, whereas boosting adds functions that improve a loss. This section develops their data flows, inductive biases, design choices and research trade-offs. For the broader role of inductive bias, see Section 9.4; for the compact model-family comparison, see Section 4.4.

10.1 A single decision tree: rules become regions

Consider predicting machine-failure risk from temperature T and vibration V. A tree might apply "$V > 6$?", then (if not) "$T > 70$?", with a class or risk estimate at the end of each path. Inference consists of traversing one path from the root to a *leaf*; no optimisation happens at inference time. The thresholds are learned during training, not specified by an expert in the standard algorithm.

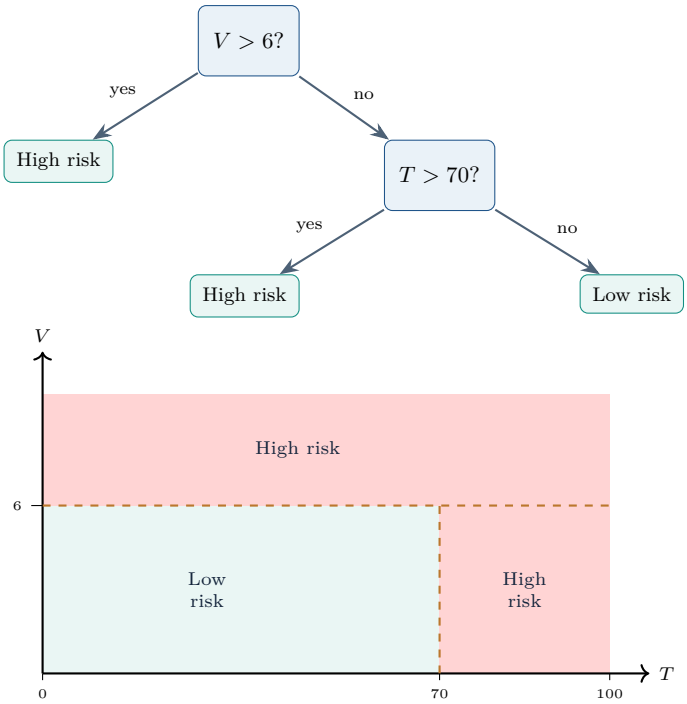


Figure 10.1: The same decision tree as a rule hierarchy (top) and a feature-space partition (bottom). Every rectangular leaf region corresponds to one terminal prediction.

A conventional **axis-aligned** tree splits on one feature at a time, $x_j \le \theta_{\text{threshold}}$ versus $x_j > \theta_{\text{threshold}}$. Its leaves divide feature space into N_{leaves} disjoint regions. A standard constant-leaf regression tree is

$$\hat{f}(x) = \sum_{j=1}^{N_{\text{leaves}}} \underbrace{\theta_{\text{leaf},j}}_{\text{constant prediction in leaf } j} \mathbb{1}[x \in R_j]. \tag{10.1}$$

Under squared-error loss, each $\theta_{\text{leaf},j}$ is usually the mean training target of the observations routed to that leaf. A classification leaf can return the most frequent class or a vector of empirical class proportions. Small-leaf proportions are not necessarily calibrated probabilities. A tree therefore models a *piecewise-constant function*, with conditional feature interactions created by its hierarchy: in Figure 10.1, temperature matters only for the branch with $V \le 6$.

10.1.1 Learning a split

At a node, a greedy learner generates allowable feature–threshold candidates, measures the reduction in a node criterion, selects the candidate with greatest gain, then recursively repeats on its children until a stopping rule applies. The training process is a sequence of discrete structural decisions, rather than ordinary gradient descent through split thresholds.

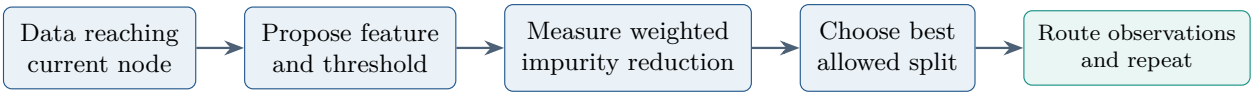


Figure 10.2: Recursive tree fitting: a local split is chosen before the resulting subproblems are solved. A stopping rule instead produces a leaf prediction.

For N_{classes} classes, a common impurity is **Gini impurity**, $G = 1 - \sum_{j=1}^{N_{\text{classes}}} p_{\text{class } j}^2$. Gini is zero in a pure node; in balanced binary classification it is 0.5. Suppose eight examples have four positives and four negatives, so $G_{\text{parent}} = 0.5$. A candidate split creates a five-sample child with class counts $(4, 1)$ and a three-sample pure child. The five-sample impurity is $1 - (4/5)^2 - (1/5)^2 = 0.32$, giving

$$\boxed{\Delta G = 0.5 - \left(\frac{5}{8}\,0.32 + \frac{3}{8}\,0\right) = 0.30.} \tag{10.2}$$

A split is useful because it creates more concentrated class distributions. *Entropy* $-\sum_j p_{\text{class } j} \log p_{\text{class } j}$ is another classification criterion. Regression trees usually select splits that reduce within-node squared error/variance, or another specified regression objective; MAE-based methods naturally use median leaf predictions. These are alternative definitions of *good partition*, not unrelated model families. See the loss-function discussion in Sections 3.2 and 3.3.

Intuition

A tree learns a conditional partition of feature space. Standard leaves output constants; complex global predictions arise by routing inputs to different local regions.

10.2 Tree structure, greedy search and regularisation

10.2.1 Greedy splits need not find the best tree

Local optimisation can miss feature interactions. For balanced XOR data, $y = x_1 \oplus x_2$, each binary input feature alone leaves a 50–50 class mixture in both children. The first split achieves zero Gini gain, even though a depth-two tree can represent XOR exactly after splitting on both features. A greedy implementation that insists on positive immediate gain may never find it. This separates *representational capacity* from *optimisation ability*.

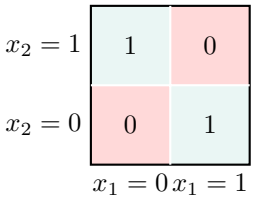


Figure 10.3: Balanced XOR: neither first-level split helps on its own, but two levels perfectly separate the classes.

10.2.2 Depth, leaves and pruning

A binary tree with maximum depth N_{depth} has at most $2^{N_{\text{depth}}}$ leaves; depth 1 is a *decision stump*. More leaves can approximate increasingly local structure, but individual leaf estimates may rely on few observations. This is the bias–variance trade-off already introduced in Section 7.7.

Control	What it constrains	Trade-off
Maximum depth	Number of sequential decisions	Less depth limits interactions; greater depth can fit noise
Minimum samples per leaf	Evidence supporting each local estimate	Larger leaves stabilise predictions but smooth local structure
Minimum samples to split	Eligibility to expand a node	Earlier stopping versus finer partitions
Maximum leaf count	Number of distinct terminal regions	Direct budget for partition complexity
Minimum split improvement	Gain required to add a split	Reject weak gains, possibly including useful precursors
Pruning strength	Number of retained branches after growing	Simpler tree versus potentially discarded genuine structure

Pre-pruning prevents a split when a limit is reached; **post-pruning** removes branches from a larger fitted tree. Cost-complexity pruning makes the trade-off explicit:

$$\underbrace{J(T)}_{\text{selection objective}} = \underbrace{L_{\text{train}}(T)}_{\text{fit}} + \underbrace{\alpha N_{\text{leaves}}(T)}_{\text{complexity cost}} . \tag{10.3}$$

Each extra leaf must therefore buy enough training improvement to justify its penalty. Pruning is not a complete solution to greedy-search errors: it simplifies already-discovered structures, rather than exploring every possible tree.

10.2.3 Growth strategy and split geometry

Level-wise growth expands eligible nodes layer by layer. **Best-first/leaf-wise** growth expands the current leaf promising the greatest loss reduction. Leaf-wise growth concentrates complexity in difficult regions but can create deep low-support paths; level-wise growth allocates depth more evenly. Level-wise does *not* imply symmetric splitting: nodes at the same depth can still use different tests.

An axis-aligned split tests one feature, such as $x_j > \theta_{\text{threshold}}$; an **oblique split** tests $\theta_{\text{split}}^{\top} x > \theta_{\text{threshold}}$. A diagonal boundary, such as $T + 2V > 80$, may require many staircase-like axis-aligned regions but only one oblique split. Oblique splits increase geometric expressivity and typically make split search harder.

Inductive Bias

If the target is well described by hierarchical single-feature thresholds and locally constant regions, standard trees represent the structure efficiently. If the boundary is smooth or strongly rotated, they may need many leaves; deeper trees add expressivity while increasing estimation variance.

Depth and leaf support are related but distinct controls.

Memory Hook

Depth limits interaction length; minimum leaf size limits evidence per local prediction.

10.3 Bagging, Random Forests and ExtraTrees

A single tree can be unstable: removing or changing a few observations may change a high-level split and many downstream predictions. **Bagging** (bootstrap aggregating) reduces this variation by fitting multiple trees to bootstrap samples and aggregating their outputs.

Each bootstrap sample draws N_{examples} observations *with replacement* from N_{examples} originals. For large N_{examples}, about 63.2% of originals appear at least once and about 36.8% are out-of-bag for that tree. Averaging regression predictions or class probabilities reduces variation when the trees' errors are not perfectly correlated. Majority voting is another classification aggregation rule.

For N_{trees} trees with common prediction variance σ^2 and pairwise correlation ρ, the equal-correlation model gives

$$\text{Var}(\bar{f}) = \underbrace{\underbrace{\sigma^2}_{\text{single-tree variance}} \underbrace{\rho}_{\text{pairwise error correlation}}}_{\text{shared variance: averaging cannot remove}} + \underbrace{\sigma^2 \frac{1-\rho}{N_{\text{trees}}}}_{\text{unshared variance: shrinks with more trees}} . \tag{10.4}$$

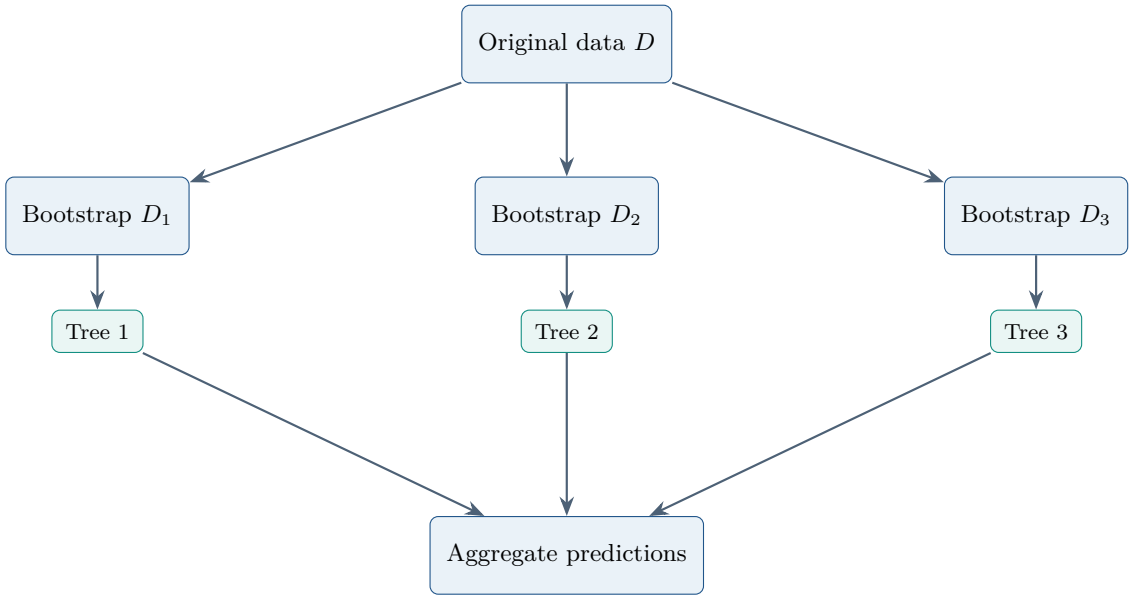


Figure 10.4: Bagging: independent bootstrap-fitted trees are combined at prediction time. Individual fits can be trained in parallel.

More trees only reduce the unshared component; decorrelation is therefore as important as tree count. This is a simplified equal-variance/equal-correlation model, not a universal forest error law; connect it to the broader bias–variance picture in Section 7.7.

Random Forests add random selection of eligible features *at each split*, commonly together with bootstrapping. This decorrelates trees that might otherwise make the same dominant early choices. Too much randomisation weakens individual trees, so diversity and individual predictive quality trade off. **ExtraTrees** also randomise candidate thresholds and often use the original sample rather than bootstrapping by default. The extra randomness can improve diversity and split-search efficiency while increasing individual-tree bias.

Method	Randomness	Primary idea
Bagged trees	Resample training rows	Average unstable learners
Random Forest	Resample rows; random features per node	Reduce correlation between learners
ExtraTrees	Random candidate thresholds; often random features	Additional split diversity, often less split-search work

Out-of-bag (OOB) evaluation predicts each training observation using only trees whose bootstrap samples omitted it. It offers an internal performance estimate under suitable sampling assumptions, but does not fix temporal or grouped leakage (Sections 7.6 and 7.8).

Memory Hook

Random Forests win by averaging diversity; more trees cannot remove shared error.

10.4 Boosting: learning successive corrections

Where bagging trains diverse trees independently, **boosting** trains a sequence of weak or limited-complexity learners that collectively improve a predictor. **AdaBoost** reweights training examples according to previous classification errors; hard examples gain relative emphasis. This can be useful, but mislabeled examples are also hard and may attract excessive attention.

10.4.1 Gradient boosting as functional optimisation

Gradient boosting updates the current predictor by adding a new correction tree:

$$\underbrace{F_t(x)}_{\text{updated ensemble}} = \underbrace{F_{t-1}(x)}_{\text{current ensemble}} + \underbrace{\eta}_{\text{shrinkage}} \underbrace{h_t(x)}_{\text{new correction tree}}. \tag{10.5}$$

Each stage improves the current ensemble rather than refitting the original target from scratch.

For squared-error regression, suppose the current model predicts 5 for targets $(3, 5, 7)$. The residuals are $(-2, 0, 2)$. If a correction tree fits these residuals exactly and $\eta = 0.5$, the updated predictions become $(4, 5, 6)$. The ensemble improves through a series of moderate adjustments.

More generally, each training example supplies the prediction-space direction

$$\underbrace{r_{i,t}}_{\text{pseudo-residual}} = - \underbrace{\frac{\partial L(y_i, F)}{\partial F}}_{\text{loss slope in prediction space}} \Bigg|_{F=F_{t-1}(x_i)} . \tag{10.6}$$

A tree approximates this direction as a function of inputs; its terminal output values may subsequently be optimised for the specified loss. Squared-error pseudo-residuals are ordinary residuals (up to loss convention). For binary logistic loss differentiated with respect to logits, the negative gradient is label minus predicted probability. This connects tree boosting to loss design in Sections 3.2 and 3.3.

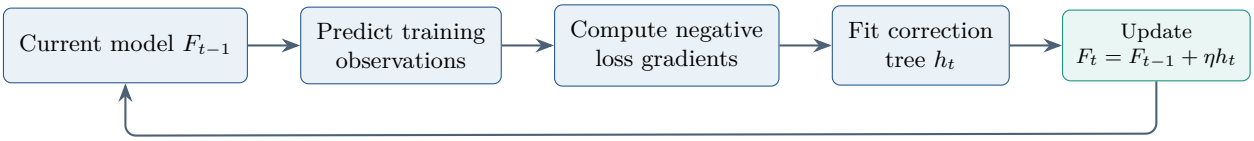


Figure 10.5: Gradient boosting is stage-wise optimisation in prediction-function space: fit a correction, add it, recompute the next direction.

This is related to gradient descent but not identical to backpropagating through a fixed neural network. Neural-network training typically updates continuously parameterised weights within an existing architecture. Boosting adds a *new fitted function* at each iteration; hard tree split decisions are not usually differentiated end to end. The SGD and adaptive-optimiser discussion in Sections 5.2 and 5.6 concerns a different parameterisation of the optimisation problem.

10.4.2 Ranking-aware boosting: LambdaMART

A ranking metric such as NDCG changes only when item order changes, so it is awkward as a direct smooth training loss (Sections 3.4 and 7.9). LambdaRank-style training instead creates pairwise correction signals and weights them by the ranking consequence of swapping the pair. If reversing two items would greatly change NDCG, that ordering error receives a stronger update than a swap deep in the list with almost no metric effect.

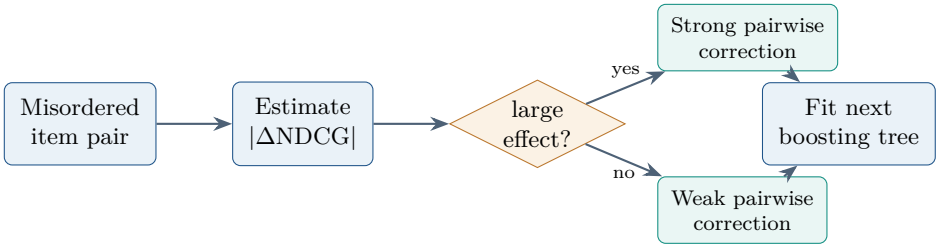


Figure 10.6: LambdaMART keeps the stage-wise tree-boosting mechanism but makes the correction signal sensitive to where a ranking mistake occurs.

LambdaMART combines these ranking-aware "lambda" updates with MART-style gradient-boosted regression trees. The important conceptual change is not a new tree primitive: it is a new objective signal layered onto the boosting machinery already developed above.

Memory Hook

LambdaMART is boosting where high-impact ranking mistakes get stronger correction signals.

10.4.3 Curvature and second-order updates

Second-order tree boosting also uses curvature of the loss with respect to the current prediction. For one constant-output leaf, a common local quadratic update is

$$\underbrace{\theta_{\text{leaf update}}}_{\text{constant correction in this leaf}} = -\frac{\underbrace{\sum_{i\in\text{leaf}} \frac{\partial L_i}{\partial F_i}}_{\text{accumulated loss slope}}}{\underbrace{\sum_{i\in\text{leaf}} \frac{\partial^2 L_i}{\partial F_i^2}}_{\text{prediction-space curvature}} + \underbrace{\lambda_{\text{leaf}}}_{\text{shrink large leaf updates}}}. \tag{10.7}$$

This is scalar curvature in prediction space, not the full parameter Hessian; compare Sections 1.4.2 and 5.9.

10.4.4 Learning rate, depth and early stopping

Smaller η gives more conservative corrections, normally requiring more boosting stages. Large η can fit quickly but may overstep or fit noise. The *learning rate and number of trees must be tuned together*; doubling tree count does not exactly undo halving η, because the intermediate ensemble and all subsequent correction targets change.

A depth-one stump contributes a function of one feature; an additive ensemble of such stumps cannot represent genuine cross-feature interactions on its raw score scale without richer components. Depth-two and deeper trees can encode conditional interactions within each stage. Validation-based early stopping selects the iteration before continuing improvement on the training loss ceases to help generalisation (Sections 6.7 and 7.8).

Design	Bagging / Random Forest	Gradient boosting
Tree dependence	Trees independently fitted	Each stage depends on preceding model
Combination	Average or vote	Add weighted corrections
Main benefit	Stabilise high-variance predictors	Build fit through stage-wise loss reduction
Typical trees	Often deep	Often shallow or leaf-limited
Parallelism	Trees can be trained concurrently	Boosting stages are sequential
Important controls	Tree count, randomness, leaf size	Learning rate, stages, leaf size, regularisation

Practical Tip

For gradient boosting, monitor validation loss across rounds and use early stopping. A falling training loss does not prove the next tree improves deployment performance.

10.5 XGBoost, LightGBM and CatBoost: what changed?

The historical progression is best seen as multiple related developments rather than one mandatory chain. The useful story is the accumulation of problems and fixes, not a claim that each method replaced the previous one.

Step	What changed	Why it was added	Consequence / new trade-off
CART	Recursive threshold partitioning	Represent nonlinear conditional rules	Flexible but unstable single trees
Bagging / Random Forest	Average diversified trees	Reduce variance from unstable fits	More compute; averaging cannot remove shared error
AdaBoost / Gradient Boosting	Add learners sequentially	Correct current mistakes or descend a loss	Strong staged fit; stages become dependent
XGBoost	Second-order updates, explicit penalties, scalable split search	Regularise and accelerate boosting	More design knobs and systems complexity
LightGBM / CatBoost	Selective computation, categorical handling, constrained growth choices	Improve efficiency or data handling	Extra approximation and inductive-bias choices

The modern libraries below share a core boosting framework but emphasise different implementation choices.

10.5.1 XGBoost: explicit regularisation and efficient optimisation

XGBoost popularised a scalable second-order tree-boosting system with explicit structural penalties. A representative objective separates predictive fit from structural/leaf penalties:

$$\underbrace{\sum_i L(y_i, F(x_i))}_{\text{predictive loss}} + \sum_{t=1}^{N_{\text{trees}}} \left(\underbrace{\gamma N_{\text{leaves},t}}_{\text{cost of adding leaves}} + \underbrace{\frac{\lambda}{2} \sum_{j=1}^{N_{\text{leaves},t}} \theta_{\text{leaf},t,j}^2}_{\text{shrink large leaf outputs}} \right). \tag{10.8}$$

The two penalties regulate structural complexity and correction magnitude separately. This is regularisation of tree structure/leaf outputs rather than a generic neural-network weight penalty; compare Section 6.4. Other design choices include row/column subsampling, missing-value-aware routing, approximate or histogram split search, and efficient memory/parallel processing. Leaf-count penalties and leaf-weight shrinkage regulate different forms of complexity.

10.5.2 Histogram split search

Searching every distinct numerical threshold can be expensive. A histogram learner maps numerical values to a finite set of bins, accumulates gradient/curvature statistics within each bin, and searches boundaries between bins. For example, $(1.1, 1.2, 1.5, 2.8, 3.0, 3.1, 4.9, 5.0)$ may be quantised into three bins containing $(1.1, 1.2, 1.5)$, $(2.8, 3.0, 3.1)$ and $(4.9, 5.0)$.

This reduces split-search and memory cost at the expense of threshold resolution. The approximation need not worsen validation performance, but its quality and runtime depend on bin count and the distribution of feature values. Histogram methods now appear in several GBDT libraries; they are not unique to LightGBM.

10.5.3 LightGBM: selective use of computation

LightGBM is associated with several efficiency choices:

- **Leaf-wise growth**: preferentially expand the leaf with greatest gain; efficient use of a leaf budget, but potentially deep low-support branches.
- **GOSS (Gradient-based One-Side Sampling)**: keep high-gradient observations, sample some low-gradient observations, then reweight as appropriate for split-gain estimation. This saves work while retaining influential gradient information; it is an available sampling strategy, not necessarily enabled in every configuration.
- **EFB (Exclusive Feature Bundling)**: bundle sparse features that are mutually or nearly mutually exclusive, reducing effective dimensionality. Exclusivity means seldom nonzero together, not merely low correlation.
- **Histogram-based processing**: quantise features and aggregate split statistics, reducing work relative to exhaustive threshold scans.

The common principle is to allocate compute to observations, features and regions where it is likely to matter. These approximations can alter accuracy and regularisation, and must be tested under equal resource budgets.

10.5.4 CatBoost: ordered categories and symmetric trees

Categorical features create a representation problem. One-hot encoding can greatly expand high-cardinality features. Global target encoding compresses them by replacing each category with a target statistic, but can leak an observation's own label into its input. For a category represented by one example, naive full-training target encoding can directly expose the answer.

CatBoost uses **ordered categorical statistics**: conceptually, an example's category statistic depends on earlier observations in a chosen ordering, not on its own target, together with smoothing and other implementation details. **Ordered boosting** is a distinct, related training procedure designed to reduce prediction shift arising from reused target information in gradient estimation. Neither idea removes the need for appropriate temporal and group validation; a random permutation is not a replacement for chronology.

CatBoost also characteristically uses **symmetric (oblivious) trees**, where all nodes at the same depth use the same test. This constrains the hypothesis class and supports efficient routing. Unlike merely *level-wise* trees, symmetric trees cannot choose separate splits at different nodes of one level. CatBoost also offers other growth policies.

	XGBoost	LightGBM	CatBoost
Emphasis	Scalable regularised boosting	Efficiency and selective allocation	Categories, prediction shift, symmetric trees
Split search	Exact/approximate/histogram options	Histogram-based	Quantised feature processing
Growth	Depth-wise or loss-guided variants	Leaf-wise characteristically	Symmetric by default, alternatives
Categories	Native handling or external encoding	Native handling or encoding	Ordered statistics and other transforms
Characteristic ideas	Second-order updates, sparse-aware handling, explicit penalties	GOSS, EFB, histogram subtraction	Ordered boosting, ordered category statistics

Practical Tip

Library names are not independent hypotheses: compare specific growth, categorical-handling, sampling and regularisation mechanisms under a fair validation and compute budget.

10.6 Research optionality: what can be changed?

A tree system has several distinct design layers: input representation, base-tree structure, ensemble mechanism, optimisation objective, sampling procedure and computational implementation. Different modifications target different limitations.

Table 10.1: Degrees of experimentation for tree-based methods.

Design axis	Representative alternatives	Mechanism to measure
Split geometry	Axis-aligned versus oblique	Fit to rotated boundaries; split count; compute
Split search	Greedy versus lookahead	Discover interactions with initially weak splits
Leaf outputs	Constant versus linear local models	Approximation of smooth local behaviour
Growth policy	Level-wise versus best-first	Leaf depths, support and loss reduction
Ensemble	Single tree, bagging, boosting	Stability versus progressive loss reduction
Loss	Squared, robust, logistic, quantile	Alignment to task and error costs
Sampling	Bootstrap, feature subsets, gradient-aware	Prediction correlation, training cost and variance
Regularisation	Pruning, leaf penalties, shrinkage	Complexity, noisy-fit sensitivity
Categories	One-hot, ordered statistics, native splits	High-cardinality generalisation and leakage
Split resolution	Exact values versus histogram bins	Speed–accuracy–memory trade-off

Design axis	Representative alternatives	Mechanism to measure
Uncertainty	Point, quantile, distributional predictions	Calibration and coverage under assumptions

A research improvement can come from a richer *hypothesis class* (oblique splits or linear leaves), better *optimisation* (lookahead or second-order updates), better *regularisation* (leaf penalties or shrinkage), better *representation* (categorical statistics), or better *systems engineering* (histograms and memory layout). These are different causal explanations of performance gains.

Three useful controlled experiments are: compare bagging with Random Forests while logging tree correlation and ensemble error; compare level-wise with leaf-wise growth under a matched leaf budget while measuring leaf support and depth; and compare first- with second-order boosting using matched objectives while reporting wall-clock cost *as well as* improvement per round.

Degree of Experiment

Vary split geometry/search, growth policy, leaf model, ensemble type, objective, row/feature sampling, regularisation, category handling, histogram resolution, uncertainty output, and compute budget.

10.7 Inductive bias and model selection

A tree's built-in preferences are: useful single-feature thresholds, hierarchical interactions, locally constant predictions, and adaptive refinement of regions. These assumptions can help with heterogeneous tabular features and abrupt conditional effects. They can be inefficient for oblique boundaries, smooth globally varying functions and behaviour far outside the training support (Section 9.4).

10.7.1 Extrapolation and feature transformations

Suppose the physical target is $y = 2x$ and training data only cover $0 \leq x \leq 10$. A linear model may extrapolate the same slope to $x = 15$. A standard regression tree routes every sufficiently large x to its terminal rightmost leaf and returns its constant estimate. A finite sum or average of such trees likewise stays constant beyond all its final thresholds along that coordinate, holding other features fixed. More trees improve within-support resolution but do not automatically recover the outside-support physical law.

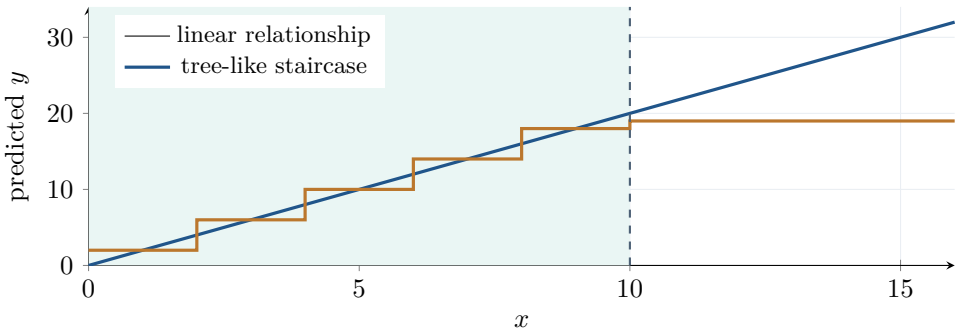


Figure 10.7: Schematic extrapolation: a constant-leaf tree saturates once an input lies beyond the last training-supported split; a linear function can continue its slope. Curves are illustrative, not fitted experimental results.

Because exact axis-aligned splits primarily depend on feature ordering, strictly monotonic transforms of a numerical feature preserve the representable partitions (after transforming thresholds), unlike a general rotation that mixes features. Practical fitted models may still change owing to histogram binning, ties and numerical detail. Standard scaling is often unnecessary for exact tree splits; it may still matter for models or preprocessing stages around the tree (Section 2.2).

Practical Tip

Known physical law plus unexplained residuals: investigate $\hat{y}(x) = f_{\text{physics}}(x) + f_{\text{trees}}(x)$. The physical term can encode extrapolative structure; the learned correction still needs out-of-domain validation.

Model family	Preferred structure	Notable limitation to test
Linear / additive model	Global linear or additive effects	Missing interactions and discontinuities
Single tree	Hierarchical threshold rules	Instability; staircase approximation
Random Forest	Average diverse local partitions	Correlated errors; extrapolation
Boosted trees	Sequential additive partition corrections	Overfit to residual noise; extrapolation
Neural network	Learned continuous representations	Data/optimisation and architectural bias
Physics-informed/hybrid	Governing laws plus fitted deviations	Error from incorrect scientific assumptions

10.8 Evaluation, explanation and failure modes

10.8.1 Evaluate the deployment task, not a random row split

In longitudinal data, measurements from one machine or person can be strongly dependent. Randomly dividing rows may put the same entity in training and test sets. For forecasting, training on later observations than those being tested can introduce future-information leakage. Choose chronological and/or grouped validation according to the actual deployment question (Sections 7.6 and 8.2).

Supervised feature engineering is another leakage route. Computing global category target means, performing label-dependent feature selection before splitting, or using a row's own label during its target encoding can all inflate apparent validation results. Fit transformations within the training fold, with cross-fitting or ordered encodings when appropriate. Missingness may itself be predictive, but changing collection practices can create shift; an integer category code should not silently become an ordinal physical quantity.

10.8.2 Metrics, importance and causal interpretation

Rare-event prediction should consider precision, recall and PR curves, not accuracy alone; log loss and calibration measure different properties from ranking AUC. Operating thresholds should reflect false-alarm and missed-event costs. See the worked definitions in Sections 7.2 to 7.4.

Three common explanations answer different questions:

- **Impurity importance** accumulates training-criterion reductions attributed to features; continuous/high-cardinality features can receive a structural advantage.
- **Permutation importance** measures held-out performance deterioration when one feature is shuffled; correlated predictors can substitute for one another, masking individual importance.
- **SHAP** assigns additive contributions to a prediction relative to a chosen reference/distributional convention; an attribution is not an intervention or an automatic causal effect.

Partial dependence averages predictions while changing a chosen feature; individual conditional expectation traces this for one observation. With strongly dependent features, intervening on one coordinate while holding others unchanged can generate unrealistic feature combinations. Such plots describe a fitted model, not necessarily the true data-generating mechanism.

Failure mode	Mechanism	Useful check
Overfitting	Tiny leaves; late boosting stages fit noise	Leaf limits, early stopping, held-out trends
Leakage	Targets or future/group information in features	Pipeline audit, grouped/time splits
Poor calibration	Predicted scores do not match event frequencies	Reliability curves; probability scores such as log loss or Brier score
Shift	Deployment population differs from training	Time/site holdouts; drift monitoring
Misleading importance	Correlated predictors or split-cardinality effects	Refit stability; grouped perturbations
Slow inference	Many trees, deep paths, expensive preprocessing	Profile end-to-end latency and memory
Extrapolation error	Terminal leaves return learned local values	Validate beyond training support; hybrid model

Memory Hook

Feature attribution describes model dependence, not causal mechanism.

10.9 Extensions and a complete research example

10.9.1 Beyond standard trees

Conventional trees are not the end of the design space:

- **Differentiable trees** soften routing, for example $p_{\text{right}}(x) = \sigma((\theta_{\text{split}}^{\top} x - \theta_{\text{threshold}})/\tau_{\text{routing softness}})$, enabling gradients through route probabilities. Small $\tau_{\text{routing softness}}$ approaches hard routing but may cause saturated gradients; soft routing can also make inference costlier.
- **Model trees** predict $\theta_{\text{leaf},j}^{\top} x + \theta_{\text{bias},j}$ inside region R_j rather than a constant. This fits smooth local effects more directly, but local parameter fitting and continuity across boundaries become design issues.
- **Quantile or distributional boosting** targets uncertainty rather than only a conditional mean. Conformal calibration can provide prediction sets when future cases behave like the calibration cases under the method's stated assumptions; coverage under shift still requires examination.
- **Constrained trees** may enforce monotonic effects or limit interactions, adding domain-specific bias. An incorrect constraint can create systematic error.
- **Neural–tree hybrids and tabular foundation models** change representation learning or transfer information across tasks rather than only improving the split search of a newly fitted tree.

10.9.2 Worked design: failure forecasting on repeated measurements

Suppose 500 machines produce repeated sensor measurements over several years. The inputs include numerical readings and categorical manufacturers; failures are rare; labels are noisy and inference must be fast. A defensible workflow is:

1. Define the unit of prediction (next-month failure per machine) and the information actually available at decision time.
2. Use chronological validation, and hold out complete machines as well if the deployment goal includes unseen machines. Repeated rows do not equal independent machines.
3. Fit a simple physical or parametric baseline, a regularised single tree, a Random Forest and a regularised GBDT; evaluate suitable categorical treatment without target leakage.
4. Compare minimum leaf size, tree complexity, boosting shrinkage and early stopping; report stability across machines, time windows and random seeds.
5. Evaluate PR behaviour, calibration and application-specific false-alarm costs, not accuracy in isolation.
6. Profile end-to-end inference latency and memory. Investigate whether manufacturer effects and feature importances remain stable across valid holdouts.

This experiment tests specific modelling assumptions as well as parameter values. The appropriate ensemble cannot be inferred from its library name alone.

Memory Hook

Trees partition; bagging averages diverse partitions; boosting adds loss-driven corrections.

10.10 Further reading

- Chen and Guestrin (2016), *XGBoost: A Scalable Tree Boosting System*: regularised boosting and scalable split search.
- Ke et al. (2017), *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*: GOSS and EFB.
- Prokhorenkova et al. (2018), *CatBoost: Unbiased Boosting with Categorical Features*: ordered statistics and boosting.

- Grinsztajn et al. (2022), *Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?*: comparative inductive-bias study with a limited benchmark scope.

SECTION 11

Architecture: CNNs

Section 4.6 introduced convolution's two core assumptions: useful patterns are local and the same detector can be reused across positions. This chapter extends that mental model into efficient vision systems, detection, temporal video and pretrained visual representations.

11.1 Convolution as a reusable local detector

A convolution does not need to be remembered as tensor-index arithmetic. Read one filter as a small detector swept over the image:

$$\underbrace{\text{feature response at one position}}_{\text{one output value}} = \sum \underbrace{\text{local pixels/features}}_{\text{small receptive patch}} \times \underbrace{\theta_{\text{shared filter}}}_{\text{same detector everywhere}} .$$

Weight sharing means the detector for an edge, texture or other local pattern is reused across spatial positions instead of relearned independently at every coordinate.

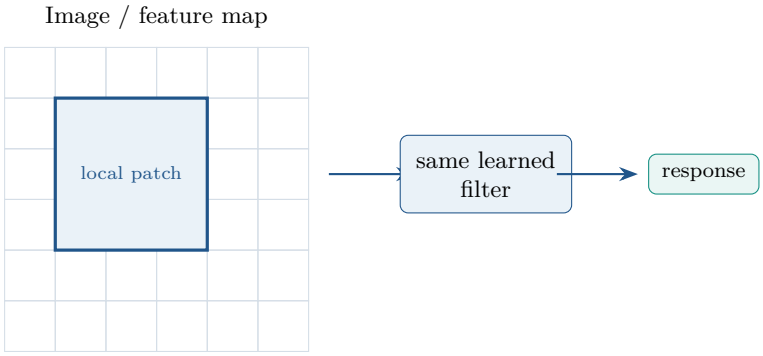


Figure 11.1: Convolution is easiest to read as one reusable local detector evaluated at many positions.

Channels are kinds of evidence. RGB begins with three maps. After a learned layer, the channels are better viewed as different feature maps: edge evidence, texture evidence, colour transitions, or more abstract learned patterns. Later layers mix these channels into progressively richer evidence.

Memory Hook

A CNN reuses local pattern detectors across space.

11.2 Receptive field, resolution and depth

A shallow unit sees a small patch; deeper units combine evidence from many earlier patches and therefore depend on a larger part of the input. Depth builds a hierarchy from local evidence toward larger-context evidence.

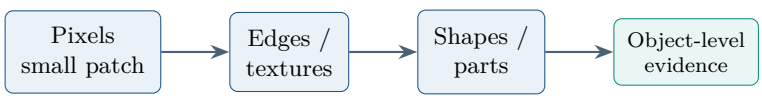


Figure 11.2: Stacking local operations increases effective context; the labels are a useful hierarchy intuition rather than fixed neuron semantics.

Stride and pooling reduce spatial resolution. The practical trade-off is

$$\underbrace{\text{high resolution}}_{\text{fine localisation, more memory/compute}} \longleftrightarrow \underbrace{\text{low resolution}}_{\text{cheaper, coarser spatial detail}} .$$

Detection and segmentation usually care more about preserving location than image-level classification.

11.3 Efficient convolution: separate spatial filtering from channel mixing

A standard convolution performs two jobs together: inspect local spatial structure and mix information across channels. A **depthwise-separable convolution** factorises them:

$$\boxed{\text{spatial filtering per channel} \longrightarrow 1 \times 1 \text{ channel mixing}}.$$

For a 3×3 layer with 64 input and 128 output channels, the rough per-position weight interaction count changes from

$$\underbrace{3 \times 3 \times 64 \times 128}_{73{,}728 \text{ standard}} \longrightarrow \underbrace{3 \times 3 \times 64 + 64 \times 128}_{8{,}768 \text{ factorised}}.$$

The reduction comes from a restriction: spatial and cross-channel reasoning are no longer represented by one unrestricted operation. MobileNet-style architectures exploit this trade-off for resource-constrained inference.

Inductive Bias

If spatial filtering and channel mixing can be approximately factorised, depthwise-separable convolution saves substantial compute; if their interaction must be richly coupled, the restriction can cost accuracy.

11.4 Classification, detection and cheap classical stages

Classification maps an image to an image-level label. Detection must additionally return locations, so its output is a set of boxes/classes/confidences rather than one class.

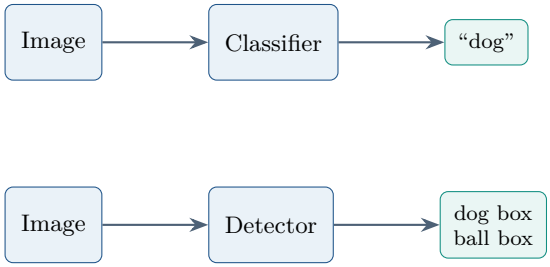


Figure 11.3: Classification asks what is present; detection asks what is present and where.

A learned model is not automatically the cheapest valid solution. For a strongly coloured marker, HSV thresholding plus contour geometry can provide a fast first-stage detector:

$$\text{RGB frame} \rightarrow \text{HSV} \rightarrow \text{colour threshold} \rightarrow \text{contours} \rightarrow \text{candidate location}.$$

A production cascade can therefore begin with deterministic image structure and use a neural detector only for harder appearance variation.

Practical Tip

Use the cheapest hypothesis class that captures the structure you actually need; a learned model should earn its extra flexibility.

11.5 Video: use time when single frames are ambiguous

Motion blur, occlusion or lighting can make one frame unreliable even when neighbouring frames are clear. A temporal stage uses nearby observations to stabilise the decision:

$$\underbrace{\hat{y}_t}_{\text{current decision}} = f\Big(\underbrace{x_{t-k:t}}_{\text{recent frames}} \Big).$$

The useful assumption is that the physical scene evolves continuously enough for neighbouring frames to provide evidence about the present.

Inductive Bias

If nearby frames describe one continuously evolving scene, temporal consistency can suppress transient visual noise; abrupt scene cuts or genuinely fast changes weaken that assumption.

11.6 Pretrained visual representations: CLIP-like versus DINO-like

The most important difference is the pretraining signal rather than the brand name. A CLIP-like system aligns an image encoder with a text encoder:

$$\text{image} \to z_{\text{image}} \quad \text{close to} \quad z_{\text{text}} \leftarrow \text{matching caption.}$$

Its representation is therefore pressured toward linguistically describable semantics. A DINO-like self-supervised visual encoder learns reusable image structure from transformed views without requiring paired captions. Both can become backbones for a small downstream classifier; Section 8.5 gives the zero-shot image–text connection.

Representation	Training pressure	Useful consequence
CLIP-like	Match paired image/text semantics	Language-aligned similarity and zero-shot prompts
DINO-like	Self-supervised consistency across image views	General visual features without captions

11.7 How vision systems evolved

The useful evolution is an accumulation of constraints and reusable representation learning rather than a replacement ladder.

Step	What changed	Problem targeted	New trade-off
Classical CV	Hand-written colour/geometry rules	Extremely cheap structured tasks	Brittle to appearance variation
CNN	Learn local shared filters	Hand-written feature limits	Training/data/compute cost
Residual/deeper CNN	Easier optimisation of depth	Need larger effective hierarchy	More compute
Depthwise conv.	Factorise spatial/channel work	Edge/mobile serving cost	More constrained layer
Pretrained encoder	Reuse broad learned representation	Limited labelled target data	Inherit pretraining bias
Multimodal/self-supervised	Broaden training signal	Reusable semantics without task labels	Larger pretraining systems

Degree of Experiment

Vary input resolution, kernel/receptive-field design, depth, width, standard versus factorised convolution, downsampling schedule, pretrained backbone, temporal context and deployment precision.

SECTION 12

Architecture: Transformers

Section 4.8 gave the compact architectural bias. This chapter builds the mechanism visually: token vectors are repeatedly rewritten using information routed from other token vectors.

12.1 One picture first: communication then computation

A decoder-only Transformer can be read as one repeated block:

$$\text{token vectors} \rightarrow \underbrace{\text{attention}}_{\text{communicate between tokens}} \rightarrow \underbrace{\text{MLP}}_{\text{compute within each token}} \rightarrow \text{updated token vectors}.$$

Residual connections preserve the previous state while each sublayer adds a learned correction.

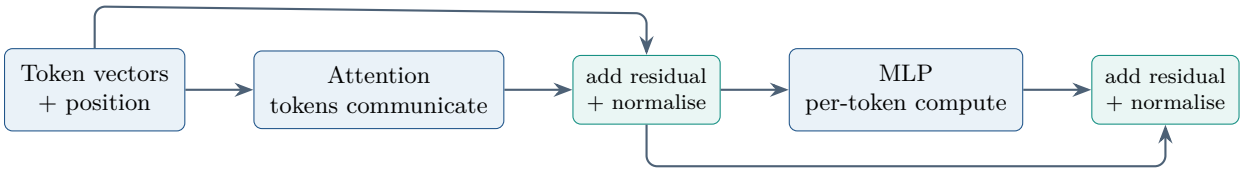


Figure 12.1: The reusable Transformer block: attention moves information between positions; the MLP transforms each position after that communication.

Memory Hook

Attention communicates; the MLP computes.

12.2 What actually flows through the network?

Words are tokenised, mapped to vectors and then progressively contextualised. The token "bank" can start from the same embedding in two sentences but move to different later representations after reading "river" or "deposit". The useful mental model is therefore not "words move through layers" but

$$\text{vector state} + \text{successive context-dependent corrections}.$$

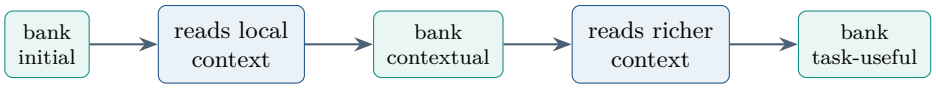


Figure 12.2: Stacked blocks progressively rewrite a token representation rather than assigning a fixed human-readable feature to each layer.

12.3 Attention before Q/K/V: choose information routes

Consider "The animal crossed the road because it was tired". For the token "it", useful context may come strongly from "animal" and weakly from other positions. Attention assigns routing weights, then mixes the selected information.

The attention map is not a guaranteed causal explanation of the model's final prediction. It is useful here as a picture of the mechanism: rows choose where information is read from.

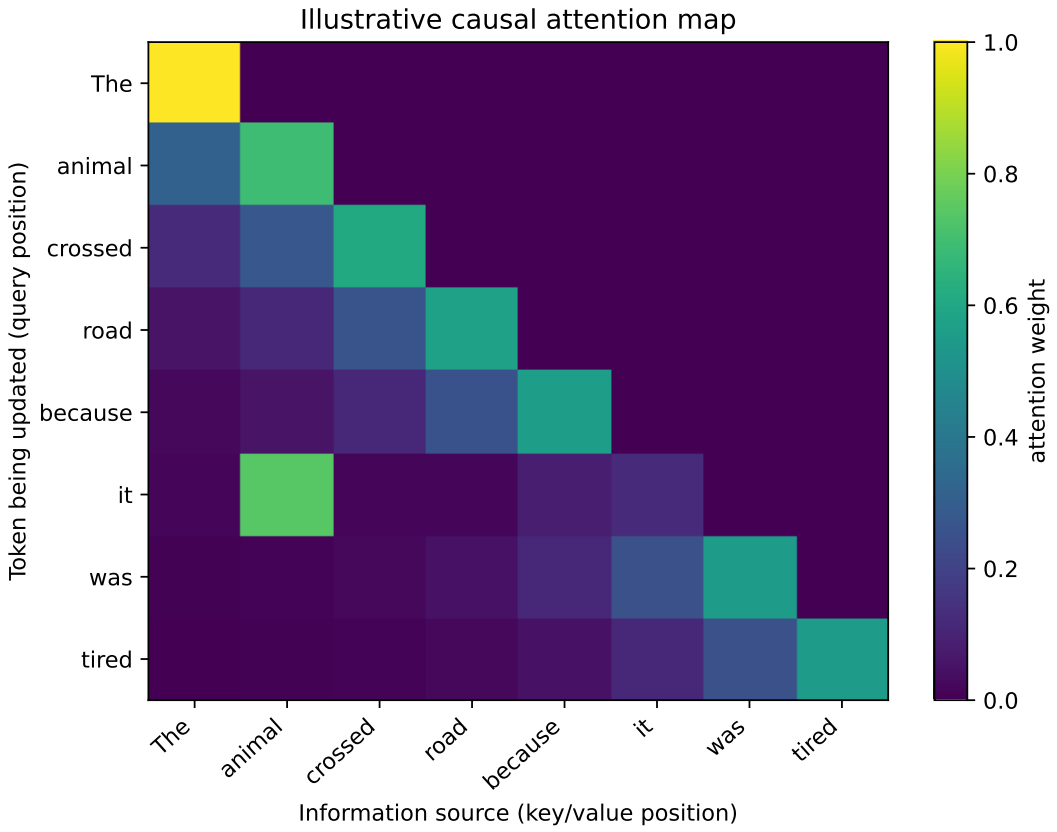


Figure 12.3: Synthetic attention weights for intuition only: each row is a token being updated and each column is a possible information source; the strong "it" ← "animal" route illustrates content-dependent communication.

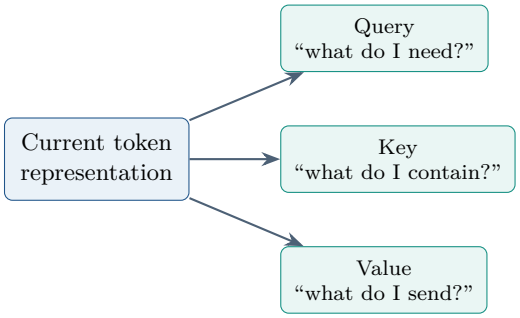


Figure 12.4: Q/K/V separates matching from message content: queries and keys choose a route; values are the information sent through it.

12.4 Queries, keys and values

Each token state is projected into three roles:

The query from token i is compared with every allowed key j through a dot product. A large dot product means the requested feature direction aligns with what that source token advertises. Softmax converts those scores into positive routing weights that sum to one.

The whole mechanism fits in one equation:

$$\underbrace{\text{Attention}(Q,K,V)}_{\text{context gathered for each token}} = \underbrace{\text{softmax}\left(\frac{\overbrace{QK^\top}^{\text{query–key compatibility}}}{\sqrt{N_{\text{key dims}}}}\right)}_{\text{who should I listen to?}} \underbrace{V}_{\text{what do they say?}}.$$

The division by $\sqrt{N_{\text{key dims}}}$ keeps typical dot-product scale controlled as the key/query dimension grows, preventing softmax from becoming excessively sharp merely because vectors are wider.

Memory Hook

Keys choose the source; values carry the message.

12.5 Causal masking: delete arrows into the future

Autoregressive training must not reveal future tokens. A causal mask therefore permits only earlier/current positions:

	key 1	key 2	key 3	key 4	key 5
query 1	✓	×	×	×	×
query 2	✓	✓	×	×	×
query 3	✓	✓	✓	×	×
query 4	✓	✓	✓	✓	×
query 5	✓	✓	✓	✓	✓

Future score entries are made effectively $-\infty$ before softmax, giving them zero routing weight. This is how training can process a whole sequence in parallel while preserving the next-token constraint.

Memory Hook

The causal mask deletes attention routes that point into the future.

12.6 Multi-head attention: several routing spaces

One attention map gives one learned query/key/value geometry. Multi-head attention learns several in parallel and then mixes their outputs:

$$\underbrace{\text{MHA}(X)}_{\text{combined communication}} = \text{Concat}(\text{head}_1,\ldots,\text{head}_{N_{\text{heads}}})\theta_{\text{output}}.$$

Do not assign a fixed human meaning to each head. The useful abstraction is simply several independent relationship subspaces that can route different information patterns.

12.7 Residual stream and normalisation

A residual sublayer learns a change instead of rebuilding the whole token state:

$$\underbrace{x_{\text{next}}}_{\text{updated state}} = \underbrace{x_{\text{old}}}_{\text{information already present}} + \underbrace{\Delta x}_{\text{attention or MLP correction}}.$$

This provides a direct information/gradient path through depth. LayerNorm or RMSNorm controls intermediate scale; their statistical axes were introduced in Section 6.3.

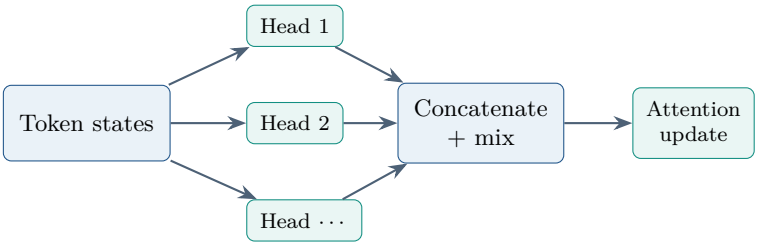


Figure 12.5: Multi-head attention runs several learned routing mechanisms in parallel before combining their messages.

12.8 Position: content alone is not order

Pure attention receives a set of token vectors and needs an additional mechanism to distinguish order/distance. Positional embeddings, relative position schemes and rotary position methods all solve the same structural problem: encode where tokens sit relative to one another. The implementation differs, but the design question is constant:

How should sequence distance/order change which interactions are easy to represent?

12.9 Why long context is expensive

Dense self-attention compares every query position with every key position. The score matrix therefore contains

$$N_{\text{tokens}}^2$$

entries per head before implementation optimisations.

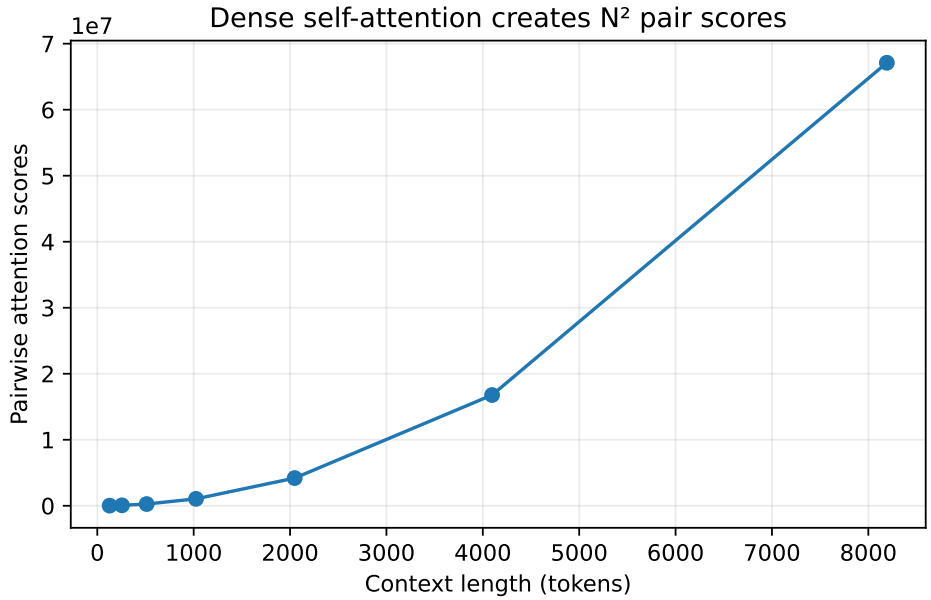


Figure 12.6: Dense attention's pair count grows quadratically with context length; doubling context produces roughly four times as many query–key scores.

This motivates local/sparse attention, grouped key/value variants, specialised kernels and alternative sequence architectures when full all-pairs interaction is unnecessary. At autoregressive inference, the KV cache changes the repeated-compute trade-off; see Section 14.5.

12.10 Research evolution: how should sequence information move?

Step	What changed	Problem targeted	New trade-off
RNN	Compress history into recurrent state	Sequence dependence	Sequential bottleneck / long-memory difficulty
Attention on RNNs	Directly retrieve useful past states	One-state bottleneck	Added pairwise matching cost
Transformer	Make attention the main communication mechanism	Parallel training and direct long-range routes	Dense N^2 interaction cost
Efficient attention / KV variants	Restrict or share interaction structure	Long-context/serving cost	Extra architectural bias
SSM branch	Compress long history into evolving state	Linear-time long sequences	Less explicit all-pairs interaction

Inductive Bias

If useful relationships between sequence elements depend on content, attention can dynamically route information; if most useful interaction is local or compressible into state, unrestricted dense attention may spend unnecessary compute.

Degree of Experiment

Vary depth, representation width, head count/dimension, attention pattern, positional mechanism, context length, MLP expansion, normalisation/residual placement and key/value sharing.

SECTION 13

Architecture: LLMs

A Transformer is an architecture. A large language model (LLM) is a Transformer-based language system trained at scale, then usually adapted and embedded inside retrieval/tooling/serving infrastructure. The useful lifecycle is

$$\boxed{\text{tokenise} \to \text{pretrain} \to \text{instruction-tune} \to \text{preference-tune} \to \text{deploy with context/tools}}.$$

13.1 Tokenisation and autoregressive pretraining

Raw text is converted into token IDs, mapped to vectors, and processed by the causal Transformer from section 12. Autoregressive pretraining learns

$$\boxed{P(\text{next token} \mid \text{previous tokens})}$$

through next-token cross-entropy over very large corpora. The same local training game rewards internal structure useful for syntax, semantics, factual associations, code patterns and longer-range discourse because those structures improve prediction.

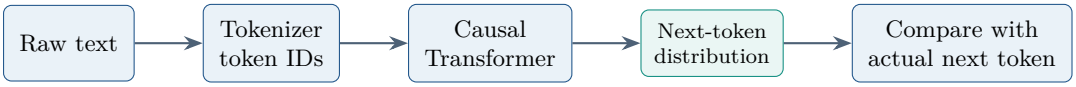


Figure 13.1: Pretraining is one repeated prediction game; the richness lies in the representation needed to play it well.

13.2 Generation: logits, probabilities and the autoregressive loop

The final token representation is mapped to one logit per vocabulary token. Softmax turns those logits into probabilities, then decoding chooses the next token. That token is appended and the process repeats:

$$\boxed{\text{prompt} \to P(\text{next token}) \to \text{choose token} \to \text{append} \to \text{repeat}}.$$

Greedy decoding always takes the highest-probability token; sampling introduces controlled randomness through choices such as temperature and top-p filtering. These are inference-time policies, not changes to the learned weights.

13.3 Supervised fine-tuning: teach interaction behaviour

A pretrained base model is primarily a continuation model. Supervised fine-tuning (SFT) changes the training distribution toward instruction–response examples:

$$\underbrace{\text{instruction / context}}_{\text{desired input format}} \longrightarrow \underbrace{\text{target response}}_{\text{desired behaviour}}.$$

The architecture and token-prediction machinery can remain largely unchanged; the model is now rewarded for producing responses that resemble the task behaviour we want.

Memory Hook

Pretraining learns broad continuation structure; SFT teaches the interaction behaviour you want.

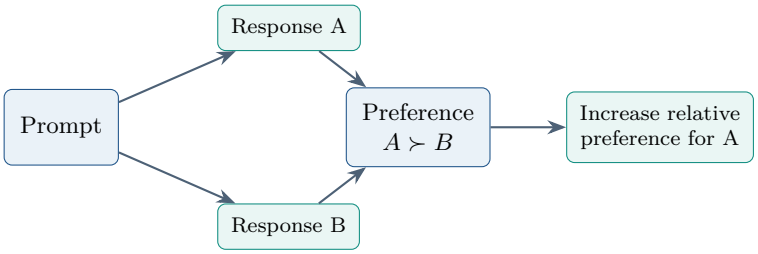


Figure 13.2: Preference tuning changes the learning signal from "match this exact answer" to "prefer this response over that one".

13.4 Preference optimisation: ranking generated responses

For many assistant behaviours there is no unique target string, but humans or evaluators can often say response A is preferable to response B. That converts alignment into the ranking structure from Section 3.4.

RLHF commonly learns a reward/preference model and then optimises the language-model policy against it. DPO-style methods target preferred-versus-rejected response probabilities more directly, reducing the amount of explicit reinforcement-learning machinery. The important mental model is the shared relative-preference signal, not the implementation brand.

13.5 Prompting, RAG and fine-tuning solve different missing pieces

Choose the intervention according to what must change.

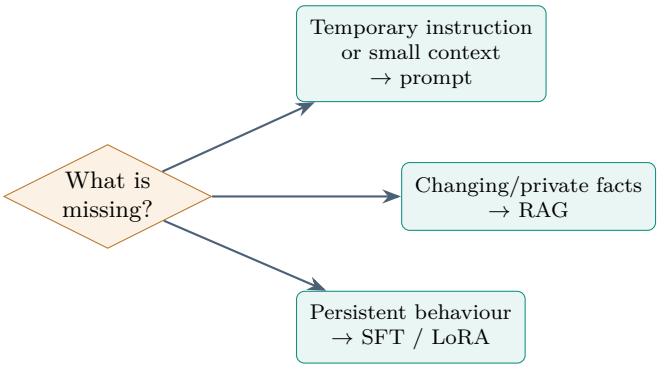


Figure 13.3: Prompting changes the current context, RAG changes accessible evidence, and fine-tuning changes persistent model behaviour.

A useful optionality axis is

$$\underbrace{\text{prompt}}_{\text{cheap/reversible}} \rightarrow \text{RAG} \rightarrow \text{LoRA} \rightarrow \text{partial fine-tuning} \rightarrow \underbrace{\text{full fine-tuning}}_{\text{more invasive}}.$$

The LoRA mechanism is already derived in Section 8.6; QLoRA adds a quantised frozen base in Section 8.6.1.

13.6 RAG: external evidence as an inference-time memory

Retrieval-augmented generation (RAG) separates external knowledge from the language-model weights. Documents are indexed offline; a query retrieves useful chunks online; those chunks become additional context for generation.

A conversational query can be rewritten into a search-specific query before embedding, which can improve retrieval when the raw wording depends on chat context. The rewrite is itself a model choice: an aggressive rewrite can also distort user intent.

13.6.1 Chunking: choose the unit that retrieval can retrieve

Very small chunks are specific but can lose surrounding context. Very large chunks preserve context but dilute the embedding signal and consume more LLM context. Overlap reduces boundary failures at the cost

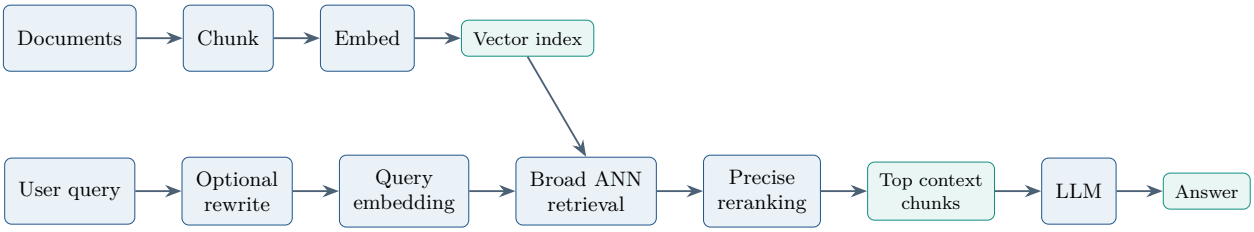


Figure 13.4: RAG separates broad cheap retrieval from expensive precise reranking before the language model sees a small evidence set.

of index duplication. Chunking is therefore a modelling choice:

$$\boxed{\text{specific retrieval unit} \longleftrightarrow \text{complete local context}}.$$

13.6.2 Bi-encoder retrieval versus cross-encoder reranking

A bi-encoder maps query and document separately:

$$\underbrace{z_{\text{query}}}_{\text{computed online}} = f_\theta(q), \qquad \underbrace{z_{\text{doc}}}_{\text{precomputed}} = g_\phi(d), \qquad \underbrace{s(q,d)}_{\text{cheap compatibility}} = z_{\text{query}}^\top z_{\text{doc}}.$$

This supports vector indexing because document vectors can be stored once. A cross-encoder instead processes query and candidate document together, allowing richer token-level interaction but requiring one expensive pass per candidate.

	Bi-encoder	**Cross-encoder**
Query/document interaction	Compressed into two vectors	Joint token-level interaction
Document preprocessing	Precompute vectors	Must score per query–document pair
Typical role	High-recall retrieval	Precision reranking
Scaling	Millions of candidates	Tens/hundreds of candidates

Memory Hook

Retrieve broadly; rerank precisely.

13.6.3 Approximate nearest neighbours: HNSW versus IVF

Exact nearest-neighbour search compares the query with every stored vector. Approximate nearest-neighbour (ANN) methods trade a small amount of retrieval exactness for much lower search work.

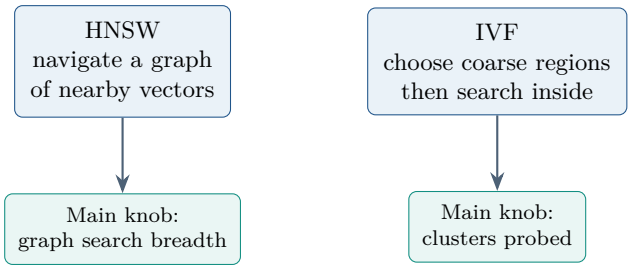


Figure 13.5: HNSW prunes search by graph navigation; IVF prunes search by coarse spatial partitioning.

HNSW usually spends extra memory on graph edges and searches by repeatedly moving toward closer neighbours. IVF first assigns vectors to coarse centroids, then searches only selected cells. The index choice should be compared on recall, latency, memory and update behaviour rather than by algorithm name alone.

Degree of Experiment

For RAG, vary chunking, embedding model/dimension, similarity, ANN index, number retrieved, reranker, number of chunks placed in context, query rewriting and generation model.

13.7 Agentic systems: an LLM inside a stateful control loop

A tool-using agent is not simply a larger model. The language model proposes the next step, an orchestrator owns state/branching/stopping, and tools perform external operations.

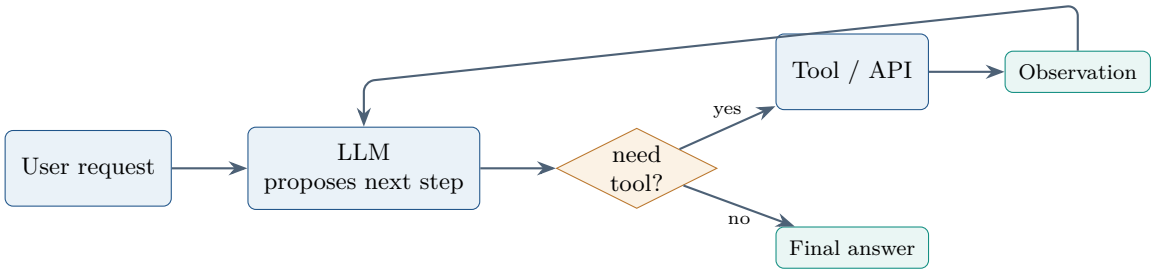


Figure 13.6: The orchestrator turns model calls into a loop; tool execution and approval policy remain outside the language model.

High-impact actions can include mandatory human approval gates. That is a stronger systems guarantee than relying on the model to remember a prose instruction. Tool protocols standardise how external capabilities are exposed; inference engines independently optimise how model calls are served.

Memory Hook

An agent is an LLM inside a stateful control loop with tools.

13.8 LLM research optionality

Layer	Knobs	What changes
Base model	size, context, architecture	raw modelling capacity / cost
Adaptation	prompt, SFT, LoRA, full FT	persistent versus temporary behaviour
Retrieval	chunking, embedding, ANN, reranking	external evidence quality / latency
Context assembly	number/order of chunks	evidence coverage versus context cost
Decoding	temperature, top-p, stopping	output diversity and latency
Tools	available actions / permissions	external capability and risk
Orchestration	state, loops, approval gates	control flow and reliability

SECTION 14

Inference and Model Compression

Training asks how to learn a useful function. Inference asks how to execute that learned function repeatedly under latency, throughput, memory and cost constraints. The central design tension is

$$\boxed{\text{quality} \longleftrightarrow \text{latency} \longleftrightarrow \text{throughput} \longleftrightarrow \text{memory} \longleftrightarrow \text{cost}}.$$

14.1 Quantisation: spend fewer bits on tolerant values

Quantisation maps continuous values onto a finite set of representable levels. One useful scalar picture is

$$\underbrace{q}_{\text{stored integer}} = \text{round}\left(\frac{\overbrace{x}^{\text{full-precision value}}}{\underbrace{s}_{\text{quantisation scale}}}\right), \qquad \underbrace{\hat{x}}_{\text{reconstructed value}} = sq.$$

The approximation saves memory and often bandwidth/compute on suitable hardware, while introducing numeric error.

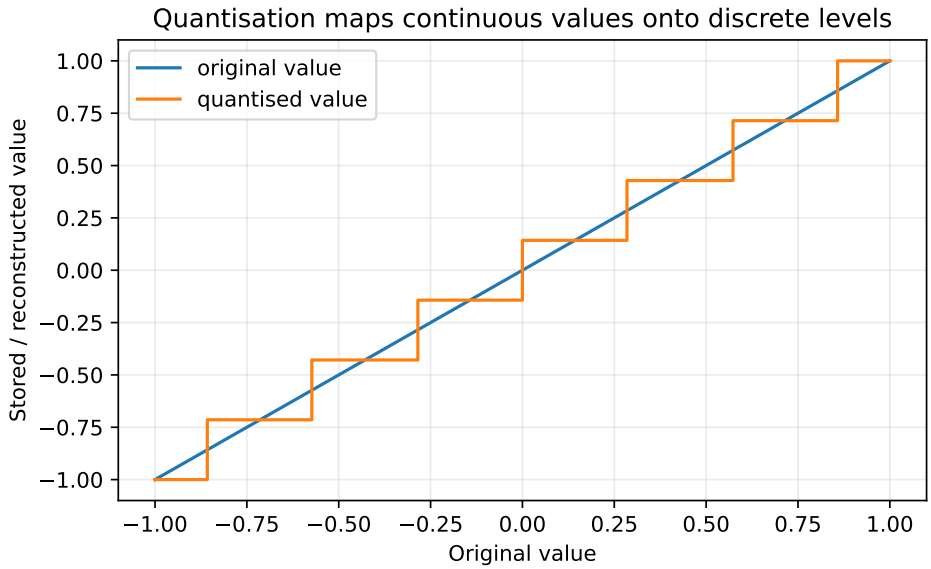


Figure 14.1: Quantisation replaces continuous values with nearby discrete levels; fewer levels increase approximation error but reduce representation cost.

14.1.1 PTQ versus QAT

Post-training quantisation (PTQ) trains normally and quantises afterwards. It is cheap and simple, but the trained model never adapted to the low-precision distortion. **Quantisation-aware training (QAT)** simulates quantisation during training so the parameters can move toward a solution that survives deployment precision.

Activation-aware methods such as AWQ allocate extra care to weight directions whose quantisation error is more consequential for observed activations. The broader principle is to spend precision where the function is sensitive instead of treating every parameter equally.

Memory Hook

PTQ tests whether the model survives low precision; QAT teaches it to survive.

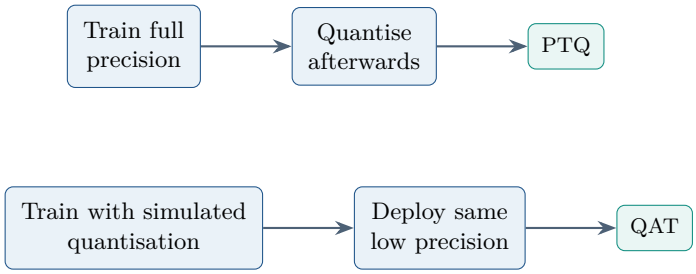


Figure 14.2: PTQ tests whether a trained model survives quantisation; QAT exposes the model to that distortion while it can still adapt.

Degree of Experiment

Vary bit width, weight-only versus weight+activation quantisation, per-channel/group scaling, calibration data and which sensitive layers remain at higher precision.

14.2 Knowledge distillation: learn from a teacher distribution

A hard label says only which class won. A teacher distribution can also reveal which alternatives are plausible:

$$[0.70,\ 0.20,\ 0.08,\ 0.02] \quad \text{contains more relative structure than} \quad [1,\ 0,\ 0,\ 0].$$

A smaller student can therefore optimise both the ground-truth objective and a teacher-matching term:

$$\underbrace{\mathcal{L}_{\text{student}}}_{\text{compact model objective}} = \underbrace{\lambda \mathcal{L}_{\text{labels}}}_{\text{task truth}} + \underbrace{(1-\lambda) D_{\text{KL}}(P_{\text{teacher}} \| P_{\text{student}})}_{\text{match teacher structure}}.$$

The student trades capacity for cheaper inference; the teacher's uncertainty/mistakes become part of the supervision.

Inductive Bias

If the teacher's probability structure captures useful similarities between outputs, matching it can transfer more information than hard labels alone; teacher bias is transferred too.

14.3 Cascades: reserve expensive models for hard cases

If most inputs are easy, running the most expensive model on every request wastes compute. A cascade routes easy/confident cases through a cheap model and escalates only uncertain or high-risk cases.

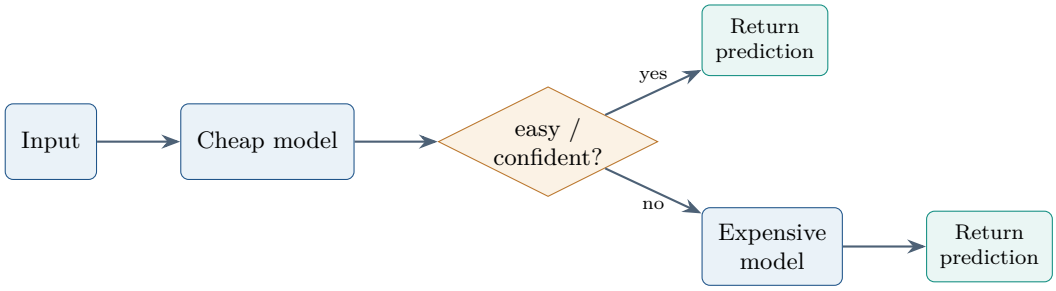


Figure 14.3: A cascade turns model difficulty into a routing problem.

Expected compute becomes

$$\underbrace{\mathbb{E}[C]}_{\text{average serving cost}} = \underbrace{C_{\text{cheap}}}_{\text{paid every request}} + \underbrace{P(\text{escalate}) C_{\text{expensive}}}_{\text{paid only on routed cases}}.$$

The gate is therefore as important as the component models.

14.4 LLM inference: prefill versus decode

Autoregressive LLM serving contains two different phases.

Prefill processes the already-known prompt and creates the attention state for its tokens. Prompt tokens can be processed with substantial parallelism because they are all available. **Decode** generates one new token at a time because token $t+1$ cannot be chosen before token t exists.

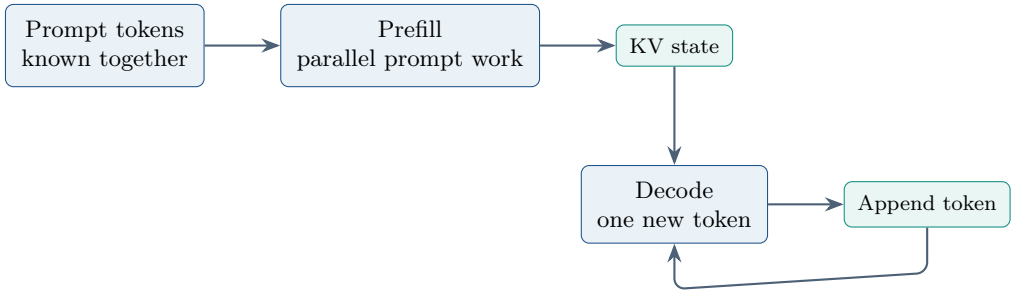


Figure 14.4: Prefill builds state from a known prompt; decode repeatedly extends the sequence one token at a time.

14.5 KV cache: save repeated attention work

During autoregressive decode, old tokens keep the same key/value representations. Recomputing them for every new token is wasteful, so serving systems cache

$$(K_1, V_1), \ldots, (K_{N_{\text{context}}}, V_{N_{\text{context}}}).$$

The next token computes only its new query/key/value and reads the cached history. This exchanges repeated compute for memory:

$$\boxed{\text{KV cache memory} \propto N_{\text{layers}} \times N_{\text{context tokens}} \times N_{\text{KV dims}} \times \text{bytes/value}}.$$

Long-context inference therefore becomes a memory-management problem even when model weights fit comfortably.

Memory Hook

KV cache remembers old attention keys/values so decode does not recompute the past.

14.6 Batching and continuous batching

Static batching is efficient when requests have similar shapes. Autoregressive requests finish at different output lengths, so a static batch can leave accelerator slots idle. Continuous batching admits new requests as old requests finish.

Static	A: ==========	B: ===	
Continuous	A: ==========	B: ===	C starts when B finishes

Paged KV-memory schemes similarly try to avoid reserving large contiguous cache blocks for every request. The precise serving engine changes, but the systems goal is stable: keep expensive accelerator memory and compute occupied by useful work.

14.7 Inference evolution and design axes

Step	What changed	Problem targeted	Trade-off
Full precision	Baseline execution	Maximum numeric fidelity	High memory/bandwidth
PTQ / QAT	Lower bit-width	Serving memory/throughput	Quantisation error / extra training
Distillation	Smaller student	Model size/latency	Student capacity limit
Cascade	Conditional model use	Waste on easy cases	Gate errors / system complexity
KV caching	Reuse old attention state	Repeated decode compute	Cache memory
Continuous batching	Dynamic request scheduling	Accelerator under-utilisation	More complex serving scheduler

Degree of Experiment

Vary numeric precision, model size, cache representation, batch/scheduling policy, cascade gate, distillation objective and context budget.

SECTION 15

ML Systems and Production

A trained model file is one component inside a data, serving and feedback loop. Production quality therefore depends on data freshness, feature consistency, serving reliability, monitoring and retraining as well as the predictor itself.

15.1 The full production loop

The useful system picture is

$$\boxed{\text{events} \to \text{features} \to \text{prediction} \to \text{outcome/logs} \to \text{future training data}}.$$

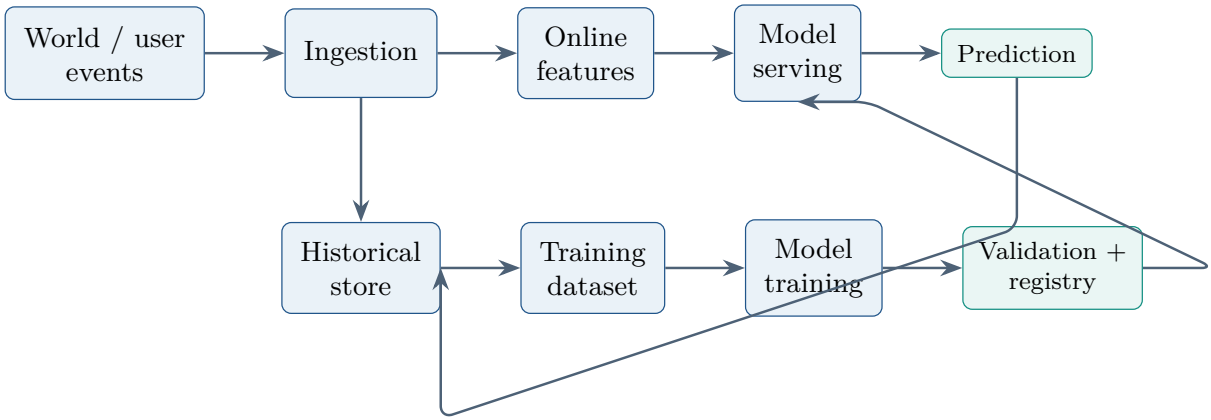


Figure 15.1: A production ML system closes the loop from observations to predictions and back into future training data.

15.2 Batch versus streaming

Batch processing waits for a collection of events and processes them together. Streaming updates state as events arrive. The core trade-off is

$$\underbrace{\text{freshness}}_{\text{streaming advantage}} \quad \longleftrightarrow \quad \underbrace{\text{operational simplicity}}_{\text{batch advantage}}.$$

If a feature changes slowly relative to the decision cadence, streaming adds complexity without much value. If session intent becomes useful within seconds, waiting for tomorrow's batch can make the feature operationally irrelevant.

Practical Tip

Ask how quickly new information changes the optimal decision before deciding that a feature needs a streaming path.

15.3 Training-serving consistency

A model is validated on training-time feature semantics. If serving computes the "same" named feature differently, deployment receives a distribution the model was never validated on. A feature store or shared transformation layer mainly helps by keeping

$$\boxed{\text{training definition} \approx \text{serving definition}}$$

for code, windows, units and missing-value behaviour. The same leakage discipline from Section 7.8 applies to historical feature construction.

15.4 Model registry, deployment and rollback

A deployed prediction should be traceable to the model version, training data/version, feature schema, code/configuration and evaluation report that produced it. A registry makes that state explicit and supports rollback when the new version fails after deployment. The practical unit is therefore not "the latest weights" but a versioned model artifact plus its reproducible context.

15.5 Monitoring as a stack

One "model health" metric cannot diagnose infrastructure, data and product failures at once.

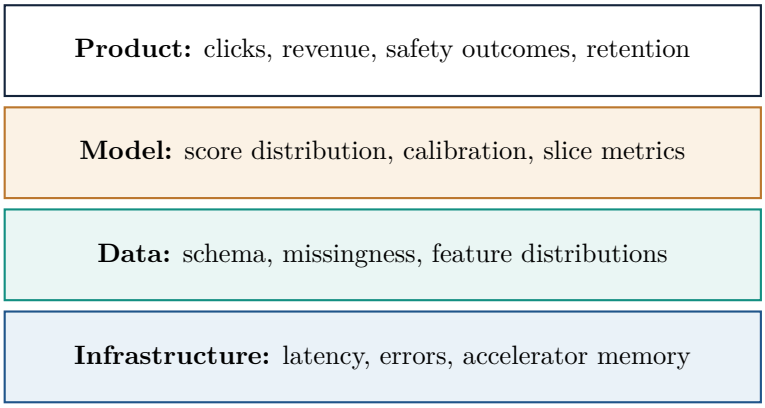


Figure 15.2: Monitoring layers answer different questions; a healthy server can host a failing model and a good model can still time out.

15.6 Drift and retraining

Covariate drift changes $P(x)$; concept drift changes $P(y \mid x)$. A moved feature histogram does not prove model quality fell, and model quality can fall without an obvious one-dimensional drift signal. Retraining should therefore follow diagnosis rather than act as an automatic cure for every drift alert.

Common policies are:

fixed schedule event-triggered retraining frequent incremental updates.

More frequent updates improve freshness but increase reproducibility, stability and feedback-loop complexity.

15.7 Queues, caching and fallbacks

Production systems must specify what happens when the preferred model is slow or unavailable. A fallback can be a cached result, a smaller model, a heuristic ranking or a safe default. Slightly weaker predictions can be operationally preferable to request failure.

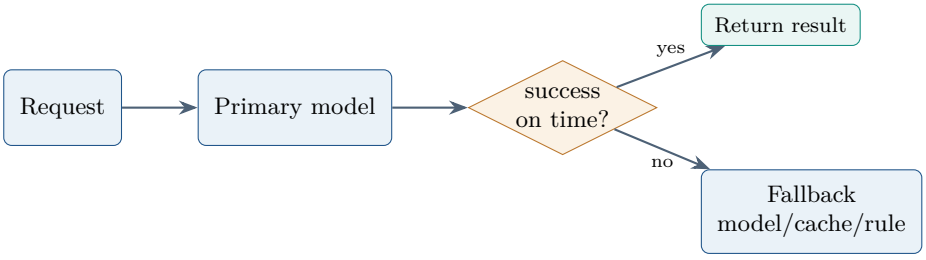


Figure 15.3: Failure handling is part of the prediction policy, not an afterthought.

15.8 Build versus buy

API serving and self-hosting have different fixed/marginal cost structures:

$$\underbrace{C_{\text{lifetime}}}_{\text{total}} = \underbrace{C_{\text{fixed engineering + infrastructure}}}_{\text{larger for self-hosting}} + \underbrace{N_{\text{requests}} C_{\text{request}}}_{\text{often larger for API use}} .$$

The break-even volume is not universal because utilisation, token lengths, hardware, staffing, privacy and model choice change both terms.

Dimension	API	Self-host
Launch speed	Usually high	Infrastructure first
Operations burden	Provider owns more	Team owns serving stack
Up-front cost	Lower	Higher
Model/serving control	Limited by provider	High
Marginal economics	Provider pricing	Depends on utilisation/hardware
Privacy/control	Provider-dependent	Stronger direct control possible

15.9 Training versus lifetime inference economics

Training is paid a small number of times; inference can be paid for every user request and every generated token. Lifecycle cost is therefore

$$\boxed{C_{\text{lifetime}} = C_{\text{training}} + N_{\text{inferences}} C_{\text{inference}}.}$$

Extra training compute can be rational if it produces a smaller model that reaches the required quality and is then served at enormous volume. The interview-guide heuristic cites Chinchilla-style compute-optimal training at roughly 20 training tokens per parameter as a historical reference point. Treat that ratio as a result for a particular modelling/compute regime rather than a universal modern constant. Do not turn a historical scaling-law ratio into a universal constant; the useful principle is the one-off training versus repeated-serving trade-off.

Memory Hook

Training is paid occasionally; serving is paid every request.

Degree of Experiment

Vary batch versus streaming paths, feature freshness, serving model, caching, fallback policy, deployment cadence, retraining trigger, observability thresholds and build-versus-buy boundary.

SECTION 16

Use Case: Real-Time Moderation

Moderation combines representation, imbalanced classification, thresholds, cascades and human escalation. The model is therefore one component inside a risk-routing policy.

16.1 The moderation system is a cascade

A useful production architecture routes obvious cases cheaply and reserves expensive reasoning for ambiguous or high-impact cases.

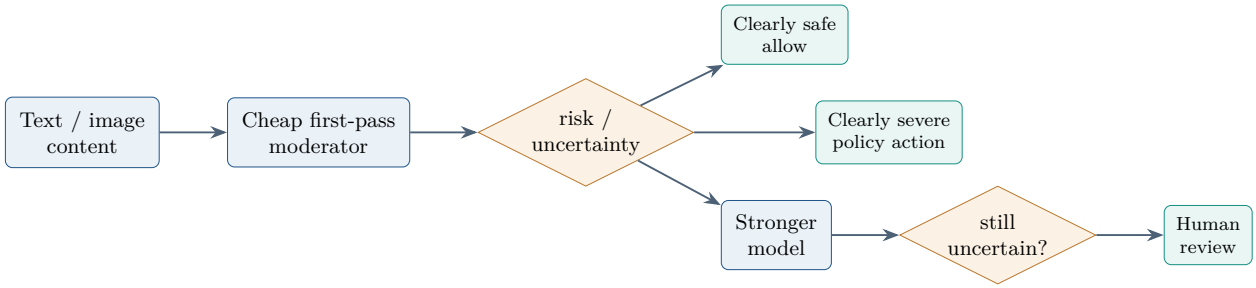


Figure 16.1: Moderation is a routing system: thresholding and escalation policy determine which model or human sees each case.

The cascade idea is the same one developed in Section 14.3. The difference is the cost function: moderation can make false negatives and false positives operationally asymmetric.

16.2 Text and image representations

Text moderation can use a pretrained Transformer encoder followed by a sequence-level classification head. Image moderation can use a language-aligned visual representation such as a CLIP-like encoder or a self-supervised visual backbone such as a DINO-like encoder (Section 11.6). The practical question is not which name is newer, but whether the pretraining signal makes the harmful/benign distinction easy to express with limited target labels.

For multimodal systems, keep the representation question separate from the final decision policy:

$$\text{text evidence} + \text{image evidence} + \text{context} \to \text{risk score} \to \text{threshold/escalation}.$$

16.3 Class imbalance: accuracy can be meaningless

If 99% of examples are negative, an always-negative classifier obtains 99% accuracy while detecting nothing. This is why the rare-event metric discussion in Sections 7.2 and 7.4 matters directly.

16.3.1 Class weighting versus focal loss

Weighted cross-entropy changes the cost by class:

$$\underbrace{\mathcal{L}_{\text{weighted}}}_{\text{asymmetric class cost}} = -\underbrace{w_+ y \log p}_{\text{positive errors}} - \underbrace{w_-(1-y)\log(1-p)}_{\text{negative errors}}.$$

Focal loss instead downweights examples that are already easy:

$$\underbrace{\mathcal{L}_{\text{focal}}}_{\text{focus on hard examples}} = -\underbrace{(1-p_{\text{correct}})^{\gamma}}_{\text{near zero when already easy}} \log p_{\text{correct}}.$$

The mechanisms are different:

class weighting: some classes matter more	focal loss: easy examples matter less.

Synthetic oversampling such as SMOTE can be useful in some low-dimensional tabular settings but does not automatically create realistic rare examples in complex text/image representation spaces.

Memory Hook

Class weighting changes class cost; focal loss reduces the influence of easy examples.

16.4 Thresholds encode the product cost

A score becomes an action only after a threshold or routing rule. Lower thresholds flag more content and typically increase recall while creating more false alarms. Higher thresholds reduce interventions but miss more positives. The correct operating point therefore depends on

$$C_{\text{miss}}, \quad C_{\text{false alarm}}, \quad C_{\text{human review}}.$$

This is exactly the deployment-threshold separation from Section 7.3.

Practical Tip

Report the precision–recall operating curve and choose thresholds from the real consequences of misses, interventions and review capacity rather than from accuracy.

16.5 Human review is an expensive model tier

Human reviewers are flexible but slow and costly. A good routing policy therefore concentrates them on cases with high uncertainty, disagreement or impact rather than sampling only the easiest high-confidence examples. Reviewer decisions can also become future labels, creating the same closed-loop data concerns developed for personalisation in Section 18.6.

16.6 Degrees of experiment

Degree of Experiment

Vary text/image/multimodal representation, loss weighting, focal parameter γ, sampling policy, thresholds, cascade gate, temporal/context history, human-routing policy and which uncertain cases are labelled for retraining.

Memory Hook

Moderation quality is the model + threshold + escalation policy.

SECTION 17

Use Case: Ranking Systems

Ranking at scale is a computational funnel. Early stages protect recall across huge candidate sets; later stages spend more compute on precise ordering.

17.1 The ranking funnel

A representative system shrinks millions of possible items before expensive scoring:

$$\boxed{\text{millions} \to \text{retrieval} \to \text{thousands} \to \text{light ranker} \to \text{hundreds} \to \text{heavy reranker} \to \text{final list}}.$$

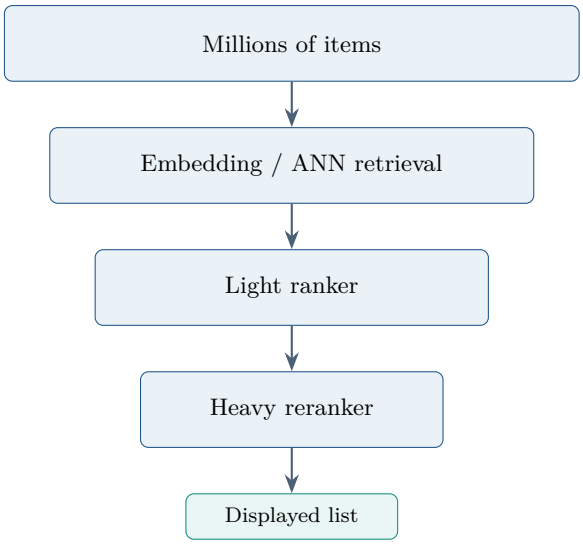


Figure 17.1: Ranking systems spend progressively more modelling capacity on progressively fewer candidates.

The same broad design appears in RAG (Section 13.6): cheap high-recall retrieval first, expensive precision later.

17.2 Retrieval with two towers

A two-tower model computes user/query and item representations independently:

$$\underbrace{z_{\text{user}}}_{\text{online}} = f_\theta(\text{user, context}), \qquad \underbrace{z_{\text{item}}}_{\text{precomputable}} = g_\phi(\text{item}),$$

then scores compatibility through

$$\underbrace{s_{\text{retrieve}}}_{\text{ANN-compatible score}} = z_{\text{user}}^\top z_{\text{item}}.$$

Because item vectors can be precomputed, nearest-neighbour search can reduce millions of items to a candidate set. The embedding/search geometry is developed in Sections 2.8 and 13.6.3.

Inductive Bias

If user–item compatibility can be usefully compressed into separate embeddings, two-tower retrieval scales to large catalogues; interactions that require joint fine-grained features are postponed to later rankers.

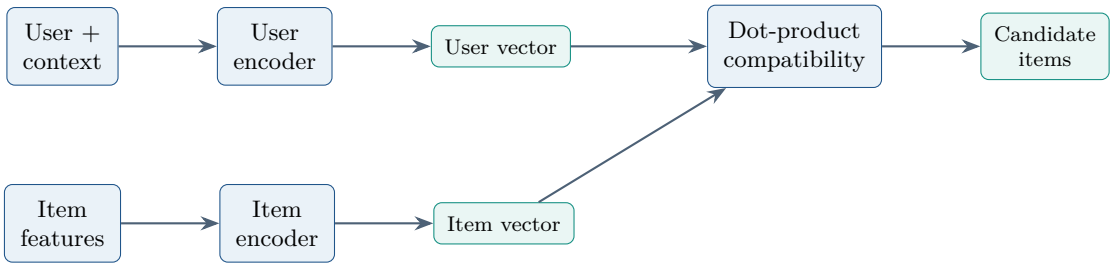


Figure 17.2: Two-tower retrieval buys scalability by forcing each side to be compressed before user–item interaction.

17.3 Ranking and reranking

Once the candidate set is small enough, a ranker can use richer cross-features:

$$\underbrace{s_i}_{\text{ranking score}} = f(\text{user}, \text{item}_i, \text{context}, \text{cross-features}).$$

Pointwise, pairwise and listwise objectives were introduced in Section 3.4. A heavy reranker can additionally compare richer text, history or candidate interactions because it now evaluates tens/hundreds rather than millions.

17.4 Why NDCG is top-heavy

NDCG gives each rank a discount approximately proportional to

$$\frac{1}{\log_2(r+1)}.$$

That is why an ordering error near the top can deserve a much stronger training signal than one near rank

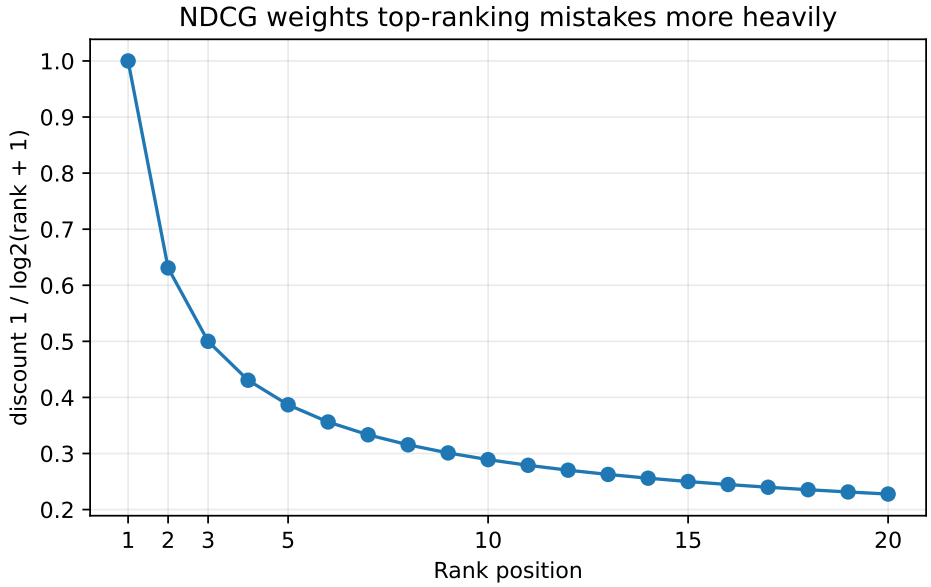


Figure 17.3: The NDCG discount falls fastest at the top, so moving a relevant item between ranks 1 and 2 matters much more than a similar swap deep in the list.

100. LambdaMART uses this idea while retaining boosted-tree stage-wise fitting; see Section 10.4.2.

17.5 Position bias: clicks mix relevance with exposure

A top-ranked item gets more opportunity to be examined than a low-ranked item. Observed clicks therefore combine at least two effects:

$$P(\text{click}) \approx \underbrace{P(\text{examined} \mid \text{position})}_{\text{exposure}} \times \underbrace{P(\text{click} \mid \text{examined, relevance})}_{\text{preference}}.$$

Training naively on clicks can make "was shown high" look like "was relevant".

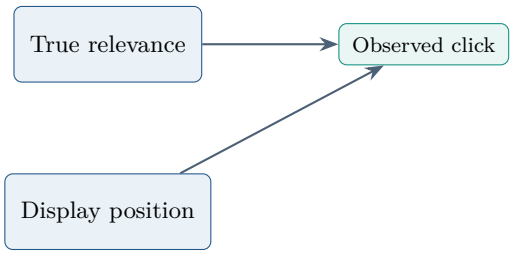


Figure 17.4: Click labels are confounded by both user preference and the system's exposure policy.

17.6 Inverse propensity scoring

If the logging policy exposes an item with probability p_i, inverse propensity scoring (IPS) weights its observed contribution by roughly

$$w_i = \frac{1}{p_i}.$$

Rarely exposed observations count more because they represent a larger set of opportunities that were usually not shown. The correction can have high variance:

$$p_i \to 0 \quad \Rightarrow \quad w_i \to \infty.$$

Practical systems therefore need credible propensity estimates plus clipping/regularisation or explicit exploration where appropriate.

Memory Hook

IPS makes rare exposures count more; tiny propensities can make variance explode.

17.7 Evaluation: diagnose the funnel stage

Use Recall@K for candidate generation, NDCG/MRR for ordering, and online experiments for product outcomes. These metrics are defined in Sections 7.9 and 7.11. Offline ranking gains do not guarantee online user-value gains because historical labels were generated under an earlier exposure policy.

Interleaving can combine results from two rankers into one list for a sensitive direct comparison under the same user/context. A/B tests remain necessary when the question is broader product impact rather than relative list preference.

17.8 Degrees of experiment

Degree of Experiment

Vary candidate sources, user/item representation, embedding dimension, ANN index, candidate count, ranker family, pointwise/pairwise/listwise objective, bias correction, reranker capacity, exploration policy and stage-specific metrics.

Memory Hook

Retrieve for recall; rank for order; rerank for precision.

SECTION 18

Use Case: Personalisation

Ranking asks which item should come first. Personalisation makes the score depend on who the user is, what the item is, the current context and recent/long-term history:

$$\boxed{s = f(\text{user}, \text{item}, \text{context}, \text{history}).}$$

18.1 From interaction matrix to latent preference space

Collaborative filtering begins from a sparse user–item interaction matrix. Matrix factorisation approximates it with low-dimensional user and item vectors:

$$\underbrace{R}_{N_{\text{users}} \times N_{\text{items}}} \approx \underbrace{U}_{N_{\text{users}} \times N_{\text{latent dims}}} \underbrace{V^\top}_{N_{\text{latent dims}} \times N_{\text{items}}}, \qquad \underbrace{\hat{r}_{u,i}}_{\text{predicted preference}} = z_u^\top z_i.$$

The low-rank prerequisite is in Section 1.2.

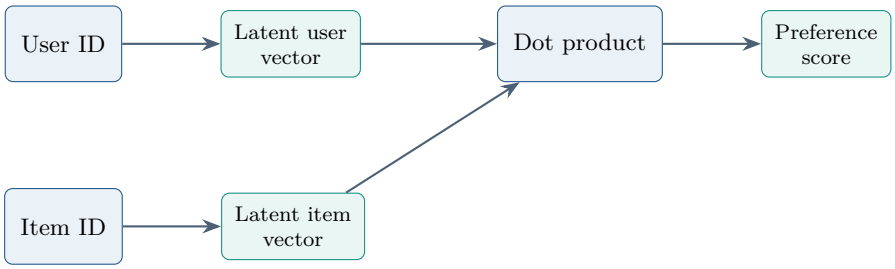


Figure 18.1: Matrix factorisation compresses sparse user–item behaviour into a shared latent geometry.

Inductive Bias

If user–item preferences have low-dimensional latent structure, matrix factorisation shares statistical strength across sparse interactions; idiosyncratic preferences outside that structure are harder to represent.

18.2 Two towers generalise stored latent vectors

A stored user/item vector cannot naturally explain a new identity with no interaction history. Two-tower models replace lookup-only representations with learned functions:

$$z_{\text{user}} = f_\theta(\text{history, session, context}), \qquad z_{\text{item}} = g_\phi(\text{text, image, metadata}).$$

This connects personalisation directly to the retrieval mechanism in Section 17.2. Content features let new items acquire a useful representation before many collaborative interactions exist.

18.3 Cold start: combine collaborative and content evidence

New users/items have little collaborative signal. Useful fallbacks include content encoders, popularity priors, onboarding choices and rapidly updated session features. The system therefore often moves from

$$\text{collaborative only} \quad \rightarrow \quad \boxed{\text{collaborative + content + context}}.$$

The exact mixture can change as interaction history accumulates.

18.4 Long-term preference versus session intent

A user can have stable months-long interests while exhibiting a very different current-session goal. One representation can therefore combine multiple timescales:

$$\underbrace{z_{\text{user now}}}_{\text{current preference state}} = f\Big(\underbrace{z_{\text{long-term}}}_{\text{persistent preference}}, \underbrace{z_{\text{session}}}_{\text{current intent}}, \underbrace{\text{context}}_{\text{time/device/location-like context}}\Big).$$

Streaming is valuable when the session state changes faster than the model parameters need to be retrained.

18.5 Temporal decay: encode how quickly preference becomes stale

A simple recency rule weights old interactions less:

$$\underbrace{w(\Delta t)}_{\text{interaction weight}} = 2^{-\Delta t/\underbrace{T_{1/2}}_{\text{behaviour half-life}}}.$$

A small $T_{1/2}$ forgets quickly; a large one gives long-term history more influence. The correct decay rate is

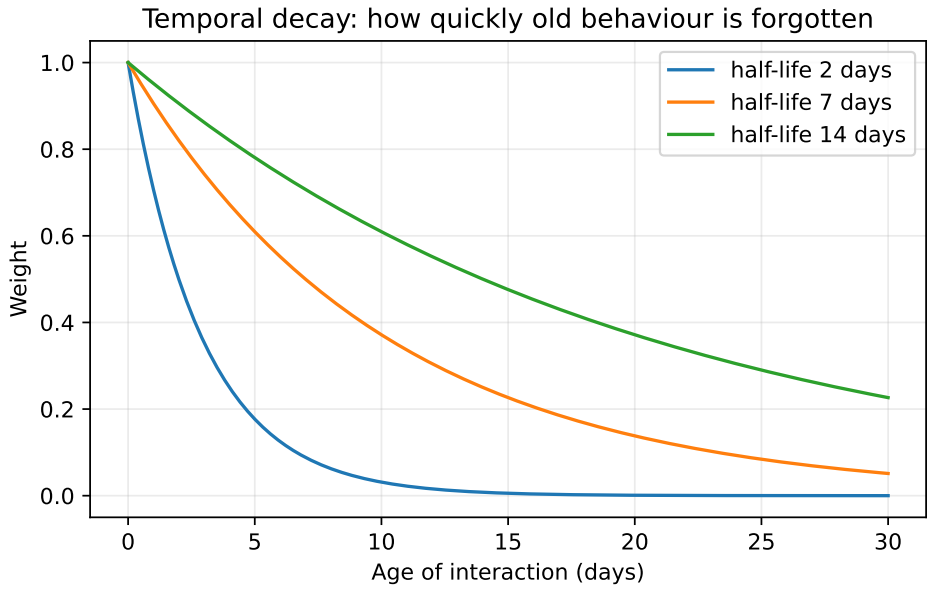


Figure 18.2: Different half-lives encode different assumptions about how quickly past behaviour stops predicting current intent.

domain-dependent rather than a universal recommendation.

Inductive Bias

If recent behaviour predicts current intent better than old behaviour, temporal decay is useful; stable long-term preferences can make aggressive decay throw away signal.

18.6 The recommendation feedback loop

A recommender changes what the user sees, and those exposures create the interaction data used to train its successor. The data distribution is therefore partly policy-generated:

$$P(\text{future training data}) = P(\text{observations} \mid \text{deployed recommender}).$$

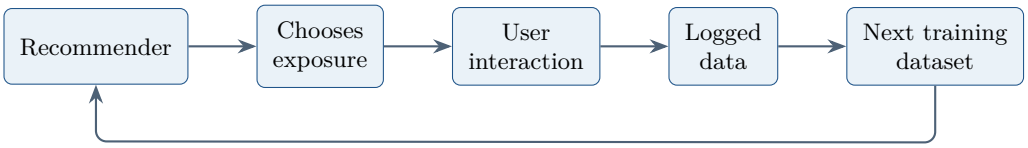


Figure 18.3: Personalisation is a closed-loop system: current recommendations help generate future labels.

18.7 Popularity bias and self-reinforcement

Popular items are exposed more, receive more interactions, become easier to estimate, and can then be scored even higher:

$$\text{popularity} \to \text{exposure} \to \text{interactions} \to \text{training signal} \to \text{future exposure}.$$

This does not prove popularity is undesirable; it means observed popularity partly reflects the system that generated exposure. Position-bias correction in Sections 17.5 and 17.6 addresses one part of this general feedback problem.

18.8 Exploration versus exploitation

Always selecting the current highest score exploits existing knowledge but can prevent the system from learning about underexposed alternatives. Exploration deliberately spends some immediate reward to gain information:

$$\underbrace{\text{exploit}}_{\text{best current estimate}} \longleftrightarrow \underbrace{\text{explore}}_{\text{learn uncertain alternatives}}.$$

Without exploration, "never shown" can become "never observed" and then "never learned to be good". Bandit methods formalise this trade-off when actions influence which rewards are observed.

18.9 Fast state updates versus slow parameter updates

A production recommender often has two update loops:

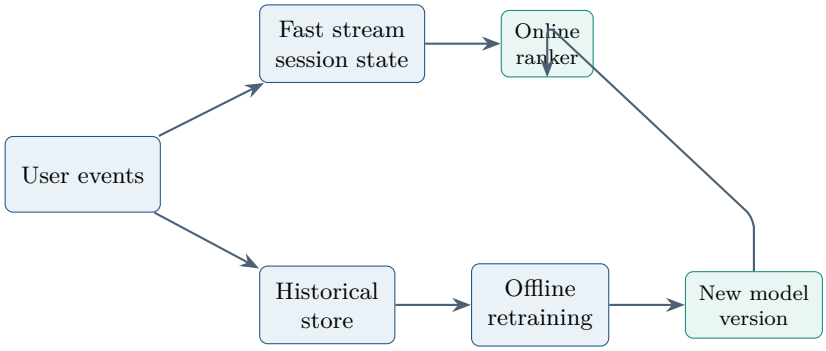


Figure 18.4: Fast features capture current intent; slower retraining changes the underlying model parameters.

A metric drop or drift alert should trigger diagnosis before retraining because data-pipeline failures, inventory changes, delayed labels and real preference shifts need different fixes. The production-system monitoring stack in Sections 15.5 and 15.6 provides the broader framework.

18.10 Degrees of experiment

Degree of Experiment

Vary user representation, item representation, long-term/session weighting, history window, temporal decay, candidate generation, ranking objective, exploration policy, feature freshness, propensity correction and retraining cadence/trigger.

Memory Hook

The recommender changes the data that trains the next recommender.